```
// ============================================================
// didiQCsys v9.12 — Backend Code.gs BAGIAN 1
// UPDATED v9.12 (Round 2):
// - Tambah validasi masal pada menu Validasi QC (ceklis + catatan masal)
// - Perbaikan OPSpecs: CV di-plot dari CV Terhitung/Observasi (bukan CV Lot/Manufaktur)
// - Updated from v9.9 features
// - Multi-password per akun (hanya superadmin yang kelola)
// - Tombol Reset & Tampilkan pada Input QC
// - Perbaikan posisi tombol form Users
// - v9.12: Auto-backup trigger otomatis saat simpan pengaturan
// - v9.12: Kolom Nama pada password tambahan + tampil di hero saat login
// - v9.12 R2: Edit & status aksi pada Kelola Password Tambahan
// - v9.12 R2: Fix username di kolom 'oleh' Histori QC saat login password tambahan
// - v9.12 R2: Brightness 5 level deskriptif (Sangat Gelap s.d. Cerah Sekali)
// - v9.12 R2: Login settings (font/brightness) persisten sebelum login via getLoginSettings()
// ============================================================

var APP_NAME = 'didiQCsys v9.12';
var APP_LOGO_URL = 'https://drive.google.com/uc?id=1pNQRaZmXb-BnsXEvJ3CG36eCRi_Kx3yH';
var SS_ID = PropertiesService.getScriptProperties().getProperty('SS_ID');
var SS = null;
var SMART_PWD = 'didikqc';
var LOCK_TIMEOUT = 30000;
var BACKUP_FOLDER_NAME = 'didiQCsys_Backups';
var MAX_BACKUP_FILES = 10;

function getSS() {
  if (!SS) {
    if (SS_ID) { SS = SpreadsheetApp.openById(SS_ID); }
    else { SS = SpreadsheetApp.getActiveSpreadsheet();
PropertiesService.getScriptProperties().setProperty('SS_ID', SS.getId()); }
  }
  return SS;
}

var SH = {
  USERS:'Users', PARAMS:'Parameters', LOTQC:'LotQC',
  INPUTQC:'InputQC', HISTORI:'HistoriQC', CALCSTATS:'CalculatedStats',
  BIASPME:'BiasPME', DAFTARTEA:'DaftarTEa', SIGMACVOPT:'SigmaCVOpt',
  LAPORAN:'LaporanCatatan', TABULASI:'TabulasiCatatan',
  KOPSURAT:'KopSurat', SETTINGS:'Settings', LOGACTIVITY:'LogActivity',
  IMGHEMATO:'ImgHemato', IMGURIN:'ImgUrin', IMGMALARIA:'ImgMalaria',
  IMGBTA:'ImgBTA', IMGLAIN:'ImgLain', IMGPATOLOGI:'ImgPatologi',
CATATANDOKTER:'CatatanDokter', USERPASSWORDS:'UserPasswords'
};
```

```
function doGet(e) {
  return HtmlService.createHtmlOutputFromFile('index')
    .setTitle(APP_NAME)
    .setXFrameOptionsMode(HtmlService.XFrameOptionsMode.ALLOWALL)
    .addMetaTag('viewport','width=device-width, initial-scale=1.0');
}

function withLock(fn) {
  var lock = LockService.getScriptLock();
  try { lock.waitLock(LOCK_TIMEOUT); return fn(); }
  finally { try { lock.releaseLock(); } catch(e){} }
}

function fD(d) {
  if(!d) return '';
  try { var dt=(d instanceof Date)?d:new Date(d); if(isNaN(dt.getTime()))return String(d);
    return
String(dt.getDate()).padStart(2,'0')+'/'+String(dt.getMonth()+1).padStart(2,'0')+'/'+dt.getFullYear();
  } catch(e){return String(d);}
}
function fDT(d) {
  if(!d) return '';
  try { var dt=(d instanceof Date)?d:new Date(d); if(isNaN(dt.getTime()))return String(d);
    return fD(dt)+' '+String(dt.getHours()).padStart(2,'0')+':'+String(dt.getMinutes()).padStart(2,'0');
  } catch(e){return String(d);}
}
function tN(){return new Date();}
function parseNumSafe(v) {
  if(v===null||v===undefined||v==='')return null;
  var n=parseFloat(String(v).replace(',','.'));
  return isNaN(n)?null:n;
}
function S(sheetName){return getSS().getSheetByName(sheetName);}
function gSD(sheetName) {
  try {
  var sh=S(sheetName);
  if(!sh||sh.getLastRow()<2) return [];
  return sh.getRange(2,1,sh.getLastRow()-1,sh.getLastColumn()).getValues();
  } catch(e){return [];}
}
function logA(username,action,detail,overrideUser) {
  try {
    var sh=S(SH.LOGACTIVITY); if(!sh) return;
    sh.appendRow([new Date(),overrideUser||username||'system',action||'',detail||'']);
```

```
    var lr=sh.getLastRow(); if(lr>2001)sh.deleteRows(2,lr-2001);
  } catch(e){}
}
function genID(prefix){return prefix+'_'+new
Date().getTime()+'_'+Math.floor(Math.random()*9999);}
function nowISO(){return new Date().toISOString();}
function parseDateStr(s) {
  if(!s) return null; if(s instanceof Date) return s;
  if(String(s).indexOf('-')>-1){var p=String(s).split('-');if(p.length===3&&p[0].length===4)return new
Date(parseInt(p[0]),parseInt(p[1])-1,parseInt(p[2]));}
  if(String(s).indexOf('/')>-1){var p2=String(s).split('/');if(p2.length===3)return new
Date(parseInt(p2[2]),parseInt(p2[1])-1,parseInt(p2[0]));}
  var d=new Date(s); return isNaN(d.getTime())?null:d;
}
function dateToISO(d) {
  if(!d) return ''; var dt=(d instanceof Date)?d:new Date(d);
  if(isNaN(dt.getTime()))return '';
  return dt.getFullYear()+'-'+String(dt.getMonth()+1).padStart(2,'0')+'-
'+String(dt.getDate()).padStart(2,'0');
}
function sanitizeReturn(obj) {
  if (obj === null || obj === undefined) return obj;
  if (typeof obj === 'string' || typeof obj === 'number' || typeof obj === 'boolean') return obj;
  if (obj instanceof Date) return dateToISO(obj);
  if (Array.isArray(obj)) return obj.map(function(item) { return sanitizeReturn(item); });
  if (typeof obj === 'object') {
    var clean = {};
    Object.keys(obj).forEach(function(key) {
      var val = obj[key];
      if (val === undefined) { clean[key] = null; }
      else if (val instanceof Date) { clean[key] = dateToISO(val); }
      else if (typeof val === 'function') { /* skip functions */ }
      else { clean[key] = sanitizeReturn(val); }
    });
    return clean;
  }
  return obj;
}
function isSameDay(d1, d2) {
  if (!d1 || !d2) return false;
  return d1.getFullYear() === d2.getFullYear() && d1.getMonth() === d2.getMonth() && d1.getDate()
=== d2.getDate();
}

// ── INITIALIZE SHEETS ──────────────────────────────────────
```

```
function initializeSheets() {
  try {
  var ss=getSS();
  var needed = [

{name:SH.USERS,headers:['Username','Password','FullName','Role','Email','Status','OTP','OTPExpiry','
ExpiryDate','ApprovedBy','ApprovedDate','CreatedDate','LastLogin','ImgAnalAccess']},

{name:SH.PARAMS,headers:['ParamID','Parameter','OwnerUsername','CreatedDate','CreatedBy','Bid
ang']},

{name:SH.LOTQC,headers:['LotID','ParamID','NoLot','NamaAlat','Methode','Satuan','ExpiredDate','Su
mber','MeanL1','SDL1','TargetL1','MeanL2','SDL2','TargetL2','MeanL3','SDL3','TargetL3','TEa','BiasPct'
,'OwnerUsername']},

{name:SH.INPUTQC,headers:['QCID','ParamID','LotID','Parameter','NoLot','NamaAlat','Tanggal','Level
1','Level2','Level3','InputBy','InputDate','Validated','ValidatedBy','ValidatedDate','CatatanValidasi','O
wnerUsername']},

{name:SH.HISTORI,headers:['HQCID','QCID','ParamID','LotID','Parameter','NoLot','NamaAlat','Tanggal
','Level1','Level2','Level3','InputBy','DeletedBy','DeletedDate','OwnerUsername','ActionType','Chang
eDetail']},

{name:SH.CALCSTATS,headers:['StatID','ParamID','LotID','Level','CalcMean','CalcSD','CalcCV','N','Start
Date','EndDate','OwnerUsername']},

{name:SH.BIASPME,headers:['PMEID','ParamID','LotID','NamaAlat','Methode','Satuan','Siklus','Tahun
','HasilL1','HasilL2','HasilL3','MeanPesertaL1','MeanPesertaL2','MeanPesertaL3','TEa','CVL1','CVL2','C
VL3','CVStartDate','CVEndDate','OwnerUsername']},

{name:SH.DAFTARTEA,headers:['TeaID','ParamID','Parameter','NilaiTEa','Referensi','OwnerUsername
']},

{name:SH.SIGMACVOPT,headers:['CVOptID','ParamID','LotID','NamaAlat','StartDate','EndDate','AvgSi
gma','AvgSigmaL12','TEa','NQC','Catatan','OwnerUsername','SiklusPME','TahunSiklus','BiasPME1','Bia
sPME2','BiasPME3']},

{name:SH.LAPORAN,headers:['CatatanID','FilterKey','BulanTahun','ParamID','LotID','NamaAlat','Catat
an','OwnerUsername']},
   {name:SH.TABULASI,headers:['PeriodKey','Catatan','By','CreatedDate']},
   {name:SH.KOPSURAT,headers:['Key','Value','OwnerUsername']},
   {name:SH.SETTINGS,headers:['Key','Value']},
   {name:SH.LOGACTIVITY,headers:['Timestamp','Username','Action','Detail']},
```

```
{name:SH.CATATANDOKTER,headers:['CatatanID','FilterKey','Bidang','ParamID','LotID','Catatan','OwnerUsername','CreatedDate']},

{name:SH.IMGHEMATO,headers:['ID','NoRM','Nama','TglLahir','Umur','JK','Alamat','Dokter','TglPeriksa','TglHasil','Diagnosis','Asal','Eritrosit','Leukosit','Trombosit','Kesan','Saran','ImageURL','ImageURL2','ImageURL3','OwnerUsername','CreatedDate']},

{name:SH.IMGURIN,headers:['ID','NoRM','Nama','TglLahir','Umur','JK','Alamat','Dokter','TglPeriksa','TglHasil','Diagnosis','Asal','Interpretasi','ImageURL','ImageURL2','ImageURL3','OwnerUsername','CreatedDate']},

{name:SH.IMGMALARIA,headers:['ID','NoRM','Nama','TglLahir','Umur','JK','Alamat','Dokter','TglPeriksa','TglHasil','Diagnosis','Asal','HasilMikro','Spesies','Kepadatan','ImageURL','ImageURL2','ImageURL3','OwnerUsername','CreatedDate']},

{name:SH.IMGBTA,headers:['ID','NoRM','Nama','TglLahir','Umur','JK','Alamat','Dokter','TglPeriksa','TglHasil','Diagnosis','Asal','HasilMikro','Kepadatan','ImageURL','ImageURL2','ImageURL3','OwnerUsername','CreatedDate']},

{name:SH.IMGLAIN,headers:['ID','NoRM','Nama','TglLahir','Umur','JK','Alamat','Dokter','TglPeriksa','TglHasil','Diagnosis','Asal','Interpretasi','ImageURL','ImageURL2','ImageURL3','OwnerUsername','CreatedDate']},

{name:SH.IMGPATOLOGI,headers:['ID','NoRM','NoPA','NamaPasien','TglLahir','Umur','JK','Alamat','TglTerima','TglJawab','AsalRuangan','AsalRujukan','StatusBiaya','DokterPengirim','Diagnosis','JenisPemeriksaan','Makroskopis','Mikroskopis','Kesan','Saran','Catatan','Topografi','Morfologi','ImageURL','ImageURL2','ImageURL3','ImageURL4','ImageURL5','OwnerUsername','CreatedDate']},

{name:SH.USERPASSWORDS,headers:['Username','Password','Nama','CreatedBy','CreatedDate','Note','LoginUsername','Status']}
  ];
  needed.forEach(function(def) {
    var sh=ss.getSheetByName(def.name);
    if(!sh){
      sh=ss.insertSheet(def.name);

sh.getRange(1,1,1,def.headers.length).setValues([def.headers]).setFontWeight('bold').setBackground('#1a73e8').setFontColor('#ffffff');
      sh.setFrozenRows(1);
    } else {
      if (def.name === SH.IMGHEMATO || def.name === SH.IMGURIN || def.name === SH.IMGMALARIA ||
          def.name === SH.IMGBTA || def.name === SH.IMGLAIN) {
        var hdrs = sh.getRange(1, 1, 1, sh.getLastColumn()).getValues()[0];
```

```
      if (hdrs.indexOf('ImageURL2') < 0) {
        var imgIdx = hdrs.indexOf('ImageURL');
        if (imgIdx >= 0) {
          sh.insertColumnAfter(imgIdx + 1);
          sh.getRange(1, imgIdx +
2).setValue('ImageURL2').setFontWeight('bold').setBackground('#1a73e8').setFontColor('#ffffff');
          sh.insertColumnAfter(imgIdx + 2);
          sh.getRange(1, imgIdx +
3).setValue('ImageURL3').setFontWeight('bold').setBackground('#1a73e8').setFontColor('#ffffff');
        }
      }
    }
    if (def.name === SH.IMGPATOLOGI) {
      var hdrsP = sh.getRange(1, 1, 1, sh.getLastColumn()).getValues()[0];
      if (hdrsP.indexOf('ImageURL2') < 0) {
        var imgIdxP = hdrsP.indexOf('ImageURL');
        if (imgIdxP >= 0) {
          for (var mi = 2; mi <= 5; mi++) {
            sh.insertColumnAfter(imgIdxP + mi);
            sh.getRange(1, imgIdxP + mi + 1).setValue('ImageURL' +
mi).setFontWeight('bold').setBackground('#1a73e8').setFontColor('#ffffff');
          }
        }
      }
    }
    // 🆕 Auto-migrate SigmaCVOpt: tambah kolom SiklusPME, TahunSiklus, BiasPME1/2/3
    if (def.name === SH.SIGMACVOPT) {
      var hdrs2 = sh.getRange(1, 1, 1, sh.getLastColumn()).getValues()[0];
      if (hdrs2.indexOf('SiklusPME') < 0) {
        sh.getRange(1,
12).setValue('SiklusPME').setFontWeight('bold').setBackground('#1a73e8').setFontColor('#ffffff');
        sh.getRange(1,
13).setValue('TahunSiklus').setFontWeight('bold').setBackground('#1a73e8').setFontColor('#ffffff');
        sh.getRange(1,
14).setValue('BiasPME1').setFontWeight('bold').setBackground('#1a73e8').setFontColor('#ffffff');
        sh.getRange(1,
15).setValue('BiasPME2').setFontWeight('bold').setBackground('#1a73e8').setFontColor('#ffffff');
        sh.getRange(1,
16).setValue('BiasPME3').setFontWeight('bold').setBackground('#1a73e8').setFontColor('#ffffff');
      }
    }
    if (def.name === SH.USERPASSWORDS) {
      var hdrsUP = sh.getRange(1, 1, 1, sh.getLastColumn()).getValues()[0];
      if (hdrsUP.indexOf('LoginUsername') < 0) {
```

```
        sh.getRange(1,
7).setValue('LoginUsername').setFontWeight('bold').setBackground('#1a73e8').setFontColor('#ffffff')
;
      }
      if (hdrsUP.indexOf('Status') < 0) {
        sh.getRange(1,
8).setValue('Status').setFontWeight('bold').setBackground('#1a73e8').setFontColor('#ffffff');
        var upLR = sh.getLastRow();
        if(upLR > 1) sh.getRange(2, 8, upLR - 1, 1).setValue('active');
      }
    }
  }
 });
 var stSh=S(SH.SETTINGS); var stData=stSh.getDataRange().getValues();
 var stKeys=stData.map(function(r){return r[0];});
 var defaults=[['theme_primary','#2563eb'],['sidebar_bg','#0f172a'],['auto_logout','180'],
  ['sb_font_size','14'],['sb_font_weight','500'],['sb_font_color','#f1f5f9'],
  ['backup_auto','false'],['archive_auto','true']];
 defaults.forEach(function(d){if(stKeys.indexOf(d[0])<0)stSh.appendRow(d);});
 return {ok:true,msg:'Sheets berhasil diinisialisasi'};
 } catch(e){return {ok:false,msg:e.message};}
}

// ── SETTINGS ──────────────────────────────────────────────────
function getSetting(key){var sh=S(SH.SETTINGS);if(!sh)return null;var
data=sh.getDataRange().getValues();for(var i=1;i<data.length;i++){if(data[i][0]===key)return
data[i][1];}return null;}
function setSetting(key,value){return withLock(function(){var sh=S(SH.SETTINGS);var
data=sh.getDataRange().getValues();for(var
i=1;i<data.length;i++){if(data[i][0]===key){sh.getRange(i+1,2).setValue(value);return{ok:true};}}sh.ap
pendRow([key,value]);return{ok:true};});}
function getSettings(){var sh=S(SH.SETTINGS);if(!sh)return{};var
data=sh.getDataRange().getValues();var obj={};for(var
i=1;i<data.length;i++){if(data[i][0])obj[data[i][0]]=data[i][1];}return obj;}
function saveSettings(settingsObj){return withLock(function(){var sh=S(SH.SETTINGS);var
data=sh.getDataRange().getValues();var keyMap={};for(var
i=1;i<data.length;i++)keyMap[data[i][0]]=i+1;Object.keys(settingsObj).forEach(function(k){if(keyMap
[k])sh.getRange(keyMap[k],2).setValue(settingsObj[k]);else
sh.appendRow([k,settingsObj[k]]);});try{if(settingsObj.backup_auto==='true'){var
hasTrigger=false;ScriptApp.getProjectTriggers().forEach(function(t){if(t.getHandlerFunction()==='aut
oBackupDaily')hasTrigger=true;});if(!hasTrigger)ScriptApp.newTrigger('autoBackupDaily').timeBased(
).everyDays(1).atHour(2).create();}else{ScriptApp.getProjectTriggers().forEach(function(t){if(t.getHan
dlerFunction()==='autoBackupDaily')ScriptApp.deleteTrigger(t);});}}catch(e){}return{ok:true};});}
function getLoginSettings(){var
s=getSettings();return{login_font_size:s.login_font_size||'16',login_font_color:s.login_font_color||'
```

```
#ffffff',login_font_weight:s.login_font_weight||'400',login_brightness:s.login_brightness||'0.8',login
_bg_color:s.login_bg_color||'#1e293b'};}

// ── USER & AUTH ──────────────────────────────────────────────
function getUserByUsername(username) {
  var data=gSD(SH.USERS);
  for(var i=0;i<data.length;i++){
    if(String(data[i][0]).toLowerCase()===String(username).toLowerCase()){

return{username:data[i][0],password:data[i][1],fullName:data[i][2],role:data[i][3],email:data[i][4],st
atus:data[i][5],otp:data[i][6],otpExpiry:data[i][7],expiryDate:data[i][8],approvedBy:data[i][9],approv
edDate:data[i][10],createdDate:data[i][11],lastLogin:data[i][12],imgAnalAccess:data[i][13],_row:i+2};
    }
  }
  return null;
}
function ownerMatch(rowOwner,username,role){if(role==='superadmin')return true;return
String(rowOwner).toLowerCase()===String(username).toLowerCase();}

function loginUser(username,password){
  try{
    if(!username||!password)return{ok:false,msg:'Username dan password wajib diisi'};
    var user=getUserByUsername(username);
    if(!user)return{ok:false,msg:'Username tidak ditemukan'};
    var loginAsName='', loginUsername='';
    if(String(user.password)!==String(password)){
      // Cek tambahan password di UserPasswords
      var upData=gSD(SH.USERPASSWORDS);
      var pwdFound=false;
      for(var pi=0;pi<upData.length;pi++){

if(String(upData[pi][0]).toLowerCase()===String(username).toLowerCase()&&String(upData[pi][1])==
=String(password)){
        if(String(upData[pi][7]||'active')!=='active')continue;

pwdFound=true;loginAsName=String(upData[pi][2]||'');loginUsername=String(upData[pi][6]||'');bre
ak;
       }
      }
      if(!pwdFound)return{ok:false,msg:'Password salah'};
    }
    if(user.status==='pending')return{ok:false,msg:'Akun menunggu persetujuan admin'};
    if(user.status==='rejected')return{ok:false,msg:'Akun ditolak oleh admin'};
    if(user.status!=='active')return{ok:false,msg:'Akun tidak aktif'};
    var now=new Date();
```

```
   if(user.expiryDate){var exp=(user.expiryDate instanceof Date)?user.expiryDate:new
Date(user.expiryDate);if(!isNaN(exp.getTime())&&exp<now)return{ok:false,msg:'EXPIRED',expiryDat
e:fD(exp)};}
   withLock(function(){S(SH.USERS).getRange(user._row,13).setValue(new Date());});
   logA(loginUsername||username,'LOGIN','Berhasil login'+(loginAsName?' sebagai
'+loginAsName:''));
   return
sanitizeReturn({ok:true,username:user.username,fullName:user.fullName,role:user.role,email:user.
email,expiryDate:dateToISO(user.expiryDate),imgAnalAccess:user.imgAnalAccess,loginAsName:login
AsName,loginUsername:loginUsername});
  }catch(e){return{ok:false,msg:'Error: '+e.message};}
}
function registerUser(data){
  return withLock(function(){
   try{
    if(!data.username||!data.password||!data.fullName||!data.email)return{ok:false,msg:'Semua
field wajib diisi'};
    if(getUserByUsername(data.username))return{ok:false,msg:'Username sudah digunakan'};
    var sh=S(SH.USERS);var lr=sh.getLastRow();
    // Hanya user pertama (superadmin pertama) yang boleh self-register
    if(lr>=2)return{ok:false,msg:'Registrasi ditutup. Hubungi superadmin untuk pembuatan akun.'};
    var role='superadmin';var status='active';
    var expiry=new Date();expiry.setFullYear(expiry.getFullYear()+1);

sh.appendRow([data.username,data.password,data.fullName,role,data.email,status,'','',expiry,'','',ne
w Date(),'',false]);
    logA(data.username,'REGISTER','role: '+role);
    return{ok:true,msg:'Akun superadmin berhasil dibuat'};
   }catch(e){return{ok:false,msg:'Error: '+e.message};}
  });
}
function
notifySuperadminNewUser(nu,fn){try{gSD(SH.USERS).forEach(function(r){if(r[3]==='superadmin'&&r
[5]==='active'&&r[4])MailApp.sendEmail({to:r[4],subject:'[didiQCsys] User Baru',body:'User baru:
'+nu+' ('+fn+')'});});}catch(e){}}
function approveUser(tu,action,au,ed){return withLock(function(){var sh=S(SH.USERS);var
data=sh.getDataRange().getValues();for(var
i=1;i<data.length;i++){if(String(data[i][0]).toLowerCase()===String(tu).toLowerCase()){sh.getRange(i+
1,6).setValue(action==='approve'?'active':'rejected');sh.getRange(i+1,10).setValue(au);sh.getRange(i
+1,11).setValue(new Date());if(action==='approve'&&ed)sh.getRange(i+1,9).setValue(new
Date(ed));try{if(data[i][4])MailApp.sendEmail({to:data[i][4],subject:'[didiQCsys] Akun
'+action,body:'Akun Anda telah di-
'+action});}catch(e2){}logA(au,action.toUpperCase()+'_USER',tu);return{ok:true};}}return{ok:false,ms
g:'User tidak ditemukan'};});}
```

```
function getUsers(cu,cr){try{if(cr!=='superadmin')return{ok:false,msg:'Akses
ditolak'};return{ok:true,data:gSD(SH.USERS).map(function(r){return{username:r[0],fullName:r[2],rol
e:r[3],email:r[4],status:r[5],expiryDate:dateToISO(r[8]),approvedBy:r[9],createdDate:fD(r[11]),lastLo
gin:fDT(r[12]),imgAnalAccess:r[13]};})}}catch(e){return{ok:false,msg:e.message};}}
function saveUser(payload,callerRole){return
withLock(function(){if(callerRole!=='superadmin')return{ok:false,msg:'Akses ditolak'};var
sh=S(SH.USERS);var
data=sh.getDataRange().getValues();if(payload.isNew){if(getUserByUsername(payload.username))re
turn{ok:false,msg:'Username sudah ada'};var expiry=payload.expiryDate?new
Date(payload.expiryDate):(function(){var d=new Date();d.setFullYear(d.getFullYear()+1);return
d;})();sh.appendRow([payload.username,payload.password||'didikqc123',payload.fullName,payload
.role||'user',payload.email,payload.status||'active','','',expiry,'','',new
Date(),'',payload.imgAnalAccess||false]);}else{for(var
i=1;i<data.length;i++){if(String(data[i][0]).toLowerCase()===String(payload.username).toLowerCase()
){if(payload.fullName)sh.getRange(i+1,3).setValue(payload.fullName);if(payload.role)sh.getRange(i+
1,4).setValue(payload.role);if(payload.email)sh.getRange(i+1,5).setValue(payload.email);if(payload.s
tatus)sh.getRange(i+1,6).setValue(payload.status);if(payload.password)sh.getRange(i+1,2).setValue(
payload.password);if(payload.expiryDate)sh.getRange(i+1,9).setValue(new
Date(payload.expiryDate));sh.getRange(i+1,14).setValue(payload.imgAnalAccess||false);break;}}}ret
urn{ok:true};});}
function deleteUser(tu,cr){return
withLock(function(){if(cr!=='superadmin')return{ok:false,msg:'Akses ditolak'};var sh=S(SH.USERS);var
data=sh.getDataRange().getValues();for(var i=data.length-1;i>=1;i--
){if(String(data[i][0]).toLowerCase()===String(tu).toLowerCase()){sh.deleteRow(i+1);return{ok:true};}
}return{ok:false,msg:'User tidak ditemukan'};});}

// ── MULTI-PASSWORD MANAGEMENT ──────────────────────────────
function getUserPasswords(targetUsername,callerRole){
  try{
    if(callerRole!=='superadmin')return{ok:false,msg:'Akses ditolak'};
    var upData=gSD(SH.USERPASSWORDS);
    var result=[];
    for(var i=0;i<upData.length;i++){
      if(String(upData[i][0]).toLowerCase()===String(targetUsername).toLowerCase()){

result.push({username:upData[i][0],password:upData[i][1],nama:upData[i][2],createdBy:upData[i][3]
,createdDate:fD(upData[i][4]),note:upData[i][5],loginUsername:upData[i][6]||'',status:upData[i][7]|
|'active',_row:i+2});
      }
    }
    return{ok:true,data:result};
  }catch(e){return{ok:false,msg:e.message};}
}
function addUserPassword(payload,callerRole,callerUsername){
  return withLock(function(){
```

```
   try{
    if(callerRole!=='superadmin')return{ok:false,msg:'Akses ditolak'};
    if(!payload.targetUsername||!payload.newPassword)return{ok:false,msg:'Username dan
password wajib'};
    if(payload.newPassword.length<6)return{ok:false,msg:'Password min 6 karakter'};
    // Cek user ada
    var user=getUserByUsername(payload.targetUsername);
    if(!user)return{ok:false,msg:'User tidak ditemukan'};
    var sh=S(SH.USERPASSWORDS);

sh.appendRow([payload.targetUsername,payload.newPassword,payload.nama||'',callerUsername,n
ew Date(),payload.note||'',payload.loginUsername||'','active']);
    logA(callerUsername,'ADD_USERPWD','Tambah password untuk: '+payload.targetUsername);
    return{ok:true,msg:'Password tambahan berhasil ditambahkan'};
   }catch(e){return{ok:false,msg:e.message};}
  });
}
function deleteUserPassword(targetUsername,targetPwd,callerRole,callerUsername){
  return withLock(function(){
   try{
    if(callerRole!=='superadmin')return{ok:false,msg:'Akses ditolak'};
    var sh=S(SH.USERPASSWORDS);
    var data=sh.getDataRange().getValues();
    for(var i=data.length-1;i>=1;i--){

if(String(data[i][0]).toLowerCase()===String(targetUsername).toLowerCase()&&String(data[i][1])===
String(targetPwd)){
       sh.deleteRow(i+1);
       logA(callerUsername,'DEL_USERPWD','Hapus password untuk: '+targetUsername);
       return{ok:true,msg:'Password dihapus'};
      }
    }
    return{ok:false,msg:'Password tidak ditemukan'};
   }catch(e){return{ok:false,msg:e.message};}
  });
}
function editUserPassword(payload,callerRole,callerUsername){
  return withLock(function(){
   try{
    if(callerRole!=='superadmin')return{ok:false,msg:'Akses ditolak'};
    if(!payload.targetUsername||!payload.oldPassword)return{ok:false,msg:'Data tidak lengkap'};
    var sh=S(SH.USERPASSWORDS);
    var data=sh.getDataRange().getValues();
```

```
    for(var
i=1;i<data.length;i++){if(String(data[i][0]).toLowerCase()===String(payload.targetUsername).toLower
Case()&&String(data[i][1])===String(payload.oldPassword)){
      if(payload.newPassword)sh.getRange(i+1,2).setValue(payload.newPassword);
      if(payload.nama!==undefined)sh.getRange(i+1,3).setValue(payload.nama);
      if(payload.note!==undefined)sh.getRange(i+1,6).setValue(payload.note);
      if(payload.loginUsername!==undefined)sh.getRange(i+1,7).setValue(payload.loginUsername);
      logA(callerUsername,'EDIT_USERPWD','Edit password untuk: '+payload.targetUsername);
      return{ok:true,msg:'Password berhasil diperbarui'};}}
    return{ok:false,msg:'Password tidak ditemukan'};
  }catch(e){return{ok:false,msg:e.message};}
 });
}
function
toggleUserPasswordStatus(targetUsername,targetPwd,newStatus,callerRole,callerUsername){
 return withLock(function(){
  try{
   if(callerRole!=='superadmin')return{ok:false,msg:'Akses ditolak'};
   if(newStatus!=='active'&&newStatus!=='inactive')return{ok:false,msg:'Status tidak valid'};
   var sh=S(SH.USERPASSWORDS);
   var data=sh.getDataRange().getValues();
   for(var
i=1;i<data.length;i++){if(String(data[i][0]).toLowerCase()===String(targetUsername).toLowerCase()&
&String(data[i][1])===String(targetPwd)){
      sh.getRange(i+1,8).setValue(newStatus);
      logA(callerUsername,'STATUS_USERPWD',newStatus+' password untuk: '+targetUsername);
      return{ok:true,msg:'Status diubah ke '+newStatus};}}
    return{ok:false,msg:'Password tidak ditemukan'};
  }catch(e){return{ok:false,msg:e.message};}
 });
}

// ── KOP SURAT ──────────────────────────────────────────────
function getKopSurat(ownerUsername){var
obj={};gSD(SH.KOPSURAT).forEach(function(r){if(String(r[2]).toLowerCase()===String(ownerUsernam
e).toLowerCase())obj[r[0]]=r[1];});return obj;}
function saveKopSurat(payload,ownerUsername,logUser){return withLock(function(){var
sh=S(SH.KOPSURAT);var
data=sh.getDataRange().getValues();Object.keys(payload).forEach(function(k){var
found=false;for(var
i=1;i<data.length;i++){if(data[i][0]===k&&String(data[i][2]).toLowerCase()===String(ownerUsername
).toLowerCase()){sh.getRange(i+1,2).setValue(payload[k]);data[i][1]=payload[k];found=true;break;}}if
(!found){sh.appendRow([k,payload[k],ownerUsername]);data.push([k,payload[k],ownerUsername]);
}});logA(ownerUsername,'SAVE_KOP','Kop surat disimpan',logUser);return{ok:true};});}
```

```
// ── PARAMETERS ──────────────────────────────────────────────
function getParameters(ownerUsername,role){try{return gSD(SH.PARAMS).filter(function(r){return
ownerMatch(r[2],ownerUsername,role);}).map(function(r){return{paramID:r[0],parameter:r[1],own
er:r[2],createdDate:fD(r[3]),createdBy:r[4],bidang:r[5]};});}catch(e){return [];}}
function saveParameter(payload,ownerUsername,logUser){return withLock(function(){var
sh=S(SH.PARAMS);if(payload.paramID){var data=sh.getDataRange().getValues();for(var
i=1;i<data.length;i++){if(data[i][0]===payload.paramID&&String(data[i][2]).toLowerCase()===String(o
wnerUsername).toLowerCase()){sh.getRange(i+1,2).setValue(payload.parameter);sh.getRange(i+1,6)
.setValue(payload.bidang||'');logA(ownerUsername,'EDIT_PARAM',payload.parameter,logUser);retu
rn{ok:true,paramID:payload.paramID};}}return{ok:false,msg:'Parameter tidak ditemukan'};}else{var
newID=genID('PAR');sh.appendRow([newID,payload.parameter,ownerUsername,new
Date(),ownerUsername,payload.bidang||'']);logA(ownerUsername,'ADD_PARAM',payload.paramete
r,logUser);return{ok:true,paramID:newID};}});}
function deleteParameter(paramID,ownerUsername,logUser){return withLock(function(){var
sh=S(SH.PARAMS);var data=sh.getDataRange().getValues();for(var i=data.length-1;i>=1;i--
){if(data[i][0]===paramID&&String(data[i][2]).toLowerCase()===String(ownerUsername).toLowerCas
e()){sh.deleteRow(i+1);logA(ownerUsername,'DEL_PARAM',paramID,logUser);return{ok:true};}}retur
n{ok:false,msg:'Parameter tidak ditemukan'};});}

// ── LOT QC ──────────────────────────────────────────────────
function getLotQC(ownerUsername,role,paramID){
 try {
 return gSD(SH.LOTQC).filter(function(r){
  return ownerMatch(r[19],ownerUsername,role)&&(!paramID||r[1]===paramID);
 }).map(function(r){
  return{lotID:r[0],paramID:r[1],noLot:r[2],namaAlat:r[3],methode:r[4],satuan:r[5],
   expiredDate:dateToISO(r[6]),sumber:r[7],
   meanL1:r[8],sdL1:r[9],targetL1:r[10],
   meanL2:r[11],sdL2:r[12],targetL2:r[13],
   meanL3:r[14],sdL3:r[15],targetL3:r[16],
   tea:r[17],biasPct:r[18],owner:r[19]};
 });
 } catch(e){return [];}
}
function saveLotQC(payload,ownerUsername,logUser){return withLock(function(){var
sh=S(SH.LOTQC);var
row=[payload.lotID||genID('LOT'),payload.paramID,payload.noLot,payload.namaAlat,payload.meth
ode||'',payload.satuan||'',payload.expiredDate?new
Date(payload.expiredDate):'',payload.sumber||'Manufaktur',parseNumSafe(payload.meanL1)||'',pa
rseNumSafe(payload.sdL1)||'',parseNumSafe(payload.targetL1)||'',parseNumSafe(payload.meanL2)
||'',parseNumSafe(payload.sdL2)||'',parseNumSafe(payload.targetL2)||'',parseNumSafe(payload.m
eanL3)||'',parseNumSafe(payload.sdL3)||'',parseNumSafe(payload.targetL3)||'',parseNumSafe(payl
oad.tea)||'',parseNumSafe(payload.biasPct)||'',ownerUsername];if(payload.lotID){var
data=sh.getDataRange().getValues();for(var
i=1;i<data.length;i++){if(data[i][0]===payload.lotID&&String(data[i][19]).toLowerCase()===String(ow
```

```
nerUsername).toLowerCase()){sh.getRange(i+1,1,1,row.length).setValues([row]);logA(ownerUserna
me,'EDIT_LOT',payload.lotID,logUser);return{ok:true,lotID:payload.lotID};}}return{ok:false,msg:'Lot
tidak ditemukan'};}else{var
newID=genID('LOT');row[0]=newID;sh.appendRow(row);logA(ownerUsername,'ADD_LOT',newID,log
User);return{ok:true,lotID:newID};}});}
function deleteLotQC(lotID,ownerUsername,logUser){return withLock(function(){var
sh=S(SH.LOTQC);var data=sh.getDataRange().getValues();for(var i=data.length-1;i>=1;i--
){if(data[i][0]===lotID&&String(data[i][19]).toLowerCase()===String(ownerUsername).toLowerCase()
){sh.deleteRow(i+1);logA(ownerUsername,'DEL_LOT',lotID,logUser);return{ok:true};}}return{ok:false,
msg:'Lot tidak ditemukan'};});}

function getLotByID(lotID,ownerUsername){
  var data=gSD(SH.LOTQC);
  for(var i=0;i<data.length;i++){
    if(data[i][0]===lotID){
      return{lotID:data[i][0],paramID:data[i][1],noLot:data[i][2],namaAlat:data[i][3],
        methode:data[i][4],satuan:data[i][5],expiredDate:dateToISO(data[i][6]),sumber:data[i][7],
        meanL1:data[i][8],sdL1:data[i][9],targetL1:data[i][10],
        meanL2:data[i][11],sdL2:data[i][12],targetL2:data[i][13],
        meanL3:data[i][14],sdL3:data[i][15],targetL3:data[i][16],
        tea:data[i][17],biasPct:data[i][18],owner:data[i][19]};
    }
  }
  return null;
}
function getParamByID(paramID){var data=gSD(SH.PARAMS);for(var
i=0;i<data.length;i++){if(data[i][0]===paramID)return{paramID:data[i][0],parameter:data[i][1],owner
:data[i][2],bidang:data[i][5]};}return null;}
function getLotInfoForAutoFill(lotID, ownerUsername) {
  try {
  var lot = getLotByID(lotID, ownerUsername);
  if (!lot) return { ok: false };
  return { ok: true, namaAlat: lot.namaAlat||'', methode: lot.methode||'', satuan: lot.satuan||'', tea:
lot.tea||'', sumber: lot.sumber||'' };
  } catch(e){return {ok:false,msg:e.message};}
}

// ── INPUT QC ──────────────────────────────────────────────
function getInputQC(ownerUsername,role,filter){
  try {
  filter=filter||{};
  var shName=filter.year&&filter.year!==new
Date().getFullYear().toString()?'InputQC_'+filter.year:SH.INPUTQC;
  var sh=S(shName);if(!sh)sh=S(SH.INPUTQC);
```

```
  var data=(sh.getLastRow()<2)?[]:sh.getRange(2,1,sh.getLastRow()-
1,sh.getLastColumn()).getValues();
  var paramBidangMap = {};
  if (filter.bidang) { gSD(SH.PARAMS).forEach(function(r) { paramBidangMap[r[0]] = r[5] || 'Lainnya';
}); }
  return data.filter(function(r){
    if(!ownerMatch(r[16],ownerUsername,role))return false;
    if(filter.paramID&&r[1]!==filter.paramID)return false;
    if(filter.lotID&&r[2]!==filter.lotID)return false;
    if(filter.paramIDs && filter.paramIDs.length && filter.paramIDs.indexOf(r[1])<0) return false;
    if(filter.bidang) { var b = paramBidangMap[r[1]] || 'Lainnya'; if (b !== filter.bidang) return false; }
    if(filter.namaAlat) { if (String(r[5]).toLowerCase().indexOf(String(filter.namaAlat).toLowerCase()) <
0) return false; }
    if(filter.startDate){var d=parseDateStr(r[6]);if(!d||d<parseDateStr(filter.startDate))return false;}
    if(filter.endDate){var d2=parseDateStr(r[6]);if(!d2||d2>parseDateStr(filter.endDate))return false;}
    return true;
  }).map(function(r){
    return{qcID:r[0],paramID:r[1],lotID:r[2],parameter:r[3],noLot:r[4],namaAlat:r[5],
      tanggal:dateToISO(r[6]),level1:r[7],level2:r[8],level3:r[9],
      inputBy:r[10],inputDate:fDT(r[11]),validated:r[12],validatedBy:r[13],
      validatedDate:fDT(r[14]),catatanValidasi:r[15],owner:r[16]};
  });
  } catch(e){return [];}
}
function getInputQCById(qcID, ownerUsername, role) {
  try {
  var sh = S(SH.INPUTQC);
  if (!sh || sh.getLastRow() < 2) return null;
  var data = sh.getRange(2, 1, sh.getLastRow()-1, sh.getLastColumn()).getValues();
  for (var i = 0; i < data.length; i++) {
    if (data[i][0] === qcID && ownerMatch(data[i][16], ownerUsername, role)) {
      return {
        qcID: data[i][0], paramID: data[i][1], lotID: data[i][2], parameter: data[i][3],
        noLot: data[i][4], namaAlat: data[i][5], tanggal: dateToISO(data[i][6]),
        level1: data[i][7], level2: data[i][8], level3: data[i][9],
        inputBy: data[i][10], inputDate: fDT(data[i][11]),
        validated: data[i][12], validatedBy: data[i][13],
        validatedDate: fDT(data[i][14]), catatanValidasi: data[i][15], owner: data[i][16]
      };
    }
  }
  return null;
  } catch(e){return null;}
}
```

```
function saveInputQC(payload,ownerUsername,logUser){return withLock(function(){try{var
sh=S(SH.INPUTQC);var lot=getLotByID(payload.lotID,ownerUsername);var
param=getParamByID(payload.paramID);var
row=[payload.qcID||genID('QC'),payload.paramID,payload.lotID,param?param.parameter:(payload.
parameter||''),lot?lot.noLot:(payload.noLot||''),lot?lot.namaAlat:(payload.namaAlat||''),payload.ta
nggal?new Date(payload.tanggal):new
Date(),parseNumSafe(payload.level1),parseNumSafe(payload.level2),parseNumSafe(payload.level3),
ownerUsername,new Date(),false,'','',ownerUsername];if(payload.qcID){var
data=sh.getDataRange().getValues();for(var
i=1;i<data.length;i++){if(data[i][0]===payload.qcID&&String(data[i][16]).toLowerCase()===String(ow
nerUsername).toLowerCase()){var
old=data[i].slice();sh.getRange(i+1,1,1,row.length).setValues([row]);addHistoriQC(old,logUser||own
erUsername,'EDIT_QC','L1:'+old[7]+'→'+payload.level1+',L2:'+old[8]+'→'+payload.level2+',L3:'+old[9
]+'→'+payload.level3);logA(ownerUsername,'EDIT_QC',payload.qcID,logUser);return{ok:true};}}retur
n{ok:false,msg:'Data tidak ditemukan'};}else{var
newID=genID('QC');row[0]=newID;sh.appendRow(row);checkAndNotifyWestgard(payload.paramID,p
ayload.lotID,ownerUsername,newID);logA(ownerUsername,'ADD_QC',newID,logUser);return{ok:tru
e,qcID:newID};}}catch(e){return{ok:false,msg:e.message};}});}
function deleteInputQC(qcID,ownerUsername,alasan,logUser){return withLock(function(){var
sh=S(SH.INPUTQC);var data=sh.getDataRange().getValues();var
by=logUser||ownerUsername;for(var i=data.length-1;i>=1;i--
){if(data[i][0]===qcID&&String(data[i][16]).toLowerCase()===String(ownerUsername).toLowerCase())
{addHistoriQC(data[i],by,'DATA
DIHAPUS',alasan||'');sh.deleteRow(i+1);logA(ownerUsername,'DEL_QC',qcID+'|'+alasan,logUser);ret
urn{ok:true};}}return{ok:false,msg:'Data tidak ditemukan'};});}
function
addHistoriQC(rowArr,deletedBy,actionType,changeDetail){try{S(SH.HISTORI).appendRow([genID('HQ
C'),rowArr[0],rowArr[1],rowArr[2],rowArr[3],rowArr[4],rowArr[5],rowArr[6],rowArr[7],rowArr[8],row
Arr[9],rowArr[10],deletedBy,new Date(),rowArr[16],actionType||'',changeDetail||'']);}catch(e){}}

function getHistoriQC(ownerUsername, role, filter) {
  try {
  filter = filter || {};
  var paramMap = {};
  gSD(SH.PARAMS).forEach(function(r) { paramMap[r[0]] = { parameter: r[1], bidang: r[5] || 'Lainnya'
}; });
  return sanitizeReturn(gSD(SH.HISTORI).filter(function(r) {
    if (!ownerMatch(r[14], ownerUsername, role)) return false;
    if (filter.actionType && r[15] !== filter.actionType) return false;
    if (filter.paramID && r[2] !== filter.paramID) return false;
    if (filter.lotID && r[3] !== filter.lotID) return false;
    if (filter.bidang) {
      var pinfo = paramMap[r[2]];
      var b = pinfo ? pinfo.bidang : 'Lainnya';
      if (b !== filter.bidang) return false;
```

```
    }
    if (filter.startDate) { var d = parseDateStr(r[7]); if (!d || d < parseDateStr(filter.startDate)) return
false; }
    if (filter.endDate) { var d2 = parseDateStr(r[7]); if (!d2 || d2 > parseDateStr(filter.endDate)) return
false; }
    return true;
  }).map(function(r) {
    var pinfo = paramMap[r[2]];
    return {
      hqcID: r[0], qcID: r[1], paramID: r[2], lotID: r[3],
      parameter: r[4] || (pinfo ? pinfo.parameter : ''),
      bidang: pinfo ? pinfo.bidang : 'Lainnya',
      noLot: r[5], namaAlat: r[6], tanggal: dateToISO(r[7]),
      level1: r[8], level2: r[9], level3: r[10], inputBy: r[11],
      deletedBy: r[12], deletedDate: fDT(r[13]), owner: r[14],
      actionType: r[15], changeDetail: r[16]
    };
  }));
  } catch(e){return [];}
}

function restoreHistoriQC(hqcID, ownerUsername) {
  return withLock(function() {
    try {
      var hSh = S(SH.HISTORI);
      var hData = hSh.getDataRange().getValues();
      var foundRow = null;
      for (var i = 1; i < hData.length; i++) {
        if (hData[i][0] === hqcID && String(hData[i][14]).toLowerCase() ===
String(ownerUsername).toLowerCase()) {
          foundRow = hData[i]; break;
        }
      }
      if (!foundRow) return { ok: false, msg: 'Data histori tidak ditemukan' };
      if (foundRow[15] !== 'DATA DIHAPUS') return { ok: false, msg: 'Hanya data DIHAPUS yang bisa di-
restore' };
      var iSh = S(SH.INPUTQC);
      var iData = iSh.getDataRange().getValues();
      for (var j = 1; j < iData.length; j++) {
        if (iData[j][0] === foundRow[1]) return { ok: false, msg: 'Data QC sudah ada di InputQC' };
      }
      var qcRow = [foundRow[1], foundRow[2], foundRow[3], foundRow[4], foundRow[5],
foundRow[6], foundRow[7],
                   foundRow[8], foundRow[9], foundRow[10], foundRow[11], new Date(), false, '', '', '',
foundRow[14]];
```

```
    iSh.appendRow(qcRow);
    var newHQC = [genID('HQC'), foundRow[1], foundRow[2], foundRow[3], foundRow[4],
foundRow[5], foundRow[6],
            foundRow[7], foundRow[8], foundRow[9], foundRow[10], foundRow[11],
ownerUsername, new Date(),
            foundRow[14], 'RESTORED', 'Restored from HQC=' + hqcID];
    hSh.appendRow(newHQC);
    logA(ownerUsername, 'RESTORE_QC', foundRow[1]);
    return { ok: true, msg: 'Data berhasil di-restore' };
   } catch (e) { return { ok: false, msg: e.message }; }
  });
}

function deleteHistoriQC(hqcID, ownerUsername) {
  return withLock(function() {
   try {
    var sh = S(SH.HISTORI);
    var data = sh.getDataRange().getValues();
    for (var i = data.length - 1; i >= 1; i--) {
     if (data[i][0] === hqcID && String(data[i][14]).toLowerCase() ===
String(ownerUsername).toLowerCase()) {
       sh.deleteRow(i + 1);
       logA(ownerUsername, 'DELETE_HISTORI', 'HQC=' + hqcID);
       return { ok: true, msg: 'Histori berhasil dihapus permanen' };
      }
    }
    return { ok: false, msg: 'Data histori tidak ditemukan' };
   } catch (e) { return { ok: false, msg: e.message }; }
  });
}

function bulkInputQC(payload,ownerUsername,logUser){return
withLock(function(){try{if(payload.smartPassword!==SMART_PWD)return{ok:false,msg:'Password
Smart Input salah'};var sh=S(SH.INPUTQC);var lot=getLotByID(payload.lotID,ownerUsername);var
param=getParamByID(payload.paramID);var
count=0,errors=[];(payload.rows||[]).forEach(function(row){if(!row.tanggal){errors.push('Tanggal
kosong');return;}var dt=new Date(row.tanggal);if(isNaN(dt.getTime())){errors.push('Tanggal invalid:
'+row.tanggal);return;}var
l1=parseNumSafe(row.level1),l2=parseNumSafe(row.level2),l3=parseNumSafe(row.level3);if(l1===n
ull&&l2===null&&l3===null)return;sh.appendRow([genID('QC'),payload.paramID,payload.lotID,para
m?param.parameter:'',lot?lot.noLot:'',lot?lot.namaAlat:'',dt,l1,l2,l3,ownerUsername,new
Date(),false,'','',ownerUsername]);count++;});logA(ownerUsername,'BULK_QC',count+' data
ditambahkan',logUser);return{ok:true,count:count,errors:errors};}catch(e){return{ok:false,msg:e.me
ssage};}});}
```

```
function getQCByDateRange(paramID,lotID,startDate,endDate,ownerUsername){try{var
filter={paramID:paramID,lotID:lotID,startDate:startDate,endDate:endDate};var
data=getInputQC(ownerUsername,'user',filter);data.sort(function(a,b){return new Date(a.tanggal)-
new Date(b.tanggal);});return{ok:true,data:data};}catch(e){return{ok:false,msg:e.message};}}

// ── VALIDASI QC ──────────────────────────────────────────────
function getValidasiData(ownerUsername,role,filter){try{filter=filter||{};var
qcData=getInputQC(ownerUsername,role,filter);var
lotMap={};gSD(SH.LOTQC).forEach(function(r){lotMap[r[0]]=r;});return
sanitizeReturn(qcData.map(function(q){var lot=lotMap[q.lotID]||[];var
mL1=parseNumSafe(lot[8]),sL1=parseNumSafe(lot[9]);var
mL2=parseNumSafe(lot[11]),sL2=parseNumSafe(lot[12]);var
mL3=parseNumSafe(lot[14]),sL3=parseNumSafe(lot[15]);function
zS(val,mean,sd){if(val===null||val===''||!mean||!sd)return null;return((parseNumSafe(val)||0)-
mean)/sd;}return
Object.assign({},q,{zL1:zS(q.level1,mL1,sL1),zL2:zS(q.level2,mL2,sL2),zL3:zS(q.level3,mL3,sL3),meanL1
:mL1,sdL1:sL1,meanL2:mL2,sdL2:sL2,meanL3:mL3,sdL3:sL3,tea:parseNumSafe(lot[17]),satuan:lot[5]}
);}))}catch(e){return [];}}
function validateQC(qcID,catatanValidasi,validatedBy,ownerUsername,logUser){return
withLock(function(){var sh=S(SH.INPUTQC);var data=sh.getDataRange().getValues();for(var
i=1;i<data.length;i++){if(data[i][0]===qcID&&String(data[i][16]).toLowerCase()===String(ownerUsern
ame).toLowerCase()){sh.getRange(i+1,13).setValue(true);sh.getRange(i+1,14).setValue(validatedBy);
sh.getRange(i+1,15).setValue(new
Date());sh.getRange(i+1,16).setValue(catatanValidasi||'');logA(validatedBy,'VALIDATE_QC',qcID,logU
ser);return{ok:true};}}return{ok:false,msg:'Data tidak ditemukan'};});}
function validateQCBulk(qcIDs,catatanValidasi,validatedBy,ownerUsername,logUser){return
withLock(function(){var sh=S(SH.INPUTQC);var data=sh.getDataRange().getValues();var
count=0;for(var i=1;i<data.length;i++){if(qcIDs.indexOf(data[i][0])>-
1&&String(data[i][16]).toLowerCase()===String(ownerUsername).toLowerCase()){if(!data[i][12]){sh.g
etRange(i+1,13).setValue(true);sh.getRange(i+1,14).setValue(validatedBy);sh.getRange(i+1,15).setVa
lue(new
Date());sh.getRange(i+1,16).setValue(catatanValidasi||'');count++;}}}if(count>0)logA(validatedBy,'VA
LIDATE_QC_BULK',count+' entries',logUser);return{ok:true,count:count};});}

// ── DAFTAR TEa ──────────────────────────────────────────────
function getDaftarTEa(ownerUsername,role){try{return gSD(SH.DAFTARTEA).filter(function(r){return
ownerMatch(r[5],ownerUsername,role);}).map(function(r){return{teaID:r[0],paramID:r[1],parameter
:r[2],nilaiTEa:r[3],referensi:r[4],owner:r[5]};});}catch(e){return [];}}
function saveDaftarTEa(payload,ownerUsername,logUser){return withLock(function(){var
sh=S(SH.DAFTARTEA);if(payload.teaID){var data=sh.getDataRange().getValues();for(var
i=1;i<data.length;i++){if(data[i][0]===payload.teaID&&String(data[i][5]).toLowerCase()===String(own
erUsername).toLowerCase()){sh.getRange(i+1,2,1,4).setValues([[payload.paramID,payload.parameter
r,parseNumSafe(payload.nilaiTEa),payload.referensi||'']]);return{ok:true};}}return{ok:false,msg:'TEa
tidak
```

```
ditemukan'};}else{sh.appendRow([genID('TEA'),payload.paramID,payload.parameter,parseNumSafe(
payload.nilaiTEa),payload.referensi||'',ownerUsername]);return{ok:true};}});}
function deleteDaftarTEa(teaID,ownerUsername,logUser){return withLock(function(){var
sh=S(SH.DAFTARTEA);var data=sh.getDataRange().getValues();for(var i=data.length-1;i>=1;i--
){if(data[i][0]===teaID&&String(data[i][5]).toLowerCase()===String(ownerUsername).toLowerCase())
{sh.deleteRow(i+1);return{ok:true};}}return{ok:false,msg:'TEa tidak ditemukan'};});}

// ── BIAS PME ──────────────────────────────────────────────────
function getBiasPME(ownerUsername, role, filter) {
 filter = filter || {};
 var lotMap = {};
 gSD(SH.LOTQC).forEach(function(l) { lotMap[l[0]] = l; });
 return gSD(SH.BIASPME).filter(function(r) {
  if (!ownerMatch(r[20], ownerUsername, role)) return false;
  if (filter.paramID && r[1] !== filter.paramID) return false;
  if (filter.lotID && r[2] !== filter.lotID) return false;
  return true;
 }).map(function(r) {
  var lot = lotMap[r[2]];
  var item = {
   pmeID: r[0], paramID: r[1], lotID: r[2], namaAlat: r[3], methode: r[4], satuan: r[5],
   siklus: r[6], tahun: r[7],
   hasilL1: r[8], hasilL2: r[9], hasilL3: r[10],
   meanPesertaL1: r[11], meanPesertaL2: r[12], meanPesertaL3: r[13],
   tea: r[14], cvL1: r[15], cvL2: r[16], cvL3: r[17],
   cvStartDate: dateToISO(r[18]), cvEndDate: dateToISO(r[19]), owner: r[20],
   details: {}
  };
  [1, 2, 3].forEach(function(lv) {
   var lotMean = lot ? parseNumSafe(lot[8 + (lv-1)*3]) : null;
   var lotSD = lot ? parseNumSafe(lot[9 + (lv-1)*3]) : null;
   var hasil = parseNumSafe(item['hasilL'+lv]);
   var meanP = parseNumSafe(item['meanPesertaL'+lv]);
   var cvUsed = parseNumSafe(item['cvL'+lv]);
   var bias = (hasil && meanP) ? Math.abs((hasil - meanP) / meanP * 100) : null;
   var cv = cvUsed;
   if (!cv && lotMean && lotSD) cv = lotSD / lotMean * 100;
   var te = (bias !== null && cv !== null) ? Math.abs(bias) + 1.65 * cv : null;
   var tea = parseNumSafe(item.tea);
   var sigma = (tea !== null && bias !== null && cv) ? (tea - bias) / cv : null;
   item.details['L'+lv] = {
    mean: lotMean ? parseFloat(lotMean.toFixed(3)) : null,
    sd: lotSD ? parseFloat(lotSD.toFixed(3)) : null,
    cv: cv !== null ? parseFloat(cv.toFixed(2)) : null,
    bias: bias !== null ? parseFloat(bias.toFixed(2)) : null,
```

```
      te: te !== null ? parseFloat(te.toFixed(2)) : null,
      tea: tea,
      sigma: sigma !== null ? parseFloat(sigma.toFixed(2)) : null
    };
   });
   return item;
  });
}
function saveBiasPME(payload, ownerUsername, logUser) {
  return withLock(function() {
   var sh = S(SH.BIASPME);
   var cvL1='', cvL2='', cvL3='';
   if (payload.cvStartDate && payload.cvEndDate && payload.lotID) {
    var cvStats = calcCVFromInputQC(payload.lotID, payload.cvStartDate, payload.cvEndDate,
ownerUsername);
    cvL1 = cvStats.cvL1; cvL2 = cvStats.cvL2; cvL3 = cvStats.cvL3;
   } else {
    cvL1 = parseNumSafe(payload.cvL1) || '';
    cvL2 = parseNumSafe(payload.cvL2) || '';
    cvL3 = parseNumSafe(payload.cvL3) || '';
   }
   var row = [payload.pmeID || genID('PME'), payload.paramID, payload.lotID,
    payload.namaAlat || '', payload.methode || '', payload.satuan || '',
    payload.siklus || '', payload.tahun || new Date().getFullYear(),
    parseNumSafe(payload.hasilL1) || '', parseNumSafe(payload.hasilL2) || '',
parseNumSafe(payload.hasilL3) || '',
    parseNumSafe(payload.meanPesertaL1) || '', parseNumSafe(payload.meanPesertaL2) || '',
parseNumSafe(payload.meanPesertaL3) || '',
    parseNumSafe(payload.tea) || '', cvL1, cvL2, cvL3,
    payload.cvStartDate ? new Date(payload.cvStartDate) : '',
    payload.cvEndDate ? new Date(payload.cvEndDate) : '', ownerUsername];
   if (payload.pmeID) {
    var data = sh.getDataRange().getValues();
    for (var i = 1; i < data.length; i++) {
     if (data[i][0] === payload.pmeID && String(data[i][20]).toLowerCase() ===
String(ownerUsername).toLowerCase()) {
       sh.getRange(i+1, 1, 1, row.length).setValues([row]);
       return { ok: true };
     }
    }
    return { ok: false, msg: 'PME tidak ditemukan' };
   } else { sh.appendRow(row); return { ok: true }; }
  });
}
```

```
function deleteBiasPME(pmeID,ownerUsername,logUser){return withLock(function(){var
sh=S(SH.BIASPME);var data=sh.getDataRange().getValues();for(var i=data.length-1;i>=1;i--
){if(data[i][0]===pmeID&&String(data[i][20]).toLowerCase()===String(ownerUsername).toLowerCase
()){sh.deleteRow(i+1);return{ok:true};}}return{ok:false,msg:'PME tidak ditemukan'};});}
function getBiasPMEById(pmeID, ownerUsername, role) {
 try {
 var data = gSD(SH.BIASPME);
 for (var i = 0; i < data.length; i++) {
  if (data[i][0] === pmeID && ownerMatch(data[i][20], ownerUsername, role)) {
   return {
    pmeID: data[i][0], paramID: data[i][1], lotID: data[i][2],
    namaAlat: data[i][3], methode: data[i][4], satuan: data[i][5],
    siklus: data[i][6], tahun: data[i][7],
    hasilL1: data[i][8], hasilL2: data[i][9], hasilL3: data[i][10],
    meanPesertaL1: data[i][11], meanPesertaL2: data[i][12], meanPesertaL3: data[i][13],
    tea: data[i][14], cvL1: data[i][15], cvL2: data[i][16], cvL3: data[i][17],
    cvStartDate: dateToISO(data[i][18]), cvEndDate: dateToISO(data[i][19]), owner: data[i][20]
   };
  }
 }
 return null;
 } catch(e){return null;}
}
function calcCVFromInputQC(lotID,startDate,endDate,ownerUsername){var
data=gSD(SH.INPUTQC);var sd=parseDateStr(startDate),ed=parseDateStr(endDate);var
vals=[[],[],[]];data.forEach(function(r){if(r[2]!==lotID||String(r[16]).toLowerCase()!==String(ownerUs
ername).toLowerCase())return;var
d=parseDateStr(r[6]);if(!d||d<sd||d>ed)return;[r[7],r[8],r[9]].forEach(function(v,idx){var
n=parseNumSafe(v);if(n!==null&&n!==0)vals[idx].push(n);});});function
calcCV(arr){if(!arr.length)return '';var mean=arr.reduce(function(a,b){return
a+b;},0)/arr.length;if(!mean)return '';var sdv=Math.sqrt(arr.reduce(function(s,v){return
s+Math.pow(v-mean,2);},0)/arr.length);return
parseFloat((sdv/mean*100).toFixed(2));}return{cvL1:calcCV(vals[0]),cvL2:calcCV(vals[1]),cvL3:calcCV(
vals[2])};}

// ── CALC STATS ──────────────────────────────────────────
function getCalcStats(ownerUsername, role, filter) {
 try {
 filter = filter || {};
 var paramMap = {}, lotMap = {};
 gSD(SH.PARAMS).forEach(function(r) { paramMap[r[0]] = r[1]; });
 gSD(SH.LOTQC).forEach(function(l) { lotMap[l[0]] = l; });
 return gSD(SH.CALCSTATS).filter(function(r) {
  if (!ownerMatch(r[10], ownerUsername, role)) return false;
  if (filter.paramID && r[1] !== filter.paramID) return false;
```

```
    return true;
  }).map(function(r) {
    var lot = lotMap[r[2]]; var lv = r[3];
    var lvNum = parseInt(String(lv).replace('L',''));
    var item = {
      statID: r[0], paramID: r[1], lotID: r[2], level: lv,
      parameter: paramMap[r[1]] || '',
      noLot: lot ? lot[2] : '', namaAlat: lot ? lot[3] : '',
      calcMean: r[4], calcSD: r[5], calcCV: r[6], n: r[7],
      startDate: dateToISO(r[8]), endDate: dateToISO(r[9]), owner: r[10],
      tea: lot ? parseNumSafe(lot[17]) : null,
      lotMean: lot ? parseNumSafe(lot[8 + (lvNum-1)*3]) : null,
      lotSD: lot ? parseNumSafe(lot[9 + (lvNum-1)*3]) : null
    };
    if (item.lotMean && item.calcMean) item.bias = parseFloat(((item.calcMean - item.lotMean) /
item.lotMean * 100).toFixed(2));
    if (item.calcCV && item.bias !== undefined) item.te = parseFloat((Math.abs(item.bias) + 1.65 *
item.calcCV).toFixed(2));
    if (item.tea && item.bias !== undefined && item.calcCV) item.sigma = parseFloat(((item.tea -
Math.abs(item.bias)) / item.calcCV).toFixed(2));
    return item;
  });
  } catch(e){return [];}
}
function saveCalcStatsAllLevels(payload, ownerUsername, logUser) {
  return withLock(function() {
    try {
      if (!payload.paramID || !payload.lotID) return { ok: false, msg: 'Parameter dan Lot wajib' };
      if (!payload.startDate || !payload.endDate) return { ok: false, msg: 'Rentang tanggal wajib' };
      var sh = S(SH.CALCSTATS);
      var results = {}; var savedLevels = [];
      ['L1', 'L2', 'L3'].forEach(function(lv) {
        var stats = calcStatsFromInputQC(payload.lotID, lv, payload.startDate, payload.endDate,
ownerUsername);
        if (!stats.n) { results[lv] = null; return; }
        var data = sh.getDataRange().getValues();
        var existingRowIdx = -1;
        for (var i = 1; i < data.length; i++) {
          if (data[i][1] === payload.paramID && data[i][2] === payload.lotID && data[i][3] === lv &&
String(data[i][10]).toLowerCase() === String(ownerUsername).toLowerCase()) {
            existingRowIdx = i + 1; break;
          }
        }
        var row = [
          existingRowIdx > -1 ? data[existingRowIdx-1][0] : genID('STAT'),
```

```
        payload.paramID, payload.lotID, lv,
        stats.mean, stats.sd, stats.cv, stats.n,
        new Date(payload.startDate), new Date(payload.endDate), ownerUsername
       ];
       if (existingRowIdx > -1) sh.getRange(existingRowIdx, 1, 1, row.length).setValues([row]);
       else sh.appendRow(row);
       results[lv] = stats; savedLevels.push(lv);
      });
      logA(ownerUsername, 'SAVE_CALC_STATS', savedLevels.join(','), logUser);
      return { ok: true, msg: savedLevels.length + ' level dihitung & disimpan', results: results,
savedLevels: savedLevels };
    } catch (e) { return { ok: false, msg: e.message }; }
  });
}
function saveCalcStats(payload,ownerUsername,logUser){return withLock(function(){var
mean='',sd='',cv='',n=0;if(payload.startDate&&payload.endDate&&payload.lotID&&payload.level){va
r
stats=calcStatsFromInputQC(payload.lotID,payload.level,payload.startDate,payload.endDate,owner
Username);mean=stats.mean;sd=stats.sd;cv=stats.cv;n=stats.n;}var sh=S(SH.CALCSTATS);var
row=[payload.statID||genID('STAT'),payload.paramID,payload.lotID,payload.level,mean,sd,cv,n,payl
oad.startDate?new Date(payload.startDate):'',payload.endDate?new
Date(payload.endDate):'',ownerUsername];if(payload.statID){var
data=sh.getDataRange().getValues();for(var
i=1;i<data.length;i++){if(data[i][0]===payload.statID&&String(data[i][10]).toLowerCase()===String(o
wnerUsername).toLowerCase()){sh.getRange(i+1,1,1,row.length).setValues([row]);return{ok:true,me
an:mean,sd:sd,cv:cv,n:n};}}return{ok:false,msg:'Stats tidak
ditemukan'};}else{sh.appendRow(row);return{ok:true,mean:mean,sd:sd,cv:cv,n:n};}});}
function deleteCalcStats(statID,ownerUsername,logUser){return withLock(function(){var
sh=S(SH.CALCSTATS);var data=sh.getDataRange().getValues();for(var i=data.length-1;i>=1;i--
){if(data[i][0]===statID&&String(data[i][10]).toLowerCase()===String(ownerUsername).toLowerCase(
)){sh.deleteRow(i+1);return{ok:true};}}return{ok:false,msg:'Stats tidak ditemukan'};});}
function calcStatsFromInputQC(lotID,level,startDate,endDate,ownerUsername){var
data=gSD(SH.INPUTQC);var sd=parseDateStr(startDate),ed=parseDateStr(endDate);var
colIdx=(level==='L1'||level==='1')?7:(level==='L2'||level==='2')?8:9;var
vals=[];data.forEach(function(r){if(r[2]!==lotID||String(r[16]).toLowerCase()!==String(ownerUsernam
e).toLowerCase())return;var d=parseDateStr(r[6]);if(!d||d<sd||d>ed)return;var
v=parseNumSafe(r[colIdx]);if(v!==null&&v!==0)vals.push(v);});if(!vals.length)return{mean:'',sd:'',cv:'',
n:0};var mean=vals.reduce(function(a,b){return a+b;},0)/vals.length;var
sdv=Math.sqrt(vals.reduce(function(s,v){return s+Math.pow(v-mean,2);},0)/vals.length);var
cv=mean?sdv/mean*100:0;return{mean:parseFloat(mean.toFixed(4)),sd:parseFloat(sdv.toFixed(4)),c
v:parseFloat(cv.toFixed(2)),n:vals.length};}
function getCalcStatById(statID, ownerUsername, role) {
 try {
 var data = gSD(SH.CALCSTATS);
 for (var i = 0; i < data.length; i++) {
```

```
    if (data[i][0] === statID && ownerMatch(data[i][10], ownerUsername, role)) {
      return { statID: data[i][0], paramID: data[i][1], lotID: data[i][2], level: data[i][3],
        calcMean: data[i][4], calcSD: data[i][5], calcCV: data[i][6], n: data[i][7],
        startDate: dateToISO(data[i][8]), endDate: dateToISO(data[i][9]), owner: data[i][10] };
    }
  }
  return null;
  } catch(e){return null;}
}

// ── SIGMA CV OPT ──────────────────────────────────────────────
function getSigmaCVOpt(ownerUsername, role, filter) {
  try {
  filter = filter || {};
  var paramMap = {}, lotMap = {};
  gSD(SH.PARAMS).forEach(function(r) { paramMap[r[0]] = r[1]; });
  gSD(SH.LOTQC).forEach(function(l) { lotMap[l[0]] = l; });
  return gSD(SH.SIGMACVOPT).filter(function(r) {
    if (!ownerMatch(r[11], ownerUsername, role)) return false;
    if (filter.paramID && r[1] !== filter.paramID) return false;
    return true;
  }).map(function(r) {
    var lot = lotMap[r[2]];
    var item = {
      cvOptID: r[0], paramID: r[1], lotID: r[2], namaAlat: r[3],
      startDate: dateToISO(r[4]), endDate: dateToISO(r[5]),
      avgSigma: r[6], avgSigmaL12: r[7], tea: r[8], nqc: r[9],
      catatan: r[10], owner: r[11], parameter: paramMap[r[1]] || '',
      noLot: lot ? lot[2] : '', details: {},
      siklusPME: r[12] || '', tahunSiklus: r[13] || '',
      biasPME1: r[14], biasPME2: r[15], biasPME3: r[16]
    };
  if (r[4] && r[5] && lot) {
    [1, 2, 3].forEach(function(lv) {
      var lotMean = parseNumSafe(lot[8 + (lv-1)*3]);
      var tea = parseNumSafe(r[8]);
      var stats = calcStatsFromInputQC(r[2], 'L'+lv, dateToISO(r[4]), dateToISO(r[5]), ownerUsername);
      if (!stats.n) { item.details['L'+lv] = null; return; }

      // PERBAIKAN: Gunakan Bias PME dari database (r[14]=L1, r[15]=L2, r[16]=L3)
      var biasPME = parseNumSafe(r[13 + lv]);
      var bias = (biasPME !== null && biasPME !== '') ? Math.abs(biasPME) : null;

      // Fallback ke lotMean jika biasPME kosong
      if (bias === null && lotMean) {
```

```
      bias = Math.abs((stats.mean - lotMean) / lotMean * 100);
    }

    var te = (bias !== null) ? bias + 1.65 * stats.cv : null;
    var sigma = (tea && bias !== null && stats.cv) ? (tea - bias) / stats.cv : null;
    item.details['L'+lv] = { mean: stats.mean, sd: stats.sd, cv: stats.cv, n: stats.n,
      bias: bias !== null ? parseFloat(bias.toFixed(2)) : null,
      te: te !== null ? parseFloat(te.toFixed(2)) : null,
      tea: tea, sigma: sigma !== null ? parseFloat(sigma.toFixed(2)) : null };
  });
}
    return item;
  });
  } catch(e){return [];}
}
function saveSigmaCVOpt(payload, ownerUsername, logUser) {
  return withLock(function() {
    var sh = S(SH.SIGMACVOPT);
    var avgSigma = '', avgSigmaL12 = '';
    if (payload.lotID && payload.startDate && payload.endDate) {
      var lot = getLotByID(payload.lotID, ownerUsername);
      var tea = parseNumSafe(payload.tea) || (lot ? parseNumSafe(lot.tea) : 0);
      var sigs = []; var sigs12 = [];
      [1, 2, 3].forEach(function(lv) {
        var stats = calcStatsFromInputQC(payload.lotID, 'L'+lv, payload.startDate, payload.endDate,
ownerUsername);
        if (!stats.n || !lot) return;

        // PERBAIKAN: Gunakan Bias PME dari inputan (tombol autofill), fallback ke lotMean
        var biasPME = parseNumSafe(payload['biasPME' + lv]);
        var bias = (biasPME !== null && biasPME !== '') ? Math.abs(biasPME) : null;

        if (bias === null) {
          var lotMean = parseNumSafe(lot[lv===1?'meanL1':lv===2?'meanL2':'meanL3']);
          if (lotMean) bias = Math.abs((stats.mean - lotMean) / lotMean * 100);
        }

        var cv = stats.cv;
        if (tea && cv && bias !== null) {
          var sg = parseFloat(((tea - bias) / cv).toFixed(2));
          sigs.push(sg);
          if (lv <= 2) sigs12.push(sg);
        }
      });
```

```
    if (sigs.length) avgSigma = parseFloat((sigs.reduce(function(a,b){return
a+b;},0)/sigs.length).toFixed(2));
    if (sigs12.length) avgSigmaL12 = parseFloat((sigs12.reduce(function(a,b){return
a+b;},0)/sigs12.length).toFixed(2));
   }
   var row = [payload.cvOptID || genID('CVOPT'), payload.paramID, payload.lotID, payload.namaAlat
|| '',
    payload.startDate ? new Date(payload.startDate) : '', payload.endDate ? new
Date(payload.endDate) : '',
    avgSigma, avgSigmaL12, parseNumSafe(payload.tea) || '', parseInt(payload.nqc) || '',
    payload.catatan || '', ownerUsername,
    payload.siklusPME || '', payload.tahunSiklus || '',
    parseNumSafe(payload.biasPME1) || '', parseNumSafe(payload.biasPME2) || '',
parseNumSafe(payload.biasPME3) || ''];
   if (payload.cvOptID) {
    var data = sh.getDataRange().getValues();
    for (var i = 1; i < data.length; i++) {
     if (data[i][0] === payload.cvOptID && String(data[i][11]).toLowerCase() ===
String(ownerUsername).toLowerCase()) {
       sh.getRange(i+1, 1, 1, row.length).setValues([row]);
       return { ok: true, avgSigma: avgSigma, avgSigmaL12: avgSigmaL12 };
      }
    }
    return { ok: false, msg: 'Data tidak ditemukan' };
   } else { sh.appendRow(row); return { ok: true, avgSigma: avgSigma, avgSigmaL12: avgSigmaL12 }; }
  });
}
function deleteSigmaCVOpt(cvOptID,ownerUsername,logUser){return withLock(function(){var
sh=S(SH.SIGMACVOPT);var data=sh.getDataRange().getValues();for(var i=data.length-1;i>=1;i--
){if(data[i][0]===cvOptID&&String(data[i][11]).toLowerCase()===String(ownerUsername).toLowerCa
se()){sh.deleteRow(i+1);return{ok:true};}}return{ok:false,msg:'Data tidak ditemukan'};});}
function getSigmaCVOptById(cvOptID, ownerUsername, role) {
  try {
  var data = gSD(SH.SIGMACVOPT);
  for (var i = 0; i < data.length; i++) {
   if (data[i][0] === cvOptID && ownerMatch(data[i][11], ownerUsername, role)) {
    return { cvOptID: data[i][0], paramID: data[i][1], lotID: data[i][2], namaAlat: data[i][3],
     startDate: dateToISO(data[i][4]), endDate: dateToISO(data[i][5]),
     avgSigma: data[i][6], avgSigmaL12: data[i][7], tea: data[i][8], nqc: data[i][9],
     catatan: data[i][10], owner: data[i][11],
     siklusPME: data[i][12] || '', tahunSiklus: data[i][13] || '',
     biasPME1: data[i][14], biasPME2: data[i][15], biasPME3: data[i][16] };
   }
  }
  return null;
```

```
 } catch(e){return null;}
}

// ── CATATAN ──────────────────────────────────────────────────
function getCatatanLaporan(ownerUsername,filterKey){try{return
gSD(SH.LAPORAN).filter(function(r){return
String(r[7]).toLowerCase()===String(ownerUsername).toLowerCase()&&(!filterKey||r[1]===filterKey)
;}).map(function(r){return{catatanID:r[0],filterKey:r[1],bulanTahun:r[2],paramID:r[3],lotID:r[4],nama
Alat:r[5],catatan:r[6],owner:r[7]};});}catch(e){return [];}}
function saveCatatanLaporan(payload,ownerUsername,logUser){return withLock(function(){var
sh=S(SH.LAPORAN);var data=sh.getDataRange().getValues();for(var
i=1;i<data.length;i++){if(data[i][1]===payload.filterKey&&data[i][3]===payload.paramID&&data[i][4]
===payload.lotID&&String(data[i][7]).toLowerCase()===String(ownerUsername).toLowerCase()){sh.g
etRange(i+1,7).setValue(payload.catatan||'');return{ok:true};}}sh.appendRow([genID('CAT'),payload.
filterKey||'',payload.bulanTahun||'',payload.paramID||'',payload.lotID||'',payload.namaAlat||'',pay
load.catatan||'',ownerUsername]);return{ok:true};});}
function getCatatanTabulasi(periodKey){var data=gSD(SH.TABULASI);for(var
i=0;i<data.length;i++){if(data[i][0]===periodKey)return{catatan:data[i][1],by:data[i][2],createdDate:f
DT(data[i][3])};}return null;}
function saveCatatanTabulasi(periodKey,catatan,username,logUser){return withLock(function(){var
sh=S(SH.TABULASI);var data=sh.getDataRange().getValues();for(var
i=1;i<data.length;i++){if(data[i][0]===periodKey){sh.getRange(i+1,2).setValue(catatan);sh.getRange(i
+1,3).setValue(username);sh.getRange(i+1,4).setValue(new
Date());return{ok:true};}}sh.appendRow([periodKey,catatan,username,new
Date()]);return{ok:true};});}

// ── CATATAN DOKTER (BARU v9.6) ──────────────────────────────
function getCatatanDokter(ownerUsername, filterKey) {
 try {
 return gSD(SH.CATATANDOKTER).filter(function(r) {
  return String(r[6]).toLowerCase() === String(ownerUsername).toLowerCase() && (!filterKey || r[1]
=== filterKey);
 }).map(function(r) {
  return {
   catatanID: r[0], filterKey: r[1], bidang: r[2], paramID: r[3], lotID: r[4],
   catatan: r[5], owner: r[6], createdDate: fDT(r[7])
  };
 });
 } catch(e){return [];}
}
function saveCatatanDokter(payload, ownerUsername) {
 return withLock(function() {
  var sh = S(SH.CATATANDOKTER);
  var data = sh.getDataRange().getValues();
  var filterKey = payload.filterKey || (payload.bidang + '_' + payload.paramID + '_' + payload.lotID);
```

```
    for (var i = 1; i < data.length; i++) {
      if (data[i][1] === filterKey && String(data[i][6]).toLowerCase() ===
String(ownerUsername).toLowerCase()) {
        sh.getRange(i+1, 5).setValue(payload.catatan || '');
        return { ok: true };
      }
    }
    sh.appendRow([genID('CD'), filterKey, payload.bidang || '', payload.paramID || '', payload.lotID ||
'', payload.catatan || '', ownerUsername, new Date()]);
    return { ok: true };
  });
}

// ── HELPER: FILTER OPTIONS (BARU v9.6) ──────────────────────
function getSiklusPMEList(ownerUsername, role) {
  try {
  var data = gSD(SH.BIASPME).filter(function(r) { return ownerMatch(r[20], ownerUsername, role); });
  var siklusSet = {};
  data.forEach(function(r) { if (r[6]) siklusSet[r[6]] = true; });
  return Object.keys(siklusSet).sort();
  } catch(e){return [];}
}
function getTahunSiklusList(ownerUsername, role) {
  try {
  var data = gSD(SH.BIASPME).filter(function(r) { return ownerMatch(r[20], ownerUsername, role); });
  var tahunSet = {};
  data.forEach(function(r) { if (r[7]) tahunSet[String(r[7])] = true; });
  return Object.keys(tahunSet).sort().reverse();
  } catch(e){return [];}
}
function getPeriodeCalcStatsList(ownerUsername, role) {
  try {
  var data = gSD(SH.CALCSTATS).filter(function(r) { return ownerMatch(r[10], ownerUsername, role);
});
  var periodeSet = {};
  data.forEach(function(r) {
    if (r[8] && r[9]) {
      var key = dateToISO(r[8]) + '|' + dateToISO(r[9]);
      periodeSet[key] = { start: dateToISO(r[8]), end: dateToISO(r[9]) };
    }
  });
  return Object.keys(periodeSet).map(function(k) { return periodeSet[k]; });
  } catch(e){return [];}
}
function getPeriodeSigmaCVOptList(ownerUsername, role) {
```

```
  try {
  var data = gSD(SH.SIGMACVOPT).filter(function(r) { return ownerMatch(r[11], ownerUsername,
role); });
  var periodeSet = {};
  data.forEach(function(r) {
    if (r[4] && r[5]) {
      var key = dateToISO(r[4]) + '|' + dateToISO(r[5]);
      periodeSet[key] = { start: dateToISO(r[4]), end: dateToISO(r[5]) };
    }
  });
  return Object.keys(periodeSet).map(function(k) { return periodeSet[k]; });
  } catch(e){return [];}
}
function getBiasPMEByFilter(paramID, lotID, siklus, tahun, ownerUsername) {
  try {
  var data = gSD(SH.BIASPME).filter(function(r) {
    if (String(r[20]).toLowerCase() !== String(ownerUsername).toLowerCase()) return false;
    if (paramID && r[1] !== paramID) return false;
    if (lotID && r[2] !== lotID) return false;
    if (siklus && r[6] !== siklus) return false;
    if (tahun && String(r[7]) !== String(tahun)) return false;
    return true;
  });
  if (!data.length) return { ok: false, msg: 'Data PME tidak ditemukan' };
  var latest = data[data.length - 1];
  var lotMap = {};
  gSD(SH.LOTQC).forEach(function(l) { lotMap[l[0]] = l; });
  var lot = lotMap[latest[2]];
  var result = { ok: true, pmeID: latest[0], siklus: latest[6], tahun: latest[7] };
  [1, 2, 3].forEach(function(lv) {
    var hasilIdx = lv === 1 ? 8 : lv === 2 ? 9 : 10;
    var meanPIdx = lv === 1 ? 11 : lv === 2 ? 12 : 13;
    var hasil = parseNumSafe(latest[hasilIdx]);
    var meanP = parseNumSafe(latest[meanPIdx]);
    var bias = (hasil && meanP) ? Math.abs((hasil - meanP) / meanP * 100) : null;
    result['biasL' + lv] = bias !== null ? parseFloat(bias.toFixed(2)) : null;
    result['hasilL' + lv] = hasil;
    result['meanPesertaL' + lv] = meanP;
  });
  return result;
  } catch(e){return [];}
}

// ── LOG ACTIVITY — HANYA HARI INI ──────────────────────────
function getLogActivity(ownerUsername,role){
```

```
 try {
 var today = new Date();
 today.setHours(0,0,0,0);
 return gSD(SH.LOGACTIVITY).filter(function(r){
  if (role !== 'superadmin' && String(r[1]).toLowerCase() !== String(ownerUsername).toLowerCase())
return false;
  var ts = r[0] instanceof Date ? r[0] : new Date(r[0]);
  if (isNaN(ts.getTime())) return false;
  return isSameDay(ts, today);
 }).slice(-500).reverse().map(function(r){
  return{timestamp:fDT(r[0]),username:r[1],action:r[2],detail:r[3]};
 });
 } catch(e){return [];}
}
function clearLogActivity(ownerUsername,role){return withLock(function(){var
sh=S(SH.LOGACTIVITY);if(role==='superadmin'){var lr=sh.getLastRow();if(lr>1)sh.deleteRows(2,lr-
1);}else{var data=sh.getDataRange().getValues();for(var i=data.length-1;i>=1;i--
){if(String(data[i][1]).toLowerCase()===String(ownerUsername).toLowerCase())sh.deleteRow(i+1);}}r
eturn{ok:true};});}
// ============================================================
// didiQCsys v9.7 — Backend Code.gs BAGIAN 2
// Westgard, Graph, Laporan, Trend, OPSpecs, Image, dll
// UPDATED v9.7:
// - Enhanced getTrendAnalisisData: instrument compare & rekomendasi alat
// - Enhanced getTabulasiData: PME rekap detail
// - Enhanced getOPSpecsData: analisis detail, kesimpulan, critical error
// - Updated from v9.6 features
// ============================================================

// ── WESTGARD RULES ──────────────────────────────────────────
function checkWestgardRules(values, mean, sd) {
  var violations = [];
  if (!values || values.length === 0 || !mean || !sd) return violations;
  var n = values.length;
  var v = values;

  function z(val) { return (val - mean) / sd; }

  var last = v[n-1];
  var lastZ = z(last);
  var lastAbsZ = Math.abs(lastZ);

  // 1-2s (Warning)
  if (lastAbsZ >= 2 && lastAbsZ < 3) {
    violations.push({ rule:'1-2s', idx:n-1, type:'warning', desc:'Peringatan: 1 nilai melampaui ±2SD' });
```

```
  }
  // 1-3s (Rejection)
  if (lastAbsZ >= 3) {
    violations.push({ rule:'1-3s', idx:n-1, type:'rejection', desc:'1 nilai melampaui ±3SD' });
  }

  // 2-2s (Within-Run): 2 nilai berturut >= +2SD atau <= -2SD
  if (n >= 2) {
    var z1 = z(v[n-2]), z2 = z(v[n-1]);
    if ((z1 >= 2 && z2 >= 2) || (z1 <= -2 && z2 <= -2)) {
      violations.push({ rule:'2-2s', idx:n-1, type:'rejection', desc:'2 nilai berturut > ±2SD sisi sama
(Within)' });
    }
  }

  // R-4s (Within-Run)
  if (n >= 2) {
    var z1 = z(v[n-2]), z2 = z(v[n-1]);
    if ((z1 >= 2 && z2 <= -2) || (z1 <= -2 && z2 >= 2) || Math.abs(z1 - z2) >= 4) {
      violations.push({ rule:'R-4s', idx:n-1, type:'rejection', desc:'Rentang 2 nilai berturut > 4SD
(Within)' });
    }
  }

  // 4-1s (Within-Run)
  if (n >= 4) {
    var last4 = v.slice(n-4).map(z);
    if (last4.every(function(zv){return zv >= 1;}) || last4.every(function(zv){return zv <= -1;})) {
      violations.push({ rule:'4-1s', idx:n-1, type:'rejection', desc:'4 nilai berturut > ±1SD sisi sama
(Within)' });
    }
  }

  // 6x, 7x, 8x, 10x
  if (n >= 6) {
    var last6 = v.slice(n-6);
    if (last6.every(function(x){return x > mean;}) || last6.every(function(x){return x < mean;}))
      violations.push({ rule:'6x', idx:n-1, type:'rejection', desc:'6 nilai berturut di satu sisi mean
(Within)' });
  }
  if (n >= 7) {
    var last7 = v.slice(n-7);
    if (last7.every(function(x){return x > mean;}) || last7.every(function(x){return x < mean;}))
      violations.push({ rule:'7x', idx:n-1, type:'rejection', desc:'7 nilai berturut di satu sisi mean
(Within)' });
```

```
  }
  if (n >= 8) {
    var last8 = v.slice(n-8);
    if (last8.every(function(x){return x > mean;}) || last8.every(function(x){return x < mean;}))
      violations.push({ rule:'8x', idx:n-1, type:'rejection', desc:'8 nilai berturut di satu sisi mean
(Within)' });
  }
  if (n >= 10) {
    var last10 = v.slice(n-10);
    if (last10.every(function(x){return x > mean;}) || last10.every(function(x){return x < mean;}))
      violations.push({ rule:'10x', idx:n-1, type:'rejection', desc:'10 nilai berturut di satu sisi mean
(Within)' });
  }

  // 7T
  if (n >= 7) {
    var last7 = v.slice(n-7);
    var allInc = true, allDec = true;
    for (var i = 1; i < 7; i++) {
      if (last7[i] <= last7[i-1]) allInc = false;
      if (last7[i] >= last7[i-1]) allDec = false;
    }
    if (allInc || allDec) violations.push({ rule:'7T', idx:n-1, type:'rejection', desc:'7 nilai trend
'+(allInc?'naik':'turun')+' (Within)' });
  }

  return violations;
}
function checkWestgardAcrossLevels(ljDataUnified, meansByLevel, sdsByLevel) {
  var violations = [];
  var dataByDate = {};

  // Kelompokkan data berdasarkan tanggal
  ['L1', 'L2', 'L3'].forEach(function(lv) {
    (ljDataUnified[lv] || []).forEach(function(p, idx) {
      if (!dataByDate[p.date]) dataByDate[p.date] = [];
      dataByDate[p.date].push({ lv: lv, value: p.value, idx: idx });
    });
  });

  Object.keys(dataByDate).forEach(function(dateStr) {
    var dayData = dataByDate[dateStr];
    if (dayData.length < 2) return;

    for (var i = 0; i < dayData.length - 1; i++) {
```

```
        for (var j = i + 1; j < dayData.length; j++) {
          var d1 = dayData[i], d2 = dayData[j];
          if (!meansByLevel[d1.lv] || !sdsByLevel[d1.lv] || !meansByLevel[d2.lv] ||
!sdsByLevel[d2.lv]) continue;

          var z1 = (d1.value - meansByLevel[d1.lv]) / sdsByLevel[d1.lv];
          var z2 = (d2.value - meansByLevel[d2.lv]) / sdsByLevel[d2.lv];

          // 2-2s Across
          if ((z1 >= 2 && z2 >= 2) || (z1 <= -2 && z2 <= -2)) {
            violations.push({
              rule: '2-2s(across)',
              date: dateStr,
              levels: [d1.lv, d2.lv],
              indices: [d1.idx, d2.idx],
              type: 'rejection',
              desc: '2 level (' + d1.lv + ' & ' + d2.lv + ') > ±2SD sisi sama (Across)'
            });
          }
          // R-4s Across
          if (Math.abs(z1 - z2) >= 4) {
            violations.push({
              rule: 'R-4s(across)',
              date: dateStr,
              levels: [d1.lv, d2.lv],
              indices: [d1.idx, d2.idx],
              type: 'rejection',
              desc: 'Selisih Z ≥4 antar level (' + d1.lv + ' & ' + d2.lv + ') (Across)'
            });
          }
        }
      }
    });
    return violations;
}
function getActiveRulesBySigma(sigma) {
  if (sigma === null || sigma === undefined || isNaN(sigma)) {
    return { rules: ['1-3s','2-2s','R-4s','4-1s','6x','10x'], mode: 'Sigma N/A (semua aturan aktif)',
warning: '' };
  }
  var s = parseFloat(sigma);
  if (s >= 6) return { rules: ['1-3s'], mode: 'Sigma ≥6 (World Class) — hanya 1-3s', warning: '' };
  if (s >= 4) return { rules: ['1-3s','2-2s','R-4s'], mode: 'Sigma 4-6 (High Quality) — 1-3s, 2-2s, R-4s',
warning: '' };
```

```
  if (s >= 3) return { rules: ['1-3s','2-2s','R-4s','4-1s'], mode: 'Sigma 3-4 (Low Quality) — 1-3s, 2-2s, R-4s, 4-1s', warning: '' };
  return { rules: ['1-3s','2-2s','R-4s','4-1s','6x','10x'], mode: 'Sigma <3 (Unreliable) — Multirule', warning: 'Perbaikan Presisi/Metode Diperlukan!' };
}

function filterViolationsBySigma(violations, sigma) {
  var info = getActiveRulesBySigma(sigma);
  var activeRules = info.rules;
  return violations.map(function(v) {
    var ruleBase = v.rule.replace('(across)', '');
    var isActive = activeRules.indexOf(ruleBase) > -1;
    return Object.assign({}, v, { sigmaActive: isActive, ignored: !isActive && v.type === 'rejection' });
  });
}

function categorizeWestgardError(rule) {
  var r = String(rule).replace('(across)', '');
  if (r === '1-2s') return { category: 'Warning', desc: 'Peringatan — tidak menolak run, perlu pengamatan tambahan' };
  if (r === '1-3s') return { category: 'Random Error', desc: 'Random Error — pengukuran terisolasi melampaui batas' };
  if (r === 'R-4s') return { category: 'Random Error', desc: 'Random Error — variabilitas dalam-run' };
  if (r === '2-2s') return { category: 'Systematic Error', desc: 'Systematic Shift — pergeseran sistematik' };
  if (r === '4-1s') return { category: 'Systematic Error', desc: 'Systematic Shift — pergeseran kecil konsisten' };
  if (r === '6x' || r === '8x' || r === '10x') return { category: 'Systematic Error', desc: 'Systematic Shift — pergeseran mean' };
  if (r === '7T') return { category: 'Systematic Error', desc: 'Systematic Trend — drift gradual' };
  return { category: 'Lainnya', desc: 'Pelanggaran lain' };
}

function computeSigmaForLevel(lot, level, qcData) {
  var lv = parseInt(String(level).replace('L', ''));
  var m = parseNumSafe(lot['meanL'+lv]), s = parseNumSafe(lot['sdL'+lv]), tea = parseNumSafe(lot.tea);
  if (!m || !s || !tea) return null;
  var vals = qcData.map(function(q) { return parseNumSafe(lv===1 ? q.level1 : lv===2 ? q.level2 : q.level3); }).filter(function(v) { return v !== null && v !== 0; });
  if (!vals.length) return null;
  var calcMean = vals.reduce(function(a,b){return a+b;},0) / vals.length;
  var bias = Math.abs((calcMean - m) / m * 100);
  var cv = s / m * 100;
  if (!cv) return null;
```

```
  return parseFloat(((tea - bias) / cv).toFixed(2));
}

function getWestgardViolations30Days(ownerUsername, role) {
  var now = new Date();
  var d30 = new Date(now.getTime() - 30*86400000);
  var qcData = getInputQC(ownerUsername, role, { startDate: dateToISO(d30), endDate:
dateToISO(now) });
  var lotMap = {};
  gSD(SH.LOTQC).forEach(function(r) { lotMap[r[0]] = r; });
  var grouped = {};
  qcData.forEach(function(q) {
    if (!grouped[q.lotID]) grouped[q.lotID] = { qcs: [], lot: lotMap[q.lotID], param: q.parameter };
    grouped[q.lotID].qcs.push(q);
  });
  var allViolations = [];
  Object.keys(grouped).forEach(function(lotID) {
    var g = grouped[lotID]; var lot = g.lot; if (!lot) return;
    var lotObj = { meanL1:lot[8], sdL1:lot[9], meanL2:lot[11], sdL2:lot[12], meanL3:lot[14],
sdL3:lot[15], tea:lot[17] };
    var qcs = g.qcs.sort(function(a,b){return new Date(a.tanggal)-new Date(b.tanggal);});
    [1,2,3].forEach(function(lv) {
      var m = parseNumSafe(lotObj['meanL'+lv]), s = parseNumSafe(lotObj['sdL'+lv]);
      if (!m||!s) return;
      var vals = qcs.map(function(q) { return parseNumSafe(lv===1?q.level1:lv===2?q.level2:q.level3);
}).filter(function(v){return v!==null&&v!==0;});
      if (!vals.length) return;
      var sigma = computeSigmaForLevel(lotObj, 'L'+lv, qcs);
      var viol = checkWestgardRules(vals, m, s);
      viol = filterViolationsBySigma(viol, sigma);
      viol.forEach(function(v) {
        if (!v.ignored) {
          var cat = categorizeWestgardError(v.rule);
          allViolations.push({
            parameter: g.param, lotID: lotID, namaAlat: lot[3],
            level: 'L'+lv, rule: v.rule, desc: v.desc,
            tanggal: qcs.length ? qcs[qcs.length-1].tanggal : '',
            sigma: sigma, category: cat.category, categoryDesc: cat.desc
          });
        }
      });
    });
  });
  return allViolations;
}
```

```
function checkAndNotifyWestgard(paramID, lotID, ownerUsername, newQCID) {
  try {
    var user = getUserByUsername(ownerUsername);
    if (!user || !user.email) return;
    var lot = getLotByID(lotID, ownerUsername); if (!lot) return;
    var param = getParamByID(paramID);
    var now = new Date(); var d14 = new Date(now.getTime() - 14*86400000);
    var qcData = getInputQC(ownerUsername, 'user', { lotID: lotID, startDate: dateToISO(d14),
endDate: dateToISO(now) });
    qcData.sort(function(a,b){return new Date(a.tanggal)-new Date(b.tanggal);});
    var violations = [];
    [1,2,3].forEach(function(lv) {
      var mv = parseNumSafe(lot['meanL'+lv]), sv = parseNumSafe(lot['sdL'+lv]);
      if (!mv||!sv) return;
      var vals = qcData.map(function(q) { return
parseNumSafe(lv===1?q.level1:lv===2?q.level2:q.level3); }).filter(function(v){return
v!==null&&v!==0;});
      if (!vals.length) return;
      var sigma = computeSigmaForLevel(lot, 'L'+lv, qcData);
      var viol = checkWestgardRules(vals, mv, sv);
      viol = filterViolationsBySigma(viol, sigma);
      viol.filter(function(v){return !v.ignored && v.type==='rejection';}).forEach(function(v) {
        violations.push('L'+lv+': '+v.rule+' — '+v.desc+' (σ='+sigma+')');
      });
    });
    if (violations.length) {
      var kop = getKopSurat(ownerUsername);
      MailApp.sendEmail({
        to: user.email,
        subject: '[didiQCsys] □ Pelanggaran Westgard: '+(param?param.parameter:paramID),
        htmlBody: '<h3>Pelanggaran Westgard</h3><p>'+(kop.namaRS||'Lab')+' —
'+(param?param.parameter:paramID)+'</p><ul>'+
          violations.map(function(v){return '<li>'+v+'</li>';}).join('')+'</ul>'
      });
    }
  } catch (e) {}
}

// ── DASHBOARD ──────────────────────────────────────────────
function getDashboardData(ownerUsername, role) {
  try {
    var now = new Date(), todayStr = dateToISO(now);
    var params = getParameters(ownerUsername, role);
    var lots = getLotQC(ownerUsername, role);
```

```
    var allQC = getInputQC(ownerUsername, role, {});
    var todayQC = allQC.filter(function(q){return q.tanggal===todayStr;});
    var validated = allQC.filter(function(q){return q.validated;});
    var pending = allQC.filter(function(q){return !q.validated;});
    var expired = lots.filter(function(l){if(!l.expiredDate)return false;var
d=parseDateStr(l.expiredDate);return d&&d<now;});
    var d30 = new Date(now.getTime()+30*86400000);
    var nearExpiry = lots.filter(function(l){if(!l.expiredDate)return false;var
d=parseDateStr(l.expiredDate);return d&&d>=now&&d<=d30;});
    var wgViolations = getWestgardViolations30Days(ownerUsername, role);
    var sigmaByBidang = computeSigmaByBidang(ownerUsername, role, params, lots, allQC);
    var cvBiasByBidang = computeCVBiasByBidang(ownerUsername, role, lots, allQC);
    var trendDetail = computeMonthTrend(ownerUsername, role, lots, allQC, params);
    var d7 = new Date(now.getTime()-7*86400000), d30b = new Date(now.getTime()-30*86400000);
    var weeklyQC = allQC.filter(function(q){return parseDateStr(q.tanggal)>=d7;}).length;
    var monthlyQC = allQC.filter(function(q){return parseDateStr(q.tanggal)>=d30b;}).length;
    return sanitizeReturn({
      ok: true,
      stats: { totalParam:params.length, totalLot:lots.length, totalQC:allQC.length,
        todayQC:todayQC.length, validated:validated.length, pending:pending.length,
        expired:expired.length, nearExpiry:nearExpiry.length },
      wgViolations: wgViolations, sigmaByBidang: sigmaByBidang,
      cvBiasByBidang: cvBiasByBidang, trendDetail: trendDetail,
      weeklyQC: weeklyQC, monthlyQC: monthlyQC
    });
  } catch (e) { return { ok: false, msg: e.message }; }
}

function computeSigmaByBidang(ownerUsername, role, params, lots, allQC) {
  var paramBidang = {};
  params.forEach(function(p){paramBidang[p.paramID]=p.bidang||'Lainnya';});
  var bidangData = {};
  lots.forEach(function(lot) {
    var bidang = paramBidang[lot.paramID] || 'Lainnya';
    if (!bidangData[bidang]) bidangData[bidang] = { sigL1:[], sigL2:[], sigL3:[] };
    var lotQCs = allQC.filter(function(q){return q.lotID===lot.lotID;});
    [1,2,3].forEach(function(lv) {
      var sigma = computeSigmaForLevel(lot, 'L'+lv, lotQCs);
      if (sigma !== null) bidangData[bidang]['sigL'+lv].push(sigma);
    });
  });
  var result = {};
  Object.keys(bidangData).forEach(function(b) {
    result[b] = {};
    [1,2,3].forEach(function(lv) {
```

```
    var arr = bidangData[b]['sigL'+lv];
    result[b]['sigL'+lv] = arr.length ? parseFloat((arr.reduce(function(a,c){return
a+c;},0)/arr.length).toFixed(2)) : null;
   });
  });
  return result;
}

function computeCVBiasByBidang(ownerUsername, role, lots, allQC) {
  var paramData = gSD(SH.PARAMS); var paramBidang = {};
  paramData.forEach(function(r){paramBidang[r[0]]=r[5]||'Lainnya';});
  var bidangData = {};
  lots.forEach(function(lot) {
   var bidang = paramBidang[lot.paramID] || 'Lainnya';
   if (!bidangData[bidang]) bidangData[bidang] = { cvs:[], biases:[] };
   var m1 = parseNumSafe(lot.meanL1), s1 = parseNumSafe(lot.sdL1);
   if (m1 && s1) bidangData[bidang].cvs.push(s1/m1*100);
   var bias = parseNumSafe(lot.biasPct);
   if (bias !== null) bidangData[bidang].biases.push(Math.abs(bias));
  });
  var result = {};
  Object.keys(bidangData).forEach(function(b) {
   var cvs = bidangData[b].cvs, bias = bidangData[b].biases;
   result[b] = {
    avgCV: cvs.length ? parseFloat((cvs.reduce(function(a,c){return a+c;},0)/cvs.length).toFixed(2)) :
null,
    avgBias: bias.length ? parseFloat((bias.reduce(function(a,c){return
a+c;},0)/bias.length).toFixed(2)) : null
   };
  });
  return result;
}

function computeMonthTrend(ownerUsername, role, lots, allQC, params) {
  var now = new Date();
  var startOfMonth = new Date(now.getFullYear(), now.getMonth(), 1);
  var lotMap = {}, paramMap = {}, paramBidang = {};
  lots.forEach(function(l){lotMap[l.lotID]=l;});

params.forEach(function(p){paramMap[p.paramID]=p;paramBidang[p.paramID]=p.bidang||'Lainnya'
;});
  var monthQC = allQC.filter(function(q){var d=parseDateStr(q.tanggal);return
d&&d>=startOfMonth&&d<=now;});
  var byParam = {};
```

```
monthQC.forEach(function(q){if(!byParam[q.paramID])byParam[q.paramID]=[];byParam[q.paramID].
push(q);});
  var result = {};
  Object.keys(byParam).forEach(function(paramID) {
    var qcs = byParam[paramID].sort(function(a,b){return new Date(a.tanggal)-new Date(b.tanggal);});
    var param = paramMap[paramID]; var bidang = paramBidang[paramID]||'Lainnya';
    if (!result[bidang]) result[bidang] = [];
    var lotGroups = {};
    qcs.forEach(function(q){if(!lotGroups[q.lotID])lotGroups[q.lotID]=[];lotGroups[q.lotID].push(q);});
    Object.keys(lotGroups).forEach(function(lotID) {
      var lot = lotMap[lotID]; if (!lot) return;
      var lqcs = lotGroups[lotID];
      var summary = { paramID:paramID, parameter:param?param.parameter:'',
namaAlat:lot.namaAlat, noLot:lot.noLot, levels:{} };
      [1,2,3].forEach(function(lv) {
        var m = parseNumSafe(lot['meanL'+lv]), s = parseNumSafe(lot['sdL'+lv]), tea =
parseNumSafe(lot.tea);
        var vals = lqcs.map(function(q){return
parseNumSafe(lv===1?q.level1:lv===2?q.level2:q.level3);}).filter(function(v){return
v!==null&&v!==0;});
        if (!vals.length||!m||!tea) { summary.levels['L'+lv] = null; return; }
        var calcMean = vals.reduce(function(a,b){return a+b;},0)/vals.length;
        var calcSD = vals.length > 1 ? Math.sqrt(vals.reduce(function(s2,v){return s2+Math.pow(v-
calcMean,2);},0)/(vals.length)) : 0;
        var cv = calcMean ? (calcSD/calcMean*100) : 0, bias = m ? (calcMean-m)/m*100 : 0;
        var te = Math.abs(bias) + 1.65*cv;
        var sigma = tea && cv ? (tea-Math.abs(bias))/cv : null;
        var wg = checkWestgardRules(vals, calcMean, calcSD);
        wg = filterViolationsBySigma(wg, sigma);
        var realViols = wg.filter(function(v){return !v.ignored && v.type==='rejection';}).length;
        summary.levels['L'+lv] = {
          n: vals.length, calcMean: parseFloat(calcMean.toFixed(3)),
          cv: parseFloat(cv.toFixed(2)), bias: parseFloat(bias.toFixed(2)),
          te: parseFloat(te.toFixed(2)), sigma: sigma ? parseFloat(sigma.toFixed(2)) : null,
          tea: tea, violations: realViols
        };
      });
      result[bidang].push(summary);
    });
  });
  return result;
}

// ── GRAPH DATA ──────────────────────────────────────────────
```

```
function getGraphData(payload, ownerUsername, role) {
  try {
    var paramID = payload.paramID, lotID = payload.lotID;
    var sumber = payload.sumber || 'Manufaktur';
    var filter = { paramID: paramID, lotID: lotID, startDate: payload.startDate, endDate:
payload.endDate };
    var qcData = getInputQC(ownerUsername, role, filter);
    qcData.sort(function(a, b) {
      var dA = new Date(a.tanggal), dB = new Date(b.tanggal);
      if (dA.getTime() !== dB.getTime()) return dA - dB;
      var idA = new Date(a.inputDate || 0).getTime();
      var idB = new Date(b.inputDate || 0).getTime();
      return idA - idB;
    });
    var lot = getLotByID(lotID, ownerUsername);
    var param = getParamByID(paramID);
    if (!lot) return { ok: false, msg: 'Lot tidak ditemukan' };
    var ljDataUnified = { L1: [], L2: [], L3: [] };
    qcData.forEach(function(q, qIdx) {
      ['L1', 'L2', 'L3'].forEach(function(lv) {
        var lvNum = parseInt(lv.replace('L',''));
        var val = parseNumSafe(lvNum===1?q.level1:lvNum===2?q.level2:q.level3);
        if (val !== null && val !== 0) {
          ljDataUnified[lv].push({ idx: qIdx, date: q.tanggal, value: val, qcID: q.qcID, catatanValidasi:
q.catatanValidasi || '' });
        }
      });
    });
    var levelMeta = {};
    var valsByLevel = {}, meansByLevel = {}, sdsByLevel = {};
    [1, 2, 3].forEach(function(lv) {
      var msInfo = getMeanSDForLevel(lot, lv, sumber, qcData, ownerUsername);
      var m = msInfo.mean, s = msInfo.sd;
      var vals = ljDataUnified['L'+lv].map(function(p) { return p.value; });
      meansByLevel['L'+lv] = m; sdsByLevel['L'+lv] = s; valsByLevel['L'+lv] = vals;
      var sigma = computeSigmaForLevel(lot, 'L'+lv, qcData);
      var wgMap = {};
      vals.forEach(function(v, i) {
        var subset = vals.slice(0, i+1);
        var viol = checkWestgardRules(subset, m, s);
        viol.forEach(function(vi) {
          var cat = categorizeWestgardError(vi.rule);
          vi.category = cat.category; vi.categoryDesc = cat.desc;
        });
        wgMap[i] = viol;
```

```
      });
      var ruleInfo = getActiveRulesBySigma(sigma);
      levelMeta['L'+lv] = {
        mean: m, sd: s, tea: parseNumSafe(lot.tea),
        target: parseNumSafe(lot['targetL'+lv]),
        westgard: wgMap, sigma: sigma,
        activeRules: ruleInfo.rules, mode: ruleInfo.mode,
        sumberInfo: msInfo.source
      };
    });
  var wgAcrossRaw = checkWestgardAcrossLevels(ljDataUnified, meansByLevel, sdsByLevel);
var wgAcross = [];

// INJEKSI pelanggaran Across ke dalam levelMeta agar muncul sebagai marker di grafik LJ!
wgAcrossRaw.forEach(function(v) {
  var cat = categorizeWestgardError(v.rule);
  v.category = cat.category;
  v.categoryDesc = cat.desc;
  wgAcross.push(v); // Simpan untuk panel teks

  // Injeksi ke setiap level yang terlibat (misal L1 dan L2)
  v.levels.forEach(function(lv, idx) {
    var ptIdx = v.indices[idx];
    if (levelMeta[lv]) {
      if (!levelMeta[lv].westgard[ptIdx]) levelMeta[lv].westgard[ptIdx] = [];
      var exists = levelMeta[lv].westgard[ptIdx].some(function(ex) { return ex.rule === v.rule; });
      if (!exists) {
        levelMeta[lv].westgard[ptIdx].push({
          rule: v.rule,
          type: v.type,
          desc: v.desc,
          category: v.category,
          categoryDesc: v.categoryDesc
        });
      }
    }
  });
});

var worstSigma = Math.min.apply(null, [1,2,3].map(function(lv) {
  return levelMeta['L'+lv].sigma === null ? 6 : levelMeta['L'+lv].sigma;
}));
wgAcross = filterViolationsBySigma(wgAcross, worstSigma);
wgAcross.forEach(function(v) {
  var cat = categorizeWestgardError(v.rule);
```

```
    v.category = cat.category; v.categoryDesc = cat.desc;
});
    var stats = computeQCStats(qcData, lot);
    var qgiData = {};
    [1,2,3].forEach(function(lv) {
      var s = stats['L'+lv];
      if (!s || !s.cv) { qgiData['L'+lv] = null; return; }
      var qgi = s.cv ? s.bias / (1.5 * s.cv) : null;
      qgiData['L'+lv] = {
        qgi: qgi !== null ? parseFloat(qgi.toFixed(3)) : null,
        interp: qgi === null ? '' : (qgi > 1.2 ? 'Imprecision dominant — perbaiki presisi' : qgi < 0.8 ?
'Inaccuracy dominant — perbaiki akurasi/kalibrasi' : 'Balanced — perbaiki keduanya')
      };
    });
    return sanitizeReturn({
      ok: true, parameter: param ? param.parameter : '', lotID: lotID,
      namaAlat: lot.namaAlat, noLot: lot.noLot, satuan: lot.satuan,
      methode: lot.methode, expiredDate: dateToISO(lot.expiredDate),
      tea: lot.tea, sumber: sumber, sumberLot: lot.sumber,
      ljDataUnified: ljDataUnified, levelMeta: levelMeta,
      stats: stats, qgiData: qgiData, wgAcross: wgAcross,
      startDate: payload.startDate, endDate: payload.endDate
    });
  } catch (e) { return { ok: false, msg: e.message }; }
}

// ── HELPER: SUMBER MEAN/SD ──────────────────────────────────
function getMeanSDForLevel(lot, lv, sumber, qcData, ownerUsername) {
  var lvNum = parseInt(String(lv).replace('L',''));
  var lotMean = parseNumSafe(lot['meanL'+lvNum]);
  var lotSD = parseNumSafe(lot['sdL'+lvNum]);
  if (!sumber || sumber === 'Manufaktur') return { mean: lotMean, sd: lotSD, source: 'Manufaktur' };
  if (sumber === 'Terhitung berjalan') {
    var vals = (qcData||[]).map(function(q) { return
parseNumSafe(lvNum===1?q.level1:lvNum===2?q.level2:q.level3); }).filter(function(v) { return v !==
null && v !== 0; });
    if (vals.length < 2) return { mean: lotMean, sd: lotSD, source: 'Manufaktur (data kurang)' };
    var mean = vals.reduce(function(a,b){return a+b;},0) / vals.length;
    var sdv = Math.sqrt(vals.reduce(function(s,v){return s+Math.pow(v-mean,2);},0) / (vals.length-1));
    return { mean: parseFloat(mean.toFixed(4)), sd: parseFloat(sdv.toFixed(4)), source: 'Terhitung
berjalan (N='+vals.length+')' };
  }
  if (sumber === 'Terhitung Fix') {
    var calcData = gSD(SH.CALCSTATS);
    var matched = null;
```

```
   for (var i = calcData.length-1; i >= 0; i--) {
     if (calcData[i][2] === lot.lotID && (calcData[i][3] === 'L'+lvNum || calcData[i][3] ===
String(lvNum)) &&
         String(calcData[i][10]).toLowerCase() === String(ownerUsername).toLowerCase()) { matched =
calcData[i]; break; }
   }
   if (matched) {
     var cm = parseNumSafe(matched[4]), cs = parseNumSafe(matched[5]);
     if (cm && cs) return { mean: cm, sd: cs, source: 'Terhitung Fix (N='+matched[7]+')' };
   }
   return { mean: lotMean, sd: lotSD, source: 'Manufaktur (CalcStats tidak ditemukan)' };
  }
  return { mean: lotMean, sd: lotSD, source: 'Manufaktur' };
}

// ── HELPER: SMALLEST SIGMA BY SOURCE (DIPERBAIKI v9.6) ────
function getSmallestSigmaBySrc(paramID, lotID, sigmaSource, ownerUsername, lot, qcData,
filterOpts) {
filterOpts = filterOpts || {};
var tea = parseNumSafe(lot.tea);
if (!tea) return null;

if (sigmaSource === 'Sigma Terkecil Terhitung') {
   var sigmas = [];
   [1,2,3].forEach(function(lv) {
      var m = parseNumSafe(lot['meanL'+lv]); // Target mean (untuk hitung Bias)
      var tea = parseNumSafe(lot.tea);
      if (!m || !tea) return;

      var vals = (qcData||[]).map(function(q){ return
parseNumSafe(lv===1?q.level1:lv===2?q.level2:q.level3); }).filter(function(v){return
v!==null&&v!==0;});
      if (!vals.length) return;

      // PERBAIKAN: Gunakan SD dan Mean Terhitung (Observasi) agar sinkron dengan Tabel Statistik
      var calcMean = vals.reduce(function(a,b){return a+b;},0) / vals.length;
      var calcSD = Math.sqrt(vals.reduce(function(s2,v){return s2+Math.pow(v-calcMean,2);},0) /
vals.length);
      var calcCV = calcMean ? (calcSD / calcMean * 100) : 0;
      var bias = m ? ((calcMean - m) / m * 100) : 0;

      if (calcCV) {
         var sigma = (tea - Math.abs(bias)) / calcCV;
         sigmas.push(parseFloat(sigma.toFixed(2)));
      }
```

```
  });
  return sigmas.length ? Math.min.apply(null, sigmas) : null;
}

if (sigmaSource === 'Sigma Terkecil PME') {
var pmeFilter = { paramID: paramID, lotID: lotID };
if (filterOpts.siklusPME) pmeFilter.siklus = filterOpts.siklusPME;
if (filterOpts.tahunSiklus) pmeFilter.tahun = filterOpts.tahunSiklus;
var pmeData = getBiasPME(ownerUsername, 'user', pmeFilter);
var sigmas2 = [];
pmeData.forEach(function(pme) {
[1,2,3].forEach(function(lv) {
var d = pme.details['L'+lv];
if (!d || d.sigma === null) return;
sigmas2.push(d.sigma);
});
});
return sigmas2.length ? Math.min.apply(null, sigmas2) : null;
}

if (sigmaSource === 'Sigma Terkecil PME CV') {
var csFilter = { paramID: paramID };
var csData = getCalcStats(ownerUsername, 'user', csFilter);
csData = csData.filter(function(cs) { return cs.lotID === lotID; });
if (filterOpts.periodeCS && filterOpts.periodeCS.start && filterOpts.periodeCS.end) {
var pStart = parseDateStr(filterOpts.periodeCS.start);
var pEnd = parseDateStr(filterOpts.periodeCS.end);
csData = csData.filter(function(cs) {
var csStart = parseDateStr(cs.startDate);
var csEnd = parseDateStr(cs.endDate);
return csStart && csEnd && csStart.getTime() === pStart.getTime() && csEnd.getTime() ===
pEnd.getTime();
});
}
var sigmas3 = [];
csData.forEach(function(cs) {
if (cs.sigma !== null && cs.sigma !== undefined) sigmas3.push(cs.sigma);
});
return sigmas3.length ? Math.min.apply(null, sigmas3) : null;
}

if (sigmaSource === 'Sigma Terkecil CV Optional') {
  var cvFilter = { paramID: paramID };
  var cvData = getSigmaCVOpt(ownerUsername, 'user', cvFilter);
  cvData = cvData.filter(function(cv) { return cv.lotID === lotID; });
```

```
  // Filter by periode jika ada
  if (filterOpts.periodeCVOpt && filterOpts.periodeCVOpt.start && filterOpts.periodeCVOpt.end) {
    var pStart2 = parseDateStr(filterOpts.periodeCVOpt.start);
    var pEnd2 = parseDateStr(filterOpts.periodeCVOpt.end);
    cvData = cvData.filter(function(cv) {
      var cvStart = parseDateStr(cv.startDate);
      var cvEnd = parseDateStr(cv.endDate);
      if (!cvStart || !cvEnd || !pStart2 || !pEnd2) return false;
      return cvStart.getTime() === pStart2.getTime() && cvEnd.getTime() === pEnd2.getTime();
    });
  }

  var sigmas4 = [];
  cvData.forEach(function(r) {
    // PRIORITAS: Ambil sigma dari details per-level (L1, L2, L3) yang sudah dihitung dengan Bias PME
    [1,2,3].forEach(function(lv) {
      var d = r.details['L'+lv];
      if (d && d.sigma !== null && d.sigma !== undefined && !isNaN(d.sigma)) {
        sigmas4.push(d.sigma);
      }
    });
    // FALLBACK: Jika details kosong, gunakan avgSigma dari database
    if (sigmas4.length === 0) {
      var s = parseNumSafe(r.avgSigma);
      if (s !== null && !isNaN(s)) sigmas4.push(s);
      var s12 = parseNumSafe(r.avgSigmaL12);
      if (s12 !== null && !isNaN(s12)) sigmas4.push(s12);
    }
  });
  return sigmas4.length ? Math.min.apply(null, sigmas4) : null;
}

return null;
}
function getSigmaBasedGraphData(payload, ownerUsername, role) {
  try {
    var graphRes = getGraphData(payload, ownerUsername, role);
    if (!graphRes.ok) return graphRes;
    var sigmaSource = payload.sigmaSource || 'Sigma Terkecil Terhitung';
    var lot = getLotByID(payload.lotID, ownerUsername);
    var qcData = getInputQC(ownerUsername, role, {
      paramID: payload.paramID, lotID: payload.lotID,
      startDate: payload.startDate, endDate: payload.endDate
    });
```

```
    var filterOpts = {
      siklusPME: payload.siklusPME || null,
      tahunSiklus: payload.tahunSiklus || null,
      periodeCS: payload.periodeCS || null,
      periodeCVOpt: payload.periodeCVOpt || null
    };
    var smallestSigma = getSmallestSigmaBySrc(payload.paramID, payload.lotID, sigmaSource,
ownerUsername, lot, qcData, filterOpts);
    var ruleInfo = getActiveRulesBySigma(smallestSigma);
    ['L1','L2','L3'].forEach(function(lv) {
      var meta = graphRes.levelMeta[lv];
      if (!meta) return;
      var newWgMap = {};
      Object.keys(meta.westgard).forEach(function(i) {
        var viol = meta.westgard[i] || [];
        newWgMap[i] = filterViolationsBySigma(viol, smallestSigma);
        newWgMap[i].forEach(function(vi) {
          var cat = categorizeWestgardError(vi.rule);
          vi.category = cat.category; vi.categoryDesc = cat.desc;
        });
      });
      meta.westgard = newWgMap;
      meta.activeRules = ruleInfo.rules;
      meta.mode = ruleInfo.mode;

// PERBAIKAN: Update sigma di levelMeta dengan smallestSigma agar konsisten
if (smallestSigma !== null && smallestSigma !== undefined) {
  meta.sigma = smallestSigma;
}
    });
    graphRes.sigmaBasedActive = true;
    graphRes.smallestSigma = smallestSigma;
    graphRes.sigmaSource = sigmaSource;
    graphRes.activeRulesMode = ruleInfo.mode;
    graphRes.warning = ruleInfo.warning;
    graphRes.activeRules = ruleInfo.rules;
    return sanitizeReturn(graphRes);
  } catch (e) { return { ok: false, msg: e.message }; }
}

function computeQCStats(qcData, lot) {
  var stats = {};
  [1,2,3].forEach(function(lv) {
    var m = parseNumSafe(lot['meanL'+lv]), s = parseNumSafe(lot['sdL'+lv]), tea =
parseNumSafe(lot.tea);
```

```
    var vals = qcData.map(function(q) {
      return parseNumSafe(lv===1?q.level1:lv===2?q.level2:q.level3);
    }).filter(function(v){return v!==null && v!==0;});
    if (!vals.length) { stats['L'+lv] = null; return; }
    var calcMean = vals.reduce(function(a,b){return a+b;},0) / vals.length;
    var calcSD = Math.sqrt(vals.reduce(function(s2,v){return s2+Math.pow(v-
calcMean,2);},0)/vals.length);
    var calcCV = calcMean ? calcSD/calcMean*100 : 0;
    var bias = m ? (calcMean-m)/m*100 : 0;
    var unc = calcCV / Math.sqrt(vals.length);
    var te = Math.abs(bias) + 1.65*calcCV;
    var sigma = tea && calcCV ? (tea-Math.abs(bias))/calcCV : null;
    stats['L'+lv] = {
      n: vals.length, mean: parseFloat(calcMean.toFixed(4)), sd: parseFloat(calcSD.toFixed(4)),
      cv: parseFloat(calcCV.toFixed(2)), bias: parseFloat(bias.toFixed(2)),
      unc: parseFloat(unc.toFixed(4)), te: parseFloat(te.toFixed(2)),
      sigma: sigma ? parseFloat(sigma.toFixed(2)) : null,
      targetMean: m, targetSD: s, tea: tea
    };
  });
  return stats;
}

// ── LAPORAN (DIPERBAIKI v9.6 - Data QC di atas Statistik) ──
function getLaporanData(payload, ownerUsername, role) {
  try {
    var filter = {
      startDate: payload.startDate || null, endDate: payload.endDate || null,
      paramID: payload.paramID || null, lotID: payload.lotID || null,
      namaAlat: payload.namaAlat || null, bidang: payload.bidang || null
    };
    var params = getParameters(ownerUsername, role);
    var lots = getLotQC(ownerUsername, role);
    if (filter.bidang) params = params.filter(function(p){return (p.bidang||'Lainnya')===filter.bidang;});
    if (filter.paramID) params = params.filter(function(p){return p.paramID===filter.paramID;});
    var paramIDs = params.map(function(p){return p.paramID;});
    lots = lots.filter(function(l){return paramIDs.indexOf(l.paramID)>-1;});
    if (filter.lotID) lots = lots.filter(function(l){return l.lotID===filter.lotID;});
    if (filter.namaAlat) lots = lots.filter(function(l){
      return String(l.namaAlat).toLowerCase().indexOf(String(filter.namaAlat).toLowerCase())>-1;
    });
    var result = [];
    var paramMap = {}; params.forEach(function(p){paramMap[p.paramID]=p;});
    lots.forEach(function(lot) {
      var qcFilter = { lotID: lot.lotID };
```

```
    if (filter.startDate) qcFilter.startDate = filter.startDate;
    if (filter.endDate) qcFilter.endDate = filter.endDate;
    var qcs = getInputQC(ownerUsername, role, qcFilter);
    if (!qcs.length) return;
    qcs.sort(function(a,b){return new Date(a.tanggal)-new Date(b.tanggal);});
    var stats = computeQCStats(qcs, lot);
    var interpretasi = {};
    [1,2,3].forEach(function(lv) {
      var s = stats['L'+lv];
      if (!s) { interpretasi['L'+lv] = null; return; }
      interpretasi['L'+lv] = buildLevelInterpretation(s, lot, qcs, lv);
    });
    var wgFlagsByQCID = {};
    [1,2,3].forEach(function(lv) {
      var m = parseNumSafe(lot['meanL'+lv]), sd = parseNumSafe(lot['sdL'+lv]);
      if (!m || !sd) return;
      var vals = qcs.map(function(q) { return parseNumSafe(lv===1?q.level1:lv===2?q.level2:q.level3);
});
      var sigma = stats['L'+lv] ? stats['L'+lv].sigma : null;
      qcs.forEach(function(q, idx) {
        if (vals[idx] === null || vals[idx] === 0) return;
        var subset = vals.slice(0, idx+1).filter(function(x){return x!==null && x!==0;});
        var viol = checkWestgardRules(subset, m, sd);
        viol = filterViolationsBySigma(viol, sigma);
        var rejs = viol.filter(function(v){return !v.ignored && v.type==='rejection';});
        if (rejs.length) {
          if (!wgFlagsByQCID[q.qcID]) wgFlagsByQCID[q.qcID] = {};
          wgFlagsByQCID[q.qcID]['L'+lv] = rejs.map(function(v) {
            var cat = categorizeWestgardError(v.rule);
            return { rule: v.rule, category: cat.category };
          });
        }
      });
    });
    result.push({
      paramID: lot.paramID,
      parameter: paramMap[lot.paramID] ? paramMap[lot.paramID].parameter : '',
      bidang: paramMap[lot.paramID] ? (paramMap[lot.paramID].bidang||'Lainnya') : 'Lainnya',
      lotID: lot.lotID, noLot: lot.noLot, namaAlat: lot.namaAlat,
      satuan: lot.satuan, methode: lot.methode, tea: lot.tea,
      nQC: qcs.length, stats: stats, qcData: qcs,
      interpretasi: interpretasi, wgFlags: wgFlagsByQCID,
      meanL1: lot.meanL1, sdL1: lot.sdL1,
      meanL2: lot.meanL2, sdL2: lot.sdL2,
      meanL3: lot.meanL3, sdL3: lot.sdL3
```

```
    });
   });
   var kop = getKopSurat(ownerUsername);
   return sanitizeReturn({ ok: true, data: result, kop: kop, filter: filter });
  } catch (e) { return { ok: false, msg: e.message }; }
}

function buildLevelInterpretation(s, lot, qcs, lv) {
  if (!s) return null;
  var notes = [];
  var status = 'baik';
  var statusColor = 'success';
  if (s.cv > 10) { notes.push('CV sangat tinggi ('+s.cv+'%) — periksa presisi alat'); status = 'buruk';
statusColor = 'danger'; }
  else if (s.cv > 5) { notes.push('CV tinggi ('+s.cv+'%) — pertimbangkan optimasi'); if (status === 'baik')
{ status = 'cukup'; statusColor = 'warning'; } }
  else { notes.push('CV baik ('+s.cv+'%)'); }
  if (Math.abs(s.bias) > 5) { notes.push('Bias signifikan ('+s.bias+'%) — periksa kalibrasi'); status =
'buruk'; statusColor = 'danger'; }
  else if (Math.abs(s.bias) > 2) { notes.push('Bias moderate ('+s.bias+'%)'); if (status === 'baik') { status
= 'cukup'; statusColor = 'warning'; } }
  else { notes.push('Bias rendah ('+s.bias+'%)'); }
  if (s.sigma !== null) {
    if (s.sigma >= 6) notes.push('Kinerja World Class (σ='+s.sigma+')');
    else if (s.sigma >= 5) notes.push('Kinerja Excellent (σ='+s.sigma+')');
    else if (s.sigma >= 4) notes.push('Kinerja Good (σ='+s.sigma+')');
    else if (s.sigma >= 3) { notes.push('Kinerja Marginal (σ='+s.sigma+') — perlu pemantauan'); if
(status === 'baik') { status = 'cukup'; statusColor = 'warning'; } }
    else { notes.push('Kinerja Poor (σ='+s.sigma+') — perbaikan diperlukan!'); status = 'buruk';
statusColor = 'danger'; }
  }
  if (s.tea && s.te > s.tea) {
    notes.push('Total Error ('+s.te+'%) melampaui TEa ('+s.tea+'%)!');
    status = 'buruk'; statusColor = 'danger';
  }
  return { status: status, statusColor: statusColor, notes: notes };
}

// ── TREND ANALISIS ──────────────────────────────────────────
function getTrendAnalisisData(payload, ownerUsername, role) {
try {
var paramID = payload.paramID || null;
var lotID = payload.lotID || null;
var bidang = payload.bidang || null;
var namaAlat = payload.namaAlat || null;
```

```
var tahun = payload.tahun || String(new Date().getFullYear());
var bulanAwal = parseInt(payload.bulanAwal) || 1;
var bulanAkhir = parseInt(payload.bulanAkhir) || 12;
var siklusPME = payload.siklusPME || null;
var tahunSiklus = payload.tahunSiklus || null;

// Generate months dari tahun + bulanAwal/bulanAkhir
var months = [];
var y = parseInt(tahun);
for (var m = bulanAwal; m <= bulanAkhir; m++) {
var monthEnd = new Date(y, m, 0);
months.push({
year: y, month: m,
label: m + '/' + y,
startISO: y + '-' + String(m).padStart(2, '0') + '-01',
endISO: y + '-' + String(m).padStart(2, '0') + '-' + String(monthEnd.getDate()).padStart(2, '0')
});
}

var lots = getLotQC(ownerUsername, role);
var params = getParameters(ownerUsername, role);
var paramMap = {}; params.forEach(function(p) { paramMap[p.paramID] = p; });
var paramBidang = {}; params.forEach(function(p) { paramBidang[p.paramID] = p.bidang || 'Lainnya';
});
var filteredLots = lots.filter(function(l) {
if (paramID && l.paramID !== paramID) return false;
if (lotID && l.lotID !== lotID) return false;
if (bidang && (paramBidang[l.paramID] || 'Lainnya') !== bidang) return false;
if (namaAlat && String(l.namaAlat).toLowerCase().indexOf(String(namaAlat).toLowerCase()) < 0)
return false;
return true;
});
if (!filteredLots.length) return sanitizeReturn({ ok: true, data: { months: months, sigmaTrend: {},
teTrend: {}, interpretasi: {} } });

var allPME = gSD(SH.BIASPME).filter(function(r) {
if (String(r[20]).toLowerCase() !== String(ownerUsername).toLowerCase()) return false;
if (paramID && r[1] !== paramID) return false;
if (lotID && r[2] !== lotID) return false;
if (siklusPME && r[6] !== siklusPME) return false;
if (tahunSiklus && String(r[7]) !== String(tahunSiklus)) return false;
return true;
});

var sigmaTrend = {
```

```
terhitung: { L1: [], L2: [], L3: [] },
pme: { L1: [], L2: [], L3: [] },
pmecv: { L1: [], L2: [], L3: [] },
cvopt: { L1: [], L2: [], L3: [] }
};
var teTrend = { L1: [], L2: [], L3: [], tea: [] };

var allQCData = getInputQC(ownerUsername, role, { startDate: months[0].startISO, endDate:
months[months.length-1].endISO });

months.forEach(function(m) {
[1, 2, 3].forEach(function(lv) {
var sigList = [];
var teList = [];
var teaList = [];
filteredLots.forEach(function(lot) {
var mStart = parseDateStr(m.startISO), mEnd = parseDateStr(m.endISO);
var qcs = allQCData.filter(function(q) {
  if (q.lotID !== lot.lotID) return false;
  var d = parseDateStr(q.tanggal);
  return d && d >= mStart && d <= mEnd;
});
if (!qcs.length) return;
var mn = parseNumSafe(lot['meanL' + lv]);
var tea = parseNumSafe(lot.tea);
var vals = qcs.map(function(q) { return parseNumSafe(lv === 1 ? q.level1 : lv === 2 ? q.level2 :
q.level3); }).filter(function(v) { return v !== null && v !== 0; });
if (!vals.length || !mn || !tea) return;

// PERBAIKAN #2 & #3: Gunakan SD dan Mean OBSERVASI (terhitung) bukan lot SD
var calcMean = vals.reduce(function(a, b) { return a + b; }, 0) / vals.length;
var calcSD = Math.sqrt(vals.reduce(function(s2, v) { return s2 + Math.pow(v - calcMean, 2); }, 0) /
vals.length);
var calcCV = calcMean ? (calcSD / calcMean * 100) : 0;
var bias = mn ? ((calcMean - mn) / mn * 100) : 0;
var te = Math.abs(bias) + 1.65 * calcCV;
var sigma = calcCV ? (tea - Math.abs(bias)) / calcCV : null;

if (sigma !== null) sigList.push(sigma);
teList.push(te);
teaList.push(tea);
});
sigmaTrend.terhitung['L' + lv].push(sigList.length ? parseFloat((sigList.reduce(function(a, b) { return a
+ b; }, 0) / sigList.length).toFixed(2)) : null);
```

```
teTrend['L' + lv].push(teList.length ? parseFloat((teList.reduce(function(a, b) { return a + b; }, 0) /
teList.length).toFixed(2)) : null);
if (lv === 1) teTrend.tea.push(teaList.length ? parseFloat((teaList.reduce(function(a, b) { return a + b;
}, 0) / teaList.length).toFixed(2)) : null);
});

// PERBAIKAN #5: Sigma PME dari tabel Bias PME (filtered by siklus & tahun)
[1, 2, 3].forEach(function(lv) {
var sigList = [];
filteredLots.forEach(function(lot) {
var lotPME = allPME.filter(function(p) { return p[2] === lot.lotID; });
if (!lotPME.length) return;
// Ambil PME terbaru yang sesuai filter
var latest = lotPME[lotPME.length - 1];
var hasilIdx = lv === 1 ? 8 : lv === 2 ? 9 : 10;
var meanPIdx = lv === 1 ? 11 : lv === 2 ? 12 : 13;
var cvIdx = lv === 1 ? 15 : lv === 2 ? 16 : 17;
var hasil = parseNumSafe(latest[hasilIdx]);
var meanP = parseNumSafe(latest[meanPIdx]);
var cvPME = parseNumSafe(latest[cvIdx]);
var tea = parseNumSafe(latest[14]) || parseNumSafe(lot.tea);
if (hasil && meanP && cvPME && tea) {
var bias = Math.abs((hasil - meanP) / meanP * 100);
sigList.push(parseFloat(((tea - bias) / cvPME).toFixed(2)));
}
});
sigmaTrend.pme['L' + lv].push(sigList.length ? parseFloat((sigList.reduce(function(a, b) { return a + b;
}, 0) / sigList.length).toFixed(2)) : null);
});

[1, 2, 3].forEach(function(lv) {
var sigList = [];
filteredLots.forEach(function(lot) {
var lotPME = allPME.filter(function(p) { return p[2] === lot.lotID; });
if (!lotPME.length) return;
var latest = lotPME[lotPME.length - 1];
var hasilIdx = lv === 1 ? 8 : lv === 2 ? 9 : 10;
var meanPIdx = lv === 1 ? 11 : lv === 2 ? 12 : 13;
var cvIdx = lv === 1 ? 15 : lv === 2 ? 16 : 17;
var hasil = parseNumSafe(latest[hasilIdx]);
var meanP = parseNumSafe(latest[meanPIdx]);
var cvPME = parseNumSafe(latest[cvIdx]);
var tea = parseNumSafe(latest[14]) || parseNumSafe(lot.tea);
if (hasil && meanP && cvPME && tea) {
var bias = Math.abs((hasil - meanP) / meanP * 100);
```

```
sigList.push(parseFloat(((tea - bias) / cvPME).toFixed(2)));
}
});
sigmaTrend.pmecv['L' + lv].push(sigList.length ? parseFloat((sigList.reduce(function(a, b) { return a +
b; }, 0) / sigList.length).toFixed(2)) : null);
});

[1, 2, 3].forEach(function(lv) {
var sigList = [];
filteredLots.forEach(function(lot) {
var lotOpt = gSD(SH.SIGMACVOPT).filter(function(r) {
return String(r[11]).toLowerCase() === String(ownerUsername).toLowerCase() && r[2] === lot.lotID;
});
if (!lotOpt.length) return;
var validOpt = lotOpt.filter(function(r) {
var sd = parseDateStr(r[4]);
if (!sd) return false;
return sd <= new Date(m.endISO);
});
if (!validOpt.length) validOpt = lotOpt;
var latest = validOpt[validOpt.length - 1];
var avgSigma = parseNumSafe(latest[6]);
if (avgSigma !== null) sigList.push(avgSigma);
});
sigmaTrend.cvopt['L' + lv].push(sigList.length ? parseFloat((sigList.reduce(function(a, b) { return a +
b; }, 0) / sigList.length).toFixed(2)) : null);
});
});

var interpretasi = {};
['terhitung', 'pme', 'pmecv', 'cvopt'].forEach(function(src) {
var labelMap = {
terhitung: 'Sigma Terhitung (data QC harian observasi)',
pme: 'Sigma PME (data uji profisiensi)',
pmecv: 'Sigma PME CV (bias PME + CV PME)',
cvopt: 'Sigma CV Optional (data eksternal)'
};
var notes = [];
var avgPerLv = {};
[1, 2, 3].forEach(function(lv) {
var arr = sigmaTrend[src]['L' + lv].filter(function(v) { return v !== null; });
if (arr.length) avgPerLv['L' + lv] = arr.reduce(function(a, b) { return a + b; }, 0) / arr.length;
});
var avgAll = [];
Object.keys(avgPerLv).forEach(function(k) { avgAll.push(avgPerLv[k]); });
```

```
var grand = avgAll.length ? avgAll.reduce(function(a, b) { return a + b; }, 0) / avgAll.length : null;
notes.push('Sumber: ' + labelMap[src]);
if (grand !== null) {
notes.push('Rata-rata sigma 3 level: ' + grand.toFixed(2));
if (grand >= 6) notes.push('Kinerja: World Class — pertahankan!');
else if (grand >= 5) notes.push('Kinerja: Excellent');
else if (grand >= 4) notes.push('Kinerja: Good — masih dalam standar');
else if (grand >= 3) notes.push('Kinerja: Marginal — perlu peningkatan');
else if (grand >= 2) notes.push('Kinerja: Poor — perbaikan diperlukan');
else notes.push('Kinerja: Unacceptable — evaluasi mendesak!');
var errorPer100 = estimateErrorPer100(grand);
notes.push('Estimasi kemungkinan error per 100 pasien: ' + errorPer100 + ' pasien');
} else {
notes.push('Data tidak cukup untuk interpretasi');
}
interpretasi[src] = {
grandSigma: grand !== null ? parseFloat(grand.toFixed(2)) : null,
notes: notes,
estimasiError: grand !== null ? estimateErrorPer100(grand) : null
};
});

var teInterp = [];
var teaAvg = teTrend.tea.filter(function(v) { return v !== null; });
var teaMean = teaAvg.length ? teaAvg.reduce(function(a, b) { return a + b; }, 0) / teaAvg.length : null;
[1, 2, 3].forEach(function(lv) {
var arr = teTrend['L' + lv].filter(function(v) { return v !== null; });
if (!arr.length) return;
var avg = arr.reduce(function(a, b) { return a + b; }, 0) / arr.length;
var status = 'baik';
if (teaMean !== null && avg > teaMean) status = 'melampaui TEa!';
teInterp.push('L' + lv + ': Rata-rata TE=' + avg.toFixed(2) + '%' + (teaMean !== null ? ' vs TEa=' +
teaMean.toFixed(2) + '%' : '') + ' — ' + status);
});
interpretasi.te = {
teaMean: teaMean,
notes: teInterp.length ? teInterp : ['Data TE tidak cukup']
};

var paramInfo = paramID && paramMap[paramID] ? paramMap[paramID].parameter : 'Semua
Parameter';

// — Instrument Compare (v9.7) —
var instrumentCompare = [];
var interpretasiDetail = '';
```

```
var rekomendasiAlat = '';
if (paramID && filteredLots.length) {
  var alatMap = {};
  filteredLots.forEach(function(lot) {
    var alat = lot.namaAlat || 'Unknown';
    if (!alatMap[alat]) alatMap[alat] = { namaAlat: alat, sigmas: [], cvs: [], biases: [], tes: [], tea: null,
lots: [] };
    alatMap[alat].lots.push(lot);
    if (!alatMap[alat].tea) alatMap[alat].tea = parseNumSafe(lot.tea);
    months.forEach(function(m) {
      var mS2 = parseDateStr(m.startISO), mE2 = parseDateStr(m.endISO);
      var qcs = allQCData.filter(function(q) {
        if (q.lotID !== lot.lotID) return false;
        var d = parseDateStr(q.tanggal);
        return d && d >= mS2 && d <= mE2;
      });
      if (!qcs.length) return;
      [1, 2, 3].forEach(function(lv) {
        var vals = qcs.map(function(q){return
parseNumSafe(lv===1?q.level1:lv===2?q.level2:q.level3);}).filter(function(v){return
v!==null&&v!==0;});
        if (!vals.length) return;
        var mn = parseNumSafe(lot['meanL'+lv]), tea = parseNumSafe(lot.tea);
        if (!mn || !tea) return;
        var calcMean = vals.reduce(function(a,b){return a+b;},0)/vals.length;
        var calcSD = Math.sqrt(vals.reduce(function(s2,v){return s2+Math.pow(v-
calcMean,2);},0)/vals.length);
        var calcCV = calcMean ? (calcSD/calcMean*100) : 0;
        var bias = Math.abs((calcMean-mn)/mn*100);
        var te = bias + 1.65*calcCV;
        var sigma = calcCV ? (tea-Math.abs(bias))/calcCV : null;
        if (sigma !== null) alatMap[alat].sigmas.push(sigma);
        alatMap[alat].cvs.push(calcCV);
        alatMap[alat].biases.push(bias);
        alatMap[alat].tes.push(te);
      });
    });
  });
  Object.keys(alatMap).forEach(function(alat) {
    var d = alatMap[alat];
    var avgSig = d.sigmas.length ? d.sigmas.reduce(function(a,b){return a+b;},0)/d.sigmas.length : null;
    var avgCV = d.cvs.length ? d.cvs.reduce(function(a,b){return a+b;},0)/d.cvs.length : null;
    var avgBias = d.biases.length ? d.biases.reduce(function(a,b){return a+b;},0)/d.biases.length : null;
    var avgTE = d.tes.length ? d.tes.reduce(function(a,b){return a+b;},0)/d.tes.length : null;
```

```
   function interpSigma(s) { if(s===null)return'N/A';if(s>=6)return'Sangat Baik (World
Class)';if(s>=5)return'Baik (Excellent)';if(s>=4)return'Cukup
(Good)';if(s>=3)return'Marginal';return'Buruk (Poor)'; }
   function interpCV(c) { if(c===null)return'N/A';if(c<=2)return'Sangat
Baik';if(c<=5)return'Baik';if(c<=10)return'Cukup';return'Buruk'; }
   function interpBias(b) { if(b===null)return'N/A';if(b<=1)return'Sangat
Baik';if(b<=2)return'Baik';if(b<=5)return'Cukup';return'Buruk'; }
   function interpTE(t) { if(t===null)return'N/A';var
tea=d.tea;if(!tea)return'N/A';if(t<=tea*0.5)return'Sangat
Baik';if(t<=tea*0.75)return'Baik';if(t<=tea)return'Marginal';return'Buruk (Melebihi TEa)'; }
   instrumentCompare.push({
    namaAlat: alat,
    avgSigma: avgSig!==null?parseFloat(avgSig.toFixed(2)):null,
    avgCV: avgCV!==null?parseFloat(avgCV.toFixed(2)):null,
    avgBias: avgBias!==null?parseFloat(avgBias.toFixed(2)):null,
    avgTE: avgTE!==null?parseFloat(avgTE.toFixed(2)):null,
    tea: d.tea,
    interpSigma: interpSigma(avgSig),
    interpCV: interpCV(avgCV),
    interpBias: interpBias(avgBias),
    interpTE: interpTE(avgTE),
    nData: d.sigmas.length
   });
  });
  instrumentCompare.sort(function(a,b){return(b.avgSigma||0)-(a.avgSigma||0);});
  var detailLines = [];
  detailLines.push('Perbandingan kinerja instrumen untuk parameter: ' + paramInfo);
  if (instrumentCompare.length < 2) {
   detailLines.push('Hanya tersedia data untuk ' + instrumentCompare.length + ' instrumen.
Perbandingan minimal memerlukan 2 instrumen.');
  } else {
   var best = instrumentCompare[0];
   var worst = instrumentCompare[instrumentCompare.length - 1];
   detailLines.push('Instrumen terbaik: ' + best.namaAlat + ' (σ=' + best.avgSigma + ', CV=' +
best.avgCV + '%, Bias=' + best.avgBias + '%)');
   detailLines.push('Instrumen terendah: ' + worst.namaAlat + ' (σ=' + worst.avgSigma + ', CV=' +
worst.avgCV + '%, Bias=' + worst.avgBias + '%)');
   if (best.avgSigma && worst.avgSigma) {
    var diff = parseFloat((best.avgSigma - worst.avgSigma).toFixed(2));
    detailLines.push('Selisih sigma: ' + diff + ' point');
   }
   instrumentCompare.forEach(function(ic) {
    detailLines.push('  - ' + ic.namaAlat + ': σ=' + (ic.avgSigma||'N/A') + ' [' + ic.interpSigma + '], CV=' +
(ic.avgCV||'N/A') + '% [' + ic.interpCV + '], Bias=' + (ic.avgBias||'N/A') + '% [' + ic.interpBias + ']');
   });
```

```
  }
  interpretasiDetail = detailLines.join(
);
  var recLines = [];
  if (instrumentCompare.length >= 2) {
    var bestAlat = instrumentCompare[0];
    var worstAlat = instrumentCompare[instrumentCompare.length - 1];
    recLines.push('REKOMENDASI ALAT:');
    recLines.push('1. Alat terbaik berdasarkan rata-rata sigma: ' + bestAlat.namaAlat + ' (σ=' +
bestAlat.avgSigma + ')');
    if (bestAlat.avgSigma && bestAlat.avgSigma >= 5) {
      recLines.push('   Kinerja ' + bestAlat.namaAlat + ' sudah Excellent/World Class. Pertahankan
kondisi alat dan kalibrasi.');
    } else if (bestAlat.avgSigma && bestAlat.avgSigma >= 4) {
      recLines.push('   Kinerja ' + bestAlat.namaAlat + ' baik (Good). Masih dapat ditingkatkan ke level
Excellent.');
    } else {
      recLines.push('   Kinerja alat terbaik masih di bawah standar optimal. Evaluasi metode dan
perawatan alat.');
    }
    recLines.push('2. Alat dengan kinerja terendah: ' + worstAlat.namaAlat + ' (σ=' +
worstAlat.avgSigma + ')');
    if (worstAlat.avgSigma && worstAlat.avgSigma < 3) {
      recLines.push('   PERHATIAN: Kinerja ' + worstAlat.namaAlat + ' Poor/Unacceptable!
Pertimbangkan evaluasi metode, kalibrasi ulang, atau penggantian alat.');
    } else if (worstAlat.avgSigma && worstAlat.avgSigma < 4) {
      recLines.push('   Kinerja ' + worstAlat.namaAlat + ' Marginal. Perlu perbaikan presisi dan/atau
akurasi.');
    }
    if (bestAlat.avgSigma && worstAlat.avgSigma && (bestAlat.avgSigma - worstAlat.avgSigma) > 2) {
      recLines.push('3. Terdapat perbedaan signifikan (>2σ) antar alat. Investigasi penyebab perbedaan
(metode, reagen, kalibrasi).');
    }
  } else if (instrumentCompare.length === 1) {
    recLines.push('REKOMENDASI ALAT:');
    recLines.push('Hanya tersedia 1 instrumen. Tidak dapat melakukan perbandingan antar alat.');
    var only = instrumentCompare[0];
    recLines.push('Kinerja ' + only.namaAlat + ': σ=' + (only.avgSigma||'N/A') + ' [' + only.interpSigma +
']');
  } else {
    recLines.push('REKOMENDASI ALAT: Tidak cukup data untuk memberikan rekomendasi.');
  }
  rekomendasiAlat = recLines.join(
);
}
```

```
return sanitizeReturn({
ok: true,
data: {
months: months,
sigmaTrend: sigmaTrend,
teTrend: teTrend,
interpretasi: interpretasi,
paramInfo: paramInfo,
filterInfo: {
paramID: paramID, lotID: lotID, bidang: bidang,
namaAlat: namaAlat, tahun: tahun,
bulanAwal: bulanAwal, bulanAkhir: bulanAkhir,
siklusPME: siklusPME, tahunSiklus: tahunSiklus
},
instrumentCompare: instrumentCompare,
interpretasiDetail: interpretasiDetail,
rekomendasiAlat: rekomendasiAlat
}
});
} catch (e) { return { ok: false, msg: e.message }; }
}

function estimateErrorPer100(sigma) {
  if (sigma >= 6) return 0;
  if (sigma >= 5.5) return 0;
  if (sigma >= 5) return 1;
  if (sigma >= 4.5) return 1;
  if (sigma >= 4) return 1;
  if (sigma >= 3.5) return 3;
  if (sigma >= 3) return 7;
  if (sigma >= 2.5) return 16;
  if (sigma >= 2) return 31;
  return 69;
}

function getDashboardAnalisisTrend(ownerUsername, role, filter) {
  return getTrendAnalisisData({
    paramID: (filter && filter.paramID) ? filter.paramID : null,
    months: (filter && filter.months) ? filter.months : 6
  }, ownerUsername, role);
}

function computeSigmaPME(pme, lv, lot) {
  var hasilIdx = lv===1?'hasilL1':lv===2?'hasilL2':'hasilL3';
```

```
  var meanIdx = lv===1?'meanPesertaL1':lv===2?'meanPesertaL2':'meanPesertaL3';
  var m = parseNumSafe(lot['meanL'+lv]), s = parseNumSafe(lot['sdL'+lv]), tea =
parseNumSafe(lot.tea);
  var hasil = parseNumSafe(pme[hasilIdx]), meanP = parseNumSafe(pme[meanIdx]);
  if (!m||!s||!tea||!hasil||!meanP) return null;
  var bias = Math.abs((hasil-meanP)/meanP*100);
  var cv = s/m*100;
  return cv ? parseFloat(((tea-bias)/cv).toFixed(2)) : null;
}

// ── INSTRUMENT COMPARE ──────────────────────────────────────
function getInstrumentCompare(ownerUsername, role, filter) {
  try {
    filter = filter || {};
    var lots = getLotQC(ownerUsername, role);
    var params = getParameters(ownerUsername, role);
    var paramMap = {}; params.forEach(function(p){paramMap[p.paramID]=p;});
    var alatData = {};
    lots.forEach(function(lot) {
      var alat = lot.namaAlat || 'Unknown';
      if (!alatData[alat]) alatData[alat] = { namaAlat: alat, scores: [], params: [] };
      var qcFilter = { lotID: lot.lotID };
      if (filter.startDate) qcFilter.startDate = filter.startDate;
      if (filter.endDate) qcFilter.endDate = filter.endDate;
      var qcData = getInputQC(ownerUsername, role, qcFilter);
      if (!qcData.length) return;
      var stats = computeQCStats(qcData, lot);
      var sigs = [];

[1,2,3].forEach(function(lv){if(stats['L'+lv]&&stats['L'+lv].sigma!==null)sigs.push(stats['L'+lv].sigma);})
;
      var avgSigma = sigs.length ? sigs.reduce(function(a,b){return a+b;},0)/sigs.length : null;
      var qgi = null, qgiInterp = '';
      if (stats.L1 && stats.L1.cv && stats.L1.bias !== undefined) {
        qgi = stats.L1.cv ? stats.L1.bias/(1.5*stats.L1.cv) : null;
        if (qgi !== null) qgiInterp = qgi>1.2?'Imprecision dominant':qgi<0.8?'Inaccuracy
dominant':'Balanced';
      }
      var paramEntry = {
        paramID: lot.paramID,
        parameter: paramMap[lot.paramID] ? paramMap[lot.paramID].parameter : '',
        bidang: paramMap[lot.paramID] ? paramMap[lot.paramID].bidang||'' : '',
        noLot: lot.noLot, tea: parseNumSafe(lot.tea), stats: stats,
        avgSigma: avgSigma ? parseFloat(avgSigma.toFixed(2)) : null,
        qgi: qgi !== null ? parseFloat(qgi.toFixed(3)) : null, qgiInterp: qgiInterp
```

```
    };
    if (filter.paramID && lot.paramID !== filter.paramID) return;
    alatData[alat].scores.push(avgSigma||0);
    alatData[alat].params.push(paramEntry);
  });
  var ranked = Object.keys(alatData).map(function(alat) {
    var d = alatData[alat]; var scores = d.scores.filter(function(s){return s!==null;});
    var avgScore = scores.length ? scores.reduce(function(a,b){return a+b;},0)/scores.length : 0;
    return {
      namaAlat: alat, avgSigma: parseFloat(avgScore.toFixed(2)), n: scores.length,
      params: d.params
    };
  }).sort(function(a,b){return b.avgSigma-a.avgSigma;});

  // — Helper interpretation functions —
  function interpSig(s) { if(s===null)return'N/A';if(s>=6)return'Sangat Baik (World
Class)';if(s>=5)return'Baik (Excellent)';if(s>=4)return'Cukup
(Good)';if(s>=3)return'Marginal';return'Buruk (Poor)'; }
  function interpCV2(c) { if(c===null)return'N/A';if(c<=2)return'Sangat
Baik';if(c<=5)return'Baik';if(c<=10)return'Cukup';return'Buruk'; }
  function interpBias2(b) { if(b===null)return'N/A';if(b<=1)return'Sangat
Baik';if(b<=2)return'Baik';if(b<=5)return'Cukup';return'Buruk'; }
  function interpTE2(t,tea) { if(t===null||!tea)return'N/A';if(t<=tea*0.5)return'Sangat
Baik';if(t<=tea*0.75)return'Baik';if(t<=tea)return'Marginal';return'Buruk (Melebihi TEa)'; }

  // — Compute per-instrument avgCV, avgBias, avgTE across all parameters —
  ranked.forEach(function(r) {
    var allCVs = [], allBiases = [], allTEs = [];
    r.params.forEach(function(p) {
      [1,2,3].forEach(function(lv) {
        var s = p.stats['L'+lv];
        if (s && s.cv !== null && s.cv !== undefined) allCVs.push(s.cv);
        if (s && s.bias !== null && s.bias !== undefined) allBiases.push(Math.abs(s.bias));
        if (s && s.te !== null && s.te !== undefined) allTEs.push(s.te);
      });
    });
    r.avgCV = allCVs.length ? parseFloat((allCVs.reduce(function(a,b){return
a+b;},0)/allCVs.length).toFixed(2)) : null;
    r.avgBias = allBiases.length ? parseFloat((allBiases.reduce(function(a,b){return
a+b;},0)/allBiases.length).toFixed(2)) : null;
    r.avgTE = allTEs.length ? parseFloat((allTEs.reduce(function(a,b){return
a+b;},0)/allTEs.length).toFixed(2)) : null;
    r.nParams = r.params.length;
  });
```

```
  // ── Generate detailColumns ──
  var detailColumns = ranked.map(function(r) {
    var qgiAvg = null;
    if (r.avgCV && r.avgBias !== null) {
      qgiAvg = r.avgCV ? r.avgBias / (1.5 * r.avgCV) : null;
    }
    var qgiInterpVal = '';
    if (qgiAvg !== null) qgiInterpVal = qgiAvg > 1.2 ? 'Imprecision dominant' : qgiAvg < 0.8 ?
'Inaccuracy dominant' : 'Balanced';
    var avgTEa = null;
    var teas = r.params.map(function(p){return p.tea;}).filter(function(t){return t!==null;});
    if (teas.length) avgTEa = teas.reduce(function(a,b){return a+b;},0)/teas.length;
    return {
      namaAlat: r.namaAlat,
      avgSigma: r.avgSigma,
      avgCV: r.avgCV,
      avgBias: r.avgBias,
      avgTE: r.avgTE,
      nParams: r.nParams,
      interpSigma: interpSig(r.avgSigma),
      interpCV: interpCV2(r.avgCV),
      interpBias: interpBias2(r.avgBias),
      interpTE: interpTE2(r.avgTE, avgTEa),
      qgiAvg: qgiAvg !== null ? parseFloat(qgiAvg.toFixed(3)) : null,
      qgiInterp: qgiInterpVal
    };
  });

  // ── Generate paramCompareDetail (per-parameter comparison across instruments) ──
  var paramCompareMap = {};
  ranked.forEach(function(r) {
    r.params.forEach(function(p) {
      if (!paramCompareMap[p.paramID]) {
        paramCompareMap[p.paramID] = {
          paramID: p.paramID,
          parameter: p.parameter,
          bidang: p.bidang,
          instruments: []
        };
      }
      var allCVs = [], allBiases = [], allTEs = [], allSigmas = [];
      [1,2,3].forEach(function(lv) {
        var s = p.stats['L'+lv];
        if (s) {
          if (s.cv !== null && s.cv !== undefined) allCVs.push(s.cv);
```

```
        if (s.bias !== null && s.bias !== undefined) allBiases.push(Math.abs(s.bias));
        if (s.te !== null && s.te !== undefined) allTEs.push(s.te);
        if (s.sigma !== null && s.sigma !== undefined) allSigmas.push(s.sigma);
       }
      });
      var pCV = allCVs.length ? parseFloat((allCVs.reduce(function(a,b){return
a+b;},0)/allCVs.length).toFixed(2)) : null;
      var pBias = allBiases.length ? parseFloat((allBiases.reduce(function(a,b){return
a+b;},0)/allBiases.length).toFixed(2)) : null;
      var pTE = allTEs.length ? parseFloat((allTEs.reduce(function(a,b){return
a+b;},0)/allTEs.length).toFixed(2)) : null;
      var pSigma = allSigmas.length ? parseFloat((allSigmas.reduce(function(a,b){return
a+b;},0)/allSigmas.length).toFixed(2)) : null;
      paramCompareMap[p.paramID].instruments.push({
       namaAlat: r.namaAlat,
       noLot: p.noLot,
       tea: p.tea,
       avgSigma: pSigma,
       avgCV: pCV,
       avgBias: pBias,
       avgTE: pTE,
       interpSigma: interpSig(pSigma),
       interpCV: interpCV2(pCV),
       interpBias: interpBias2(pBias),
       interpTE: interpTE2(pTE, p.tea)
      });
     });
   });
   var paramCompareDetail = Object.keys(paramCompareMap).map(function(k){return
paramCompareMap[k];});

   // — Generate detailedInterpretasi —
   var detailedInterpretasi = '';
   if (ranked.length === 0) {
    detailedInterpretasi = 'Tidak cukup data untuk perbandingan instrumen.';
   } else {
    var lines = [];
    lines.push('=== ANALISIS PERBANDINGAN INSTRUMEN ===');
    lines.push('Jumlah instrumen: ' + ranked.length);
    if (ranked.length >= 2) {
     var best = ranked[0], worst = ranked[ranked.length - 1];
     lines.push('');
     lines.push('Instrumen terbaik: ' + best.namaAlat);
     lines.push('  Sigma: ' + (best.avgSigma||'N/A') + ' [' + interpSig(best.avgSigma) + ']');
     lines.push('  CV: ' + (best.avgCV||'N/A') + '% [' + interpCV2(best.avgCV) + ']');
```

```
      lines.push('  Bias: ' + (best.avgBias||'N/A') + '% [' + interpBias2(best.avgBias) + ']');
      lines.push('  TE: ' + (best.avgTE||'N/A') + '% [' + interpTE2(best.avgTE, best.params.length ?
best.params[0].tea : null) + ']');
      lines.push('  Jumlah parameter: ' + best.nParams);
      lines.push('');
      lines.push('Instrumen terendah: ' + worst.namaAlat);
      lines.push('  Sigma: ' + (worst.avgSigma||'N/A') + ' [' + interpSig(worst.avgSigma) + ']');
      lines.push('  CV: ' + (worst.avgCV||'N/A') + '% [' + interpCV2(worst.avgCV) + ']');
      lines.push('  Bias: ' + (worst.avgBias||'N/A') + '% [' + interpBias2(worst.avgBias) + ']');
      lines.push('  TE: ' + (worst.avgTE||'N/A') + '% [' + interpTE2(worst.avgTE, worst.params.length ?
worst.params[0].tea : null) + ']');
      lines.push('  Jumlah parameter: ' + worst.nParams);
      if (best.avgSigma && worst.avgSigma) {
        var diff = parseFloat((best.avgSigma - worst.avgSigma).toFixed(2));
        lines.push('');
        lines.push('Selisih sigma antar instrumen terbaik dan terendah: ' + diff + ' point');
        if (diff > 2) lines.push('Perbedaan signifikan (>2σ) — investigasi penyebab perbedaan (metode,
reagen, kalibrasi).');
        else if (diff > 1) lines.push('Perbedaan moderat (1-2σ) — pemantauan berkala diperlukan.');
        else lines.push('Perbedaan kecil (<1σ) — kinerja instrumen relatif seragam.');
      }
    }
    lines.push('');
    lines.push('=== DETAIL PER INSTRUMEN ===');
    detailColumns.forEach(function(dc) {
      lines.push('');
      lines.push('■ ' + dc.namaAlat + ' (' + dc.nParams + ' parameter)');
      lines.push('  Rata-rata Sigma: ' + (dc.avgSigma||'N/A') + ' — ' + dc.interpSigma);
      lines.push('  Rata-rata CV: ' + (dc.avgCV||'N/A') + '% — ' + dc.interpCV);
      lines.push('  Rata-rata Bias: ' + (dc.avgBias||'N/A') + '% — ' + dc.interpBias);
      lines.push('  Rata-rata TE: ' + (dc.avgTE||'N/A') + '% — ' + dc.interpTE);
      if (dc.qgiAvg !== null) {
        lines.push('  QGI: ' + dc.qgiAvg + ' — ' + dc.qgiInterp);
        if (dc.qgiInterp === 'Imprecision dominant') lines.push('  → Perbaikan presisi (CV) harus
diprioritaskan.');
        else if (dc.qgiInterp === 'Inaccuracy dominant') lines.push('  → Perbaikan akurasi (kalibrasi)
harus diprioritaskan.');
        else lines.push('  → Keseimbangan presisi dan akurasi baik.');
      }
    });
    detailedInterpretasi = lines.join('\n');
  }

  return sanitizeReturn({ ok: true, data: ranked, detailedInterpretasi: detailedInterpretasi,
detailColumns: detailColumns, paramCompareDetail: paramCompareDetail });
```

```
  } catch (e) { return { ok: false, msg: e.message }; }
}

// ── TABULASI ──────────────────────────────────────────────────
function getTabulasiData(ownerUsername, role, filter) {
  try {
    filter = filter || {};
    var params = getParameters(ownerUsername, role);
    if (filter.paramIDs && filter.paramIDs.length) params = params.filter(function(p){return
filter.paramIDs.indexOf(p.paramID)>-1;});
    if (filter.bidang) params = params.filter(function(p){return (p.bidang||'Lainnya')===filter.bidang;});
    var lots = getLotQC(ownerUsername, role);
    var result = [];
    params.forEach(function(param) {
      lots.filter(function(l){return l.paramID===param.paramID;}).forEach(function(lot) {
        var qcFilter = { lotID: lot.lotID };
        if (filter.startDate) qcFilter.startDate = filter.startDate;
        if (filter.endDate) qcFilter.endDate = filter.endDate;
        var qcData = getInputQC(ownerUsername, role, qcFilter);
        if (!qcData.length) return;
        var stats = computeQCStats(qcData, lot);
        var pmeData = getBiasPME(ownerUsername, role, {});
        var lotPME = pmeData.filter(function(p){return p.lotID===lot.lotID;});
        var latestPME = lotPME.length ? lotPME[lotPME.length-1] : null;
        result.push({
          paramID: param.paramID, parameter: param.parameter, bidang: param.bidang||'Lainnya',
          lotID: lot.lotID, namaAlat: lot.namaAlat, noLot: lot.noLot,
          satuan: lot.satuan, tea: parseNumSafe(lot.tea), stats: stats, pme: latestPME, nQC:
qcData.length
        });
      });
    });
    // ── PME Rekap Detail (v9.7) ──
    var pmeRekapDetail = [];
    var allPME = getBiasPME(ownerUsername, role, {});
    var pmeFiltered = allPME.filter(function(pme) {
      if (filter.tahun && String(pme.tahun) !== String(filter.tahun)) return false;
      if (filter.bulan && pme.siklus !== filter.bulan) return false;
      if (filter.bidang) {
        var pParam = params.find(function(pp){return pp.paramID === pme.paramID;});
        if (!pParam || (pParam.bidang||'Lainnya') !== filter.bidang) return false;
      }
      if (filter.paramIDs && filter.paramIDs.length && filter.paramIDs.indexOf(pme.paramID) < 0)
return false;
      if (filter.parameter && pme.paramID !== filter.parameter) return false;
```

```
      return true;
    });
    pmeFiltered.forEach(function(pme) {
      var pName = '';
      var pParam = params.find(function(pp){return pp.paramID === pme.paramID;});
      if (pParam) pName = pParam.parameter;
      var tea = parseNumSafe(pme.tea);
      var row = {
        Parameter: pName, TEa: tea, Siklus: pme.siklus, TahunSiklus: pme.tahun,
        NamaAlat: pme.namaAlat,
        HasilL1: null, HasilL2: null, HasilL3: null,
        HasilPL1: null, HasilPL2: null, HasilPL3: null,
        BiasL1: null, BiasL2: null, BiasL3: null,
        SigmaL1: null, SigmaL2: null, SigmaL3: null
      };
      [1, 2, 3].forEach(function(lv) {
        var d = pme.details['L'+lv];
        if (!d) return;
        var hasil = parseNumSafe(pme['hasilL'+lv]);
        var meanP = parseNumSafe(pme['meanPesertaL'+lv]);
        var bias = (hasil && meanP) ? ((hasil - meanP) / meanP * 100) : null;
        var cv = parseNumSafe(pme['cvL'+lv]) || (d.cv ? parseFloat(d.cv) : null);
        var sigma = (tea !== null && bias !== null && cv) ? (tea - Math.abs(bias)) / cv : null;
        row['HasilL'+lv] = hasil;
        row['HasilPL'+lv] = meanP;
        row['BiasL'+lv] = bias !== null ? parseFloat(bias.toFixed(2)) : null;
        row['SigmaL'+lv] = sigma !== null ? parseFloat(sigma.toFixed(2)) : null;
      });
      pmeRekapDetail.push(row);
    });
    var periodKey = (filter.startDate||'')+'_'+(filter.endDate||'');
    var catatan = getCatatanTabulasi(periodKey);
    var kop = getKopSurat(ownerUsername);
    return sanitizeReturn({ ok: true, data: result, catatan: catatan, periodKey: periodKey, kop: kop,
filter: filter, pmeRekapDetail: pmeRekapDetail });
  } catch (e) { return { ok: false, msg: e.message }; }
}

// ── OPSPECS ──────────────────────────────────────────────────
function getOPSpecsData(ownerUsername, role, filter) {
  try {
    filter = filter || {};
    var params = getParameters(ownerUsername, role);
    if (filter.bidang) params = params.filter(function(p){return (p.bidang||'Lainnya')===filter.bidang;});
    var lots = getLotQC(ownerUsername, role);
```

```
  var allQC = getInputQC(ownerUsername, role, filter.startDate || filter.endDate ? { startDate:
filter.startDate, endDate: filter.endDate } : {});
  var result = [];
  var opsAnalisisDetail = [];
  var opsCriticalErrorData = [];
  params.forEach(function(param) {
   lots.filter(function(l){return l.paramID===param.paramID;}).forEach(function(lot) {
    var qcs = allQC.filter(function(q){return q.lotID===lot.lotID;});
    if (!qcs.length) return;
    var tea = parseNumSafe(lot.tea);
    if (!tea) return;
    // ── Hitung sigma per level, gunakan level sigma terkecil ──
    // PERBAIKAN v9.10: CV dihitung dari data observasi (calcSD/calcMean), BUKAN CV
Lot/Manufaktur
    var levelData = {};
    [1,2,3].forEach(function(lv) {
     var mLv = parseNumSafe(lot['meanL'+lv]);
     if (!mLv) return;
     var valsLv = qcs.map(function(q){return
parseNumSafe(lv===1?q.level1:lv===2?q.level2:q.level3);}).filter(function(v){return v!==null &&
v!==0;});
     if (!valsLv.length) return;
     var calcMeanLv = valsLv.reduce(function(a,b){return a+b;},0)/valsLv.length;
     var calcSDLv = Math.sqrt(valsLv.reduce(function(s2,v){return s2+Math.pow(v-
calcMeanLv,2);},0)/valsLv.length);
     var cvLv = calcMeanLv ? (calcSDLv/calcMeanLv*100) : 0;
     if (!cvLv) return;
     var biasLv = Math.abs((calcMeanLv-mLv)/mLv*100);
     var sigmaLv = (tea-biasLv)/cvLv;
     levelData['L'+lv] = { cv: cvLv, bias: biasLv, sigma: sigmaLv, mean: mLv };
    });
    var smallestLevel = null, smallestSigma = null;
    ['L1','L2','L3'].forEach(function(lv) {
     var ld = levelData[lv];
     if (ld && ld.sigma !== null && (smallestSigma === null || ld.sigma < smallestSigma)) {
      smallestSigma = ld.sigma;
      smallestLevel = lv;
     }
    });
    if (!smallestLevel) return;
    var cv = levelData[smallestLevel].cv;
    var bias = levelData[smallestLevel].bias;
    var sigma = smallestSigma;
    var ped = sigma ? computePed(sigma) : null, pfr = sigma ? computePfr(sigma) : null;
    var biasAbs = Math.abs(bias);
```

```
      var sec = (tea !== null && cv) ? (tea - biasAbs) / cv - 1.65 : null;
      var rec = (tea !== null && cv) ? (tea - biasAbs) / (1.65 * cv) - 1 : null;
      // ── Detailed OPSpecs Analysis (v9.7) ──
      var pedInterp = '';
      if (ped === null) pedInterp = 'Tidak dapat dihitung (data tidak cukup)';
      else if (ped >= 90) pedInterp = 'Sangat Baik - Probabilitas deteksi error sangat tinggi';
      else if (ped >= 50) pedInterp = 'Cukup Baik - Probabilitas deteksi error memadai';
      else pedInterp = 'Kurang Baik - Perlu evaluasi metode dan aturan QC';
      var dSec = sec ? parseFloat(sec.toFixed(2)) : null;
      var dRec = rec ? parseFloat(rec.toFixed(4)) : null;
      var critErrInterp = '';
      if (dSec === null || dRec === null) critErrInterp = 'Data tidak cukup';
      else if (dSec > 3 && dRec < 1.5) critErrInterp = 'Baik - Critical error terdeteksi dengan baik';
      else if (dSec > 2) critErrInterp = 'Cukup - Kemampuan deteksi critical error moderat';
      else critErrInterp = 'Kurang - Risiko critical error tinggi, evaluasi metode';
      var detailText = 'Parameter: ' + param.parameter + ' | Alat: ' + lot.namaAlat + ' | CV=' +
parseFloat(cv.toFixed(2)) + '% | Bias=' + parseFloat(bias.toFixed(2)) + '% | TEa=' + tea + '% | Sigma=' +
(sigma?sigma.toFixed(2):'N/A') + ' | PED=' + ped + '% [' + pedInterp + ']';
      opsAnalisisDetail.push(detailText);
      opsCriticalErrorData.push({
        parameter: param.parameter, namaAlat: lot.namaAlat,
        deltaSEc: dSec, deltaREc: dRec,
        criticalErrorInterp: critErrInterp,
        ped: ped, pedInterp: pedInterp,
        cv: parseFloat(cv.toFixed(2)), bias: parseFloat(bias.toFixed(2)),
        tea: tea, sigma: sigma ? parseFloat(sigma.toFixed(2)) : null
      });
      result.push({
        paramID: param.paramID, parameter: param.parameter, bidang: param.bidang||'Lainnya',
        lotID: lot.lotID, namaAlat: lot.namaAlat, noLot: lot.noLot,
        cv: parseFloat(cv.toFixed(2)), bias: parseFloat(bias.toFixed(2)),
        tea: tea, sigma: sigma ? parseFloat(sigma.toFixed(2)) : null,
        ped: ped, pfr: pfr, sec: sec, rec: rec
      });
    });
  });
  var kop = getKopSurat(ownerUsername);
  // ── OPSpecs Kesimpulan & Analisis Detail (Enhanced) ──
  var opsKesimpulan = '';
  if (result.length > 0) {
    var totalPEDok = result.filter(function(r){return r.ped && r.ped >= 90;}).length;
    var totalPECukup = result.filter(function(r){return r.ped && r.ped >= 50 && r.ped < 90;}).length;
    var totalPEKurang = result.filter(function(r){return r.ped && r.ped < 50;}).length;
    var avgSigmaAll = result.filter(function(r){return r.sigma !== null;}).map(function(r){return
r.sigma;});
```

```
    var avgSigma = avgSigmaAll.length ? avgSigmaAll.reduce(function(a,b){return
a+b;},0)/avgSigmaAll.length : null;
    var kesLines = [];
    kesLines.push('═══════════════════════════════════════════════════════════');
    kesLines.push('KESIMPULAN ANALISIS OPSPECS & CRITICAL-ERROR');
    kesLines.push('═══════════════════════════════════════════════════════════');
    kesLines.push('');
    kesLines.push('▸ Total parameter dievaluasi: ' + result.length);
    kesLines.push('  – PED Sangat Baik (≥90%): ' + totalPEDok + ' parameter');
    kesLines.push('  – PED Cukup Baik (50-89%): ' + totalPECukup + ' parameter');
    kesLines.push('  – PED Kurang Baik (<50%): ' + totalPEKurang + ' parameter');
    if (avgSigma !== null) kesLines.push('  – Rata-rata Sigma: ' + avgSigma.toFixed(2));
    kesLines.push('');
    kesLines.push('──────────────────────────────────────────────────');
    kesLines.push('ANALISIS DETAIL PER PARAMETER (OPSpecs & Critical-Error):');
    kesLines.push('──────────────────────────────────────────────────');
    result.forEach(function(r, idx) {
      kesLines.push('');
      kesLines.push((idx+1) + '. ' + r.parameter + ' (' + r.namaAlat + ')');
      kesLines.push('   CV: ' + r.cv + '% | Bias: ' + r.bias + '% | TEa: ' + r.tea + '% | Sigma: ' + (r.sigma !==
null ? r.sigma : 'N/A'));
      kesLines.push('');
      kesLines.push('   ■ ANALISIS OPSpecs (Grafik OPSpecs):');
      kesLines.push('   – Titik koordinat (CV=' + r.cv + '%, Bias=' + r.bias + '%) berada di ' + (r.sigma >= 4
? 'ZONA AMAN (hijau)' : r.sigma >= 2 ? 'ZONA KRITIS (kuning)' : 'ZONA BAHAYA (merah)') + '.');
      if (r.sigma !== null && r.sigma >= 6) {
        kesLines.push('   – Sigma ' + r.sigma + ' (World Class): Kinerja QC sangat baik. Aturan 1-3s saja
sudah cukup untuk mendeteksi error. Probabilitas error tidak terdeteksi sangat rendah.');
      } else if (r.sigma !== null && r.sigma >= 5) {
        kesLines.push('   – Sigma ' + r.sigma + ' (Excellent): Kinerja sangat baik. Sistem QC mampu
mendeteksi error dengan baik menggunakan aturan multi-rule standar.');
      } else if (r.sigma !== null && r.sigma >= 4) {
        kesLines.push('   – Sigma ' + r.sigma + ' (Good): Kinerja baik. Gunakan aturan 1-3s, 2-2s, R-4s
untuk menjaga kualitas.');
      } else if (r.sigma !== null && r.sigma >= 3) {
        kesLines.push('   – Sigma ' + r.sigma + ' (Marginal): Kinerja marginal. Perlu multi-rule lengkap (1-
3s, 2-2s, R-4s, 4-1s) dan pemantauan ketat. Pertimbangkan perbaikan presisi atau metode.');
      } else if (r.sigma !== null) {
        kesLines.push('   – Sigma ' + r.sigma + ' (Poor/Unacceptable): KINERJA BURUK! QC tidak reliable.
Diperlukan evaluasi menyeluruh: perbaikan kalibrasi, ganti reagen, atau pertimbangkan penggantian
metode/alat.');
      }

      kesLines.push('');
      kesLines.push('   ■ HUBUNGAN OPSpecs DAN CRITICAL-ERROR:');
```

```
      var ceData = opsCriticalErrorData.filter(function(c){return c.parameter === r.parameter &&
c.namaAlat === r.namaAlat;})[0];
      if (r.sigma !== null && ceData) {
        if (r.sigma >= 4) {
          kesLines.push('   – Grafik OPSpecs menunjukkan titik di ZONA AMAN, dan grafik Critical-Error
mengonfirmasi ΔSEc besar (≥3 SD) dan ΔREc kecil (≤1 CV). Ini berarti: (1) Sistem QC memiliki
performa sangat baik, (2) Error sistematik maupun acak dapat terdeteksi sebelum melewati
spesifikasi, (3) Risiko error klinis sangat rendah. Tidak ada tindakan korektif yang mendesak.');
        } else if (r.sigma >= 3) {
          kesLines.push('   – Grafik OPSpecs menunjukkan titik mendekati batas zona kritis, sedangkan
grafik Critical-Error menunjukkan ΔSEc moderat dan ΔREc meningkat. Ini berarti: (1) Sistem QC masih
mampu mendeteksi error besar, namun mungkin melewatkan error kecil-kecil, (2) Peningkatan CV
atau Bias sedikit saja dapat mendorong titik masuk zona bahaya, (3) Diperlukan peningkatan presisi
dan pemantauan bias secara berkala. Rekomendasi: pertahankan multi-rule lengkap dan evaluasi
tren sigma.');
        } else {
          kesLines.push('   – Grafik OPSpecs menunjukkan titik di ZONA BAHAYA/KEKRITIS, dan grafik
Critical-Error mengonfirmasi ΔSEc kecil (<2 SD) dan/atau ΔREc besar (>1.5 CV). Ini berarti: (1) Sistem
QC TIDAK MAMPU mendeteksi error secara memadai, (2) Bahkan error kecil sudah mendekati atau
melampaui batas spesifikasi, (3) Risiko hasil QC keliru yang masuk ke pasien sangat tinggi.
TINDAKAN: (a) Investigasi sumber error — periksa kalibrasi, reagen, pipetting, dan kondisi alat, (b)
Pertimbangkan perubahan metode atau penggantian alat jika perbaikan tidak memadai, (c)
Tingkatkan frekuensi QC sementara, (d) Lakukan PME untuk konfirmasi akurasi.');
        }
      } else if (r.sigma !== null) {
        kesLines.push('   – Sigma = ' + r.sigma + ' menunjukkan kinerja ' + (r.sigma >= 4 ? 'baik' :
'marginal/buruk') + '. Tindakan perbaikan ' + (r.sigma >= 4 ? 'tidak mendesak' : 'DIPERLUKAN') + '.');
      }
    });
    kesLines.push('');
    kesLines.push('————————————————————————————————————————————————————');
    kesLines.push('KESIMPULAN UMUM & REKOMENDASI:');
    kesLines.push('————————————————————————————————————————————————————');
    if (totalPEDok === result.length && result.length > 0) {
      kesLines.push('✅ SELURUH parameter memiliki kinerja OPSpecs SANGAT BAIK (Ped ≥90%, Sigma
≥4).');
      kesLines.push('   Sistem QC saat ini dapat diandalkan. Pertahankan strategi QC dan monitoring
berkala.');
      kesLines.push('   Grafik OPSpecs dan Critical-Error saling mengonfirmasi bahwa kemampuan
deteksi error sangat baik untuk semua parameter.');
    } else if (totalPEKurang === 0) {
      kesLines.push('⚠ Sebagian parameter memiliki kinerja OPSpecs CUKUP (Ped 50-89%).');
      kesLines.push('   Perlu pemantauan dan evaluasi berkala. Pastikan tidak terjadi degradasi
kinerja.');
      kesLines.push('   Tinjau parameter dengan Ped terendah untuk perbaikan preventif.');
```

```
    } else {
      kesLines.push('⚠ ' + totalPEKurang + ' parameter memiliki kinerja OPSpecs KURANG (Ped <50%).');
      kesLines.push('   Parameter ini memerlukan evaluasi dan perbaikan SEGERA:');
      var krgParams = opsCriticalErrorData.filter(function(c){return c.ped !== null && c.ped < 50;});
      krgParams.forEach(function(c) {
        kesLines.push('   → ' + c.parameter + ' (' + c.namaAlat + '): σ=' + (c.sigma||'N/A') + ', Ped=' + c.ped + '%, ΔSEc=' + (c.deltaSEc||'N/A') + ', ΔREc=' + (c.deltaREC||'N/A'));
      });
      kesLines.push('   Hubungan OPSpecs-Critical-Error: Titik-titik di zona bahaya pada grafik OPSpecs berkorelasi dengan ΔSEc kecil dan ΔREc besar pada grafik Critical-Error, menunjukkan bahwa error yang tidak terdeteksi oleh aturan QC saat ini sudah mendekati/melampaui batas spesifikasi (TEa).');
      kesLines.push('');
      kesLines.push('   REKOMENDASI TINDAKAN:');
      kesLines.push('   1. Evaluasi dan perbaiki presisi (turunkan CV) — periksa teknik, reagen, kalibrasi');
      kesLines.push('   2. Evaluasi dan perbaiki akurasi (turunkan Bias) — periksa kalibrasi, lot reagen');
      kesLines.push('   3. Tingkatkan frekuensi QC untuk parameter kritis');
      kesLines.push('   4. Pertimbangkan perubahan metode/alat jika sigma tetap <3 setelah perbaikan');
      kesLines.push('   5. Lakukan PME untuk memvalidasi kinerja terhadap laboratorium lain');
    }
    opsKesimpulan = kesLines.join('\n');
  } else {
    opsKesimpulan = 'Tidak cukup data untuk membuat kesimpulan OPSpecs.';
  }
  return sanitizeReturn({ ok: true, data: result, kop: kop, opsAnalisisDetail: opsAnalisisDetail, opsKesimpulan: opsKesimpulan, opsCriticalErrorData: opsCriticalErrorData });
 } catch (e) { return { ok: false, msg: e.message }; }
}
function computePed(sigma) { if(sigma>=6)return 99.9;if(sigma>=5)return 99.0;if(sigma>=4)return 90.0;if(sigma>=3)return 70.0;if(sigma>=2)return 40.0;return 10.0;}
function computePfr(sigma) { if(sigma>=6)return 0.1;if(sigma>=5)return 0.5;if(sigma>=4)return 1.0;if(sigma>=3)return 2.0;if(sigma>=2)return 5.0;return 10.0;}

// ── SMART IMPORT ──────────────────────────────────────────
function smartImportQC(payload, ownerUsername) {
 return withLock(function() {
  try {
   if (payload.smartPassword !== SMART_PWD) return { ok: false, msg: 'Password salah' };
   var rows = payload.rows || [];
   var params = getParameters(ownerUsername, 'user');
   var lots = getLotQC(ownerUsername, 'user');
   var sh = S(SH.INPUTQC);
   var count = 0, skipped = 0, errors = [];
```

```
    rows.forEach(function(row) {
      var matched = params.find(function(p){return
p.parameter.toLowerCase()===String(row.parameter||'').toLowerCase();});
      if (!matched) matched = params.find(function(p) {
        return String(row.parameter||'').toLowerCase().indexOf(p.parameter.toLowerCase())>-1 ||
            p.parameter.toLowerCase().indexOf(String(row.parameter||'').toLowerCase())>-1;
      });
      if (!matched) { errors.push('Param: '+row.parameter); skipped++; return; }
      var matchedLot = null;
      var paramLots = lots.filter(function(l){return l.paramID===matched.paramID;});
      if (row.noLot) matchedLot = paramLots.find(function(l){return
l.noLot.toLowerCase()===String(row.noLot).toLowerCase();});
      if (!matchedLot && paramLots.length > 0) matchedLot = paramLots[0];
      if (!matchedLot) { errors.push('Lot tdk ditemukan: '+row.parameter); skipped++; return; }
      if (!row.tanggal) { errors.push('Tanggal kosong'); skipped++; return; }
      var dt = parseDateStr(row.tanggal);
      if (!dt) { errors.push('Tgl invalid: '+row.tanggal); skipped++; return; }
      sh.appendRow([genID('QC'), matched.paramID, matchedLot.lotID,
        matched.parameter, matchedLot.noLot, matchedLot.namaAlat,
        dt, parseNumSafe(row.level1), parseNumSafe(row.level2), parseNumSafe(row.level3),
        ownerUsername, new Date(), false, '', '', '', ownerUsername]);
      count++;
    });
    logA(ownerUsername, 'SMART_IMPORT', count+' imported, '+skipped+' skipped');
    return { ok: true, count: count, skipped: skipped, errors: errors };
  } catch (e) { return { ok: false, msg: e.message }; }
 });
}

// ── HAPUS DATA PRIVAT ──────────────────────────────────────
function hapusDataPrivat(type, ids, ownerUsername, alasan, gatePassword, logUser) {
 return withLock(function() {
  if (gatePassword !== SMART_PWD) return { ok: false, msg: 'Password salah' };
  if (!alasan || alasan.length < 5) return { ok: false, msg: 'Alasan min 5 karakter' };
  var count = 0;
  ids.forEach(function(id) {
    var res;
    if (type === 'inputqc') res = deleteInputQC(id, ownerUsername, alasan);
    else if (type === 'lotqc') res = deleteLotQC(id, ownerUsername);
    else if (type === 'parameter') res = deleteParameter(id, ownerUsername);
    if (res && res.ok) count++;
  });
  logA(ownerUsername, 'HAPUS_'+type.toUpperCase(), count+' item, alasan: '+alasan, logUser);
  return { ok: true, count: count };
 });
```

```
}

function getInputQCForHapus(ownerUsername, payload) {
  try {
  payload = payload || {};
  var filter = {
    startDate: payload.startDate || null,
    endDate: payload.endDate || null,
    paramIDs: payload.paramIDs || null
  };
  return getInputQC(ownerUsername, 'user', filter);
  } catch(e){return [];}
}

// ── ARCHIVE & BACKUP ──────────────────────────────────────
function autoArchiveYearly() {
  try {
    var now = new Date(); var year = now.getFullYear()-1;
    var archiveName = 'InputQC_'+year; var ss = getSS();
    if (ss.getSheetByName(archiveName)) return;
    var srcSh = S(SH.INPUTQC); if (!srcSh || srcSh.getLastRow() < 2) return;
    var srcData = srcSh.getDataRange().getValues();
    var headers = srcData[0];
    var archiveSh = ss.insertSheet(archiveName);

archiveSh.getRange(1,1,1,headers.length).setValues([headers]).setFontWeight('bold').setBackgroun
d('#7c4dff').setFontColor('#ffffff');
    var toArchive = srcData.filter(function(r,i){if(i===0)return false;var d=parseDateStr(r[6]);return
d&&d.getFullYear()===year;});
    if (toArchive.length)
archiveSh.getRange(2,1,toArchive.length,headers.length).setValues(toArchive);
  } catch (e) {}
}
function setupArchiveTrigger() {

ScriptApp.getProjectTriggers().forEach(function(t){if(t.getHandlerFunction()==='autoArchiveYearly')S
criptApp.deleteTrigger(t);});
  ScriptApp.newTrigger('autoArchiveYearly').timeBased().onMonthDay(31).atHour(23).create();
  return { ok: true };
}
function getOrCreateBackupFolder() { var folders =
DriveApp.getFoldersByName(BACKUP_FOLDER_NAME); if (folders.hasNext()) return folders.next();
return DriveApp.createFolder(BACKUP_FOLDER_NAME); }
```

```
function getOrCreateSubFolder(parentFolder, name) { var subFolders =
parentFolder.getFoldersByName(name); if (subFolders.hasNext()) return subFolders.next(); return
parentFolder.createFolder(name); }
function sheetToCSV(sheet) {
  if (!sheet || sheet.getLastRow() < 1) return '';
  var data = sheet.getDataRange().getValues();
  return data.map(function(row) {
    return row.map(function(cell) {
      var s = (cell instanceof Date) ? cell.toISOString() : String(cell==null?'':cell);
      if (s.indexOf(',')>-1 || s.indexOf('"')>-1 || s.indexOf('\n')>-1) s = '"'+s.replace(/"/g,'""')+'"';
      return s;
    }).join(',');
  }).join('\n');
}
function sheetToJSON(sheet) {
  if (!sheet || sheet.getLastRow() < 2) return '[]';
  var data = sheet.getDataRange().getValues();
  var headers = data[0];
  var rows = data.slice(1).map(function(row) {
    var obj = {}; headers.forEach(function(h,i){obj[h]=(row[i] instanceof
Date)?row[i].toISOString():row[i];});
    return obj;
  });
  return JSON.stringify(rows, null, 2);
}
function cleanOldBackups(folder, maxFiles) {
  try {
    var files = folder.getFiles(); var list = [];
    while (files.hasNext()) { var f = files.next(); list.push({file:f,date:f.getDateCreated()}); }
    list.sort(function(a,b){return b.date-a.date;});
    if (list.length > maxFiles) {
      for (var i = maxFiles; i < list.length; i++) list[i].file.setTrashed(true);
    }
  } catch (e) {}
}
function backupAllSheets(username) {
  try {
    var rootFolder = getOrCreateBackupFolder();
    var ss = getSS();
    var ts = Utilities.formatDate(new Date(), Session.getScriptTimeZone(), 'yyyy-MM-dd_HH-mm-ss');
    var sheetNames = Object.keys(SH).map(function(k){return SH[k];});
    var total = 0;
    sheetNames.forEach(function(name) {
      var sh = ss.getSheetByName(name); if (!sh) return;
      var sheetFolder = getOrCreateSubFolder(rootFolder, name);
```

```
    sheetFolder.createFile(name+'_'+ts+'.json', sheetToJSON(sh), MimeType.PLAIN_TEXT);
    sheetFolder.createFile(name+'_'+ts+'.csv', sheetToCSV(sh), MimeType.CSV);
    cleanOldBackups(sheetFolder, MAX_BACKUP_FILES);
    total++;
   });
   logA(username||'system', 'BACKUP_SHEETS', total+' sheets');
   return { ok: true, msg: total+' sheet ter-backup' };
  } catch (e) { return { ok: false, msg: e.message }; }
}
function backupAppProject(username) {
  try {
   var rootFolder = getOrCreateBackupFolder();
   var appFolder = getOrCreateSubFolder(rootFolder, '_AppProject');
   var ts = Utilities.formatDate(new Date(), Session.getScriptTimeZone(), 'yyyy-MM-dd_HH-mm-ss');
   var info = { timestamp: new Date().toISOString(), creator: username||'system', appName:
APP_NAME, scriptId: ScriptApp.getScriptId(), spreadsheetId: getSS().getId(), spreadsheetUrl:
getSS().getUrl(), scriptProperties: PropertiesService.getScriptProperties().getProperties(), triggers:
ScriptApp.getProjectTriggers().map(function(t) { return { handler: t.getHandlerFunction(), eventType:
String(t.getEventType()), uniqueId: t.getUniqueId() }; }) };
   appFolder.createFile('AppProject_'+ts+'.json', JSON.stringify(info, null, 2), MimeType.PLAIN_TEXT);
   try { var ssFile = DriveApp.getFileById(getSS().getId());
ssFile.makeCopy('didiQCsys_Spreadsheet_'+ts, appFolder); } catch (e2) {}
   cleanOldBackups(appFolder, MAX_BACKUP_FILES);
   return { ok: true, msg: 'Backup aplikasi berhasil' };
  } catch (e) { return { ok: false, msg: e.message }; }
}
function backupDatabase(username, role) { try { var r1 = backupAllSheets(username); var r2 =
backupAppProject(username); return { ok: true, msg: r1.msg+' | '+r2.msg }; } catch (e) { return { ok:
false, msg: e.message }; } }
function listBackups() {
  try {
   var root = getOrCreateBackupFolder();
   var subs = root.getFolders();
   var result = {};
   while (subs.hasNext()) {
    var folder = subs.next();
    var fname = folder.getName();
    var files = folder.getFiles();
    var list = [];
    while (files.hasNext()) {
     var f = files.next();
     list.push({ id: f.getId(), name: f.getName(), date: f.getDateCreated().toISOString(), size:
f.getSize(), mimeType: f.getMimeType() });
    }
    list.sort(function(a,b){return new Date(b.date)-new Date(a.date);});
```

```
      result[fname] = list;
    }
    return { ok: true, data: result };
  } catch (e) { return { ok: false, msg: e.message }; }
}
function restoreSheetFromBackup(fileId, sheetName, callerRole) {
  return withLock(function() {
    try {
      if (callerRole !== 'superadmin') return { ok: false, msg: 'Akses ditolak' };
      var file = DriveApp.getFileById(fileId);
      var content = file.getBlob().getDataAsString();
      var fname = file.getName();
      var ss = getSS();
      var targetSheet = ss.getSheetByName(sheetName);
      if (!targetSheet) targetSheet = ss.insertSheet(sheetName);
      var rows = [];
      if (fname.toLowerCase().indexOf('.json') > -1) {
        var arr = JSON.parse(content);
        if (!Array.isArray(arr) || arr.length === 0) return { ok: false, msg: 'File JSON kosong' };
        var headers = Object.keys(arr[0]);
        rows.push(headers);
        arr.forEach(function(obj) { rows.push(headers.map(function(h){return obj[h] !== undefined ?
obj[h] : '';})); });
      } else if (fname.toLowerCase().indexOf('.csv') > -1) {
        var lines = content.split(/\r?\n/).filter(function(l){return l.trim();});
        lines.forEach(function(line) {
          var arr = [], cur = '', inQ = false;
          for (var i = 0; i < line.length; i++) {
            var c = line[i];
            if (c === '"') { if (inQ && line[i+1] === '"') { cur += '"'; i++; } else inQ = !inQ; }
            else if (c === ',' && !inQ) { arr.push(cur); cur = ''; }
            else cur += c;
          }
          arr.push(cur);
          rows.push(arr);
        });
      } else return { ok: false, msg: 'Format tidak didukung' };
      targetSheet.clearContents();
      if (rows.length) targetSheet.getRange(1, 1, rows.length, rows[0].length).setValues(rows);
      return { ok: true, msg: 'Sheet '+sheetName+' di-restore' };
    } catch (e) { return { ok: false, msg: e.message }; }
  });
}
function setupAutoBackupTrigger() {
  try {
```

```
ScriptApp.getProjectTriggers().forEach(function(t){if(t.getHandlerFunction()==='autoBackupDaily')Scr
iptApp.deleteTrigger(t);});
  ScriptApp.newTrigger('autoBackupDaily').timeBased().everyDays(1).atHour(2).create();
  return { ok: true };
  } catch(e){}
}
function removeAutoBackupTrigger() {
  try {

ScriptApp.getProjectTriggers().forEach(function(t){if(t.getHandlerFunction()==='autoBackupDaily')Scr
iptApp.deleteTrigger(t);});
  return { ok: true };
  } catch(e){ return { ok: false, msg: e.message }; }
}
function isBackupTriggerActive(){
  try{
    var active=false;
    ScriptApp.getProjectTriggers().forEach(function(t){
      if(t.getHandlerFunction()==='autoBackupDaily')active=true;
    });
    return active;
  }catch(e){return false;}
}
function autoBackupDaily() { if (getSetting('backup_auto') === 'true') { backupAllSheets('system');
backupAppProject('system'); } }
function exportSheet(sheetName, format) {
  try {
    var sh = S(sheetName);
    if (!sh) return { ok: false, msg: 'Sheet tidak ditemukan' };
    if (format === 'json') return { ok: true, content: sheetToJSON(sh), filename: sheetName+'.json',
mime: 'application/json' };
    return { ok: true, content: sheetToCSV(sh), filename: sheetName+'.csv', mime: 'text/csv' };
  } catch (e) { return { ok: false, msg: e.message }; }
}
function getEmailQuota() { try { return { ok: true, remaining: MailApp.getRemainingDailyQuota() }; }
catch (e) { return { ok: false, msg: e.message, remaining: 0 }; } }
function testEmail(toEmail, username) {
  try {
    MailApp.sendEmail({ to: toEmail, subject: '[didiQCsys] Test Email', body: 'Halo '+username });
    return { ok: true, msg: 'Email terkirim' };
  } catch (e) { return { ok: false, msg: e.message }; }
}
function resetDatabase(username, role, confirm) {
  return withLock(function() {
```

```
  if (role !== 'superadmin') return { ok: false, msg: 'Akses ditolak' };
  if (confirm !== 'RESET CONFIRM') return { ok: false, msg: 'Konfirmasi tidak valid' };
  var ss = getSS();
  var clearSheets = [SH.INPUTQC, SH.HISTORI, SH.CALCSTATS, SH.BIASPME, SH.LAPORAN,
SH.TABULASI, SH.LOGACTIVITY];
  clearSheets.forEach(function(name) {
   var sh = ss.getSheetByName(name);
   if (sh && sh.getLastRow() > 1) sh.deleteRows(2, sh.getLastRow()-1);
  });
  return { ok: true, msg: 'Database direset' };
 });
}

// ── IMAGE ANALYSIS ──────────────────────────────────────────
function getImgSheetName(type) { var
map={hemato:SH.IMGHEMATO,urin:SH.IMGURIN,malaria:SH.IMGMALARIA,bta:SH.IMGBTA,lain:SH.I
MGLAIN,patologi:SH.IMGPATOLOGI}; return map[type]||null;}
function getImgData(type, ownerUsername, filter) {
 try {
 filter = filter || {};
 var shName = getImgSheetName(type);
 if (!shName) return { ok: false, msg: 'Tipe tidak valid' };
 var sh = S(shName);
 if (!sh || sh.getLastRow() < 2) return { ok: true, data: [] };
 var hdrs = sh.getRange(1,1,1,sh.getLastColumn()).getValues()[0];
 var data = sh.getRange(2,1,sh.getLastRow()-1,sh.getLastColumn()).getValues();
 var ownerIdx = hdrs.indexOf('OwnerUsername');
 // FIX v9.8.1: Deteksi kolom tanggal & nama sesuai tipe (patologi vs lainnya)
 var tglIdx = hdrs.indexOf('TglPeriksa');
 if (tglIdx < 0) tglIdx = hdrs.indexOf('TglTerima');
 var namaIdx = hdrs.indexOf('Nama');
 if (namaIdx < 0) namaIdx = hdrs.indexOf('NamaPasien');
 var normIdx = hdrs.indexOf('NoRM');
 var paIdx = hdrs.indexOf('NoPA');
 var filtered = data.filter(function(r) {
  if (ownerIdx < 0 || String(r[ownerIdx]).toLowerCase() !== String(ownerUsername).toLowerCase())
return false;
  if (filter.noRM && normIdx >= 0 &&
String(r[normIdx]).toLowerCase().indexOf(String(filter.noRM).toLowerCase()) < 0) return false;
  if (filter.nama && namaIdx >= 0 &&
String(r[namaIdx]).toLowerCase().indexOf(String(filter.nama).toLowerCase()) < 0) return false;
  if (filter.noPA && paIdx >= 0 &&
String(r[paIdx]).toLowerCase().indexOf(String(filter.noPA).toLowerCase()) < 0) return false;
  if (filter.startDate && tglIdx >= 0) { var d = parseDateStr(r[tglIdx]); if (!d || d <
parseDateStr(filter.startDate)) return false; }
```

```
   if (filter.endDate && tglIdx >= 0) { var d2 = parseDateStr(r[tglIdx]); if (!d2 || d2 >
parseDateStr(filter.endDate)) return false; }
   return true;
  });
  return { ok: true, data: filtered.map(function(r) {
   var obj = {}; hdrs.forEach(function(h,i){obj[h] = r[i];});
   // FIX v9.8.1: Tambah TglTerima & TglJawab untuk patologi
   ['TglLahir','TglPeriksa','TglHasil','TglTerima','TglJawab','CreatedDate'].forEach(function(k) {
    if (obj[k] instanceof Date) obj[k] = dateToISO(obj[k]);
    else if (obj[k]) obj[k] = dateToISO(parseDateStr(obj[k]));
   });
   return obj;
  })};
  } catch(e){return {ok:false,msg:e.message};}
}
function saveImgData(type, payload, ownerUsername) {
  return withLock(function() {
   try {
    var shName = getImgSheetName(type); if (!shName) return { ok: false, msg: 'Tipe tidak valid' };
    var sh = S(shName);
    var hdrs = sh.getRange(1,1,1,sh.getLastColumn()).getValues()[0];
    var idIdx = hdrs.indexOf('ID'), ownerIdx = hdrs.indexOf('OwnerUsername');
    if (payload.ID) {
     var data = sh.getDataRange().getValues();
     for (var i = 1; i < data.length; i++) {
      if (data[i][idIdx] === payload.ID && String(data[i][ownerIdx]).toLowerCase() ===
String(ownerUsername).toLowerCase()) {
       var row = buildImgRow(hdrs, payload, ownerUsername, false,
data[i][hdrs.indexOf('CreatedDate')]||new Date());
       sh.getRange(i+1,1,1,row.length).setValues([row]);
       return { ok: true, id: payload.ID };
      }
     }
     return { ok: false, msg: 'Data tidak ditemukan' };
    } else {
     payload.ID = genID('IMG');
     var newRow = buildImgRow(hdrs, payload, ownerUsername, true, new Date());
     sh.appendRow(newRow);
     return { ok: true, id: payload.ID };
    }
   } catch (e) { return { ok: false, msg: e.message }; }
  });
}
function buildImgRow(hdrs, payload, ownerUsername, isNew, createdDate) {
  return hdrs.map(function(h) {
```

```
   if (h === 'OwnerUsername') return ownerUsername;
   if (h === 'CreatedDate') return isNew ? new Date() : (createdDate||new Date());
   // FIX v9.8.1: Tambah TglTerima & TglJawab untuk patologi
   if (['TglLahir','TglPeriksa','TglHasil','TglTerima','TglJawab'].indexOf(h) > -1) return payload[h] ? new
Date(payload[h]) : '';
   return payload[h] !== undefined ? payload[h] : '';
  });
}
function deleteImgData(type, id, ownerUsername) {
  return withLock(function() {
   var shName = getImgSheetName(type); if (!shName) return { ok: false, msg: 'Tipe tidak valid' };
   var sh = S(shName);
   var hdrs = sh.getRange(1,1,1,sh.getLastColumn()).getValues()[0];
   var idIdx = hdrs.indexOf('ID'), ownerIdx = hdrs.indexOf('OwnerUsername');
   var data = sh.getDataRange().getValues();
   for (var i = data.length-1; i >= 1; i--) {
    if (data[i][idIdx] === id && String(data[i][ownerIdx]).toLowerCase() ===
String(ownerUsername).toLowerCase()) {
      sh.deleteRow(i+1);
      return { ok: true };
    }
   }
   return { ok: false, msg: 'Data tidak ditemukan' };
  });
}
function getImgHemato(o,f){return getImgData('hemato',o,f);}
function saveImgHemato(p,u){return saveImgData('hemato',p,u);}
function deleteImgHemato(id,u){return deleteImgData('hemato',id,u);}
function getImgUrin(o,f){return getImgData('urin',o,f);}
function saveImgUrin(p,u){return saveImgData('urin',p,u);}
function deleteImgUrin(id,u){return deleteImgData('urin',id,u);}
function getImgMalaria(o,f){return getImgData('malaria',o,f);}
function saveImgMalaria(p,u){return saveImgData('malaria',p,u);}
function deleteImgMalaria(id,u){return deleteImgData('malaria',id,u);}
function getImgBTA(o,f){return getImgData('bta',o,f);}
function saveImgBTA(p,u){return saveImgData('bta',p,u);}
function deleteImgBTA(id,u){return deleteImgData('bta',id,u);}
function getImgLain(o,f){return getImgData('lain',o,f);}
function saveImgLain(p,u){return saveImgData('lain',p,u);}
function deleteImgLain(id,u){return deleteImgData('lain',id,u);}
function getImgPatologi(o,f){try{return getImgData('patologi',o,f);}catch(e){return
{ok:false,msg:e.message};}}
function saveImgPatologi(p,u){try{return saveImgData('patologi',p,u);}catch(e){return
{ok:false,msg:e.message};}}
```

```
function deleteImgPatologi(id,u){try{return deleteImgData('patologi',id,u);}catch(e){return
{ok:false,msg:e.message};}}

// ── INIT DATA ──────────────────────────────────────────────────
function getInitData(ownerUsername, role, actualRole) {
try {
  // PERBAIKAN UTAMA: Gunakan ownerUsername sebagai pemilik data yang sebenarnya
  var params = getParameters(ownerUsername, role);
  var lots = getLotQC(ownerUsername, role);
  var teaList = getDaftarTEa(ownerUsername, role);
  var settings = getSettings();
  var kop = getKopSurat(ownerUsername);
  var user = getUserByUsername(ownerUsername);

  // Hanya superadmin yang bisa melihat daftar semua user
  var isSA = (actualRole === 'superadmin' || role === 'superadmin');
  var allUsers = isSA ? (getUsers(ownerUsername, 'superadmin').data || []) : [];

  var ss = getSS();
  var archiveYears = [];
  ss.getSheets().forEach(function(sh) {
    var name = sh.getName();
    if (/^InputQC_\d{4}$/.test(name)) archiveYears.push(parseInt(name.replace('InputQC_','')));
  });
  archiveYears.sort(function(a,b){return b-a;});

  return sanitizeReturn({
    ok: true,
    params: params,
    lots: lots,
    teaList: teaList,
    settings: settings,
    kop: kop,
    archiveYears: archiveYears,
    appLogo: APP_LOGO_URL,
    userInfo: {
      username: ownerUsername,
      role: role,
      fullName: user ? user.fullName : '',
      email: user ? user.email : '',
      imgAnalAccess: user ? user.imgAnalAccess : false
    },
    allUsers: allUsers,
    backupTriggerActive: isBackupTriggerActive()
  });
```

```
} catch (e) {
   return { ok: false, msg: e.message };
}
}
function getReportData(ownerUsername, role, filter) {
try {
filter = filter || {};
var graphData = getGraphData({
paramID: filter.paramID, lotID: filter.lotID,
startDate: filter.startDate, endDate: filter.endDate,
sumber: filter.sumber || 'Manufaktur'
}, ownerUsername, role);
var catatan = getCatatanLaporan(ownerUsername, filter.filterKey || '');
var kop = getKopSurat(ownerUsername);
return sanitizeReturn({ ok: true, graphData: graphData, catatan: catatan, kop: kop });
} catch (e) { return { ok: false, msg: e.message }; }
}

function getDashboardDetailTrend(ownerUsername, role) {
try { return getDashboardAnalisisTrend(ownerUsername, role, { months: 6 }); }
catch (e) { return { ok: false, msg: e.message }; }
}

function getAppLogo() { return APP_LOGO_URL; }

// ── UPLOAD IMAGE TO DRIVE (v9.8) ──────────────────────────
function uploadImgToDrive(base64Data, fileName, type) {
 try {
   var folderName = 'didiQCsys_Images_' + type;
   var folders = DriveApp.getFoldersByName(folderName);
   var folder = folders.hasNext() ? folders.next() : DriveApp.createFolder(folderName);
   var decoded = Utilities.base64Decode(base64Data.split(',')[1] || base64Data);
   var blob = Utilities.newBlob(decoded, 'image/jpeg', fileName || ('img_' + new Date().getTime() +
'.jpg'));
   var file = folder.createFile(blob);
   file.setSharing(DriveApp.Access.ANYONE, DriveApp.Permission.VIEW);
   return file.getDownloadUrl().replace('export=download', 'export=view');
 } catch (e) { return null; }
}

// ── ANALYZE PATOLOGI IMAGE (v9.8) ─────────────────────────
function analyzePatologiImage(imageUrls, jenisPemeriksaan, patientData) {
 try {
   var result = {
     makroskopisDesk: '', mikroskopisDesk: '', kesanDesk: '',
```

```
    saranDesk: '', topografiDesk: '', morfologiDesk: ''
  };
  var jp = (jenisPemeriksaan || '').toLowerCase();
  var nama = (patientData && patientData.NamaPasien) ? patientData.NamaPasien : '';
  var jk = (patientData && patientData.JK) ? patientData.JK : '';
  var umur = (patientData && patientData.Umur) ? patientData.Umur : '';
  var dx = (patientData && patientData.Diagnosis) ? patientData.Diagnosis : '';

  if (jp.indexOf('histologi') >= 0) {
    result.makroskopisDesk = 'Diterima spesimen jaringan berupa potongan jaringan yang difiksasi dalam formalin 10% berwarna coklat keabu-abuan, berukuran bervariasi. Jaringan tampak padat, konsistensi cukup keras, dengan permukaan potongan agak kasar. ' + (dx ? 'Klinis: ' + dx + '.' : '');
    result.mikroskopisDesk = 'Potongan histologis menunjukkan arsitektur jaringan yang dipreservasi dengan baik. Evaluasi terhadap struktur seluler, pola pertumbuhan, dan karakteristik nukleus perlu dilakukan lebih lanjut oleh patologis anatomi.\n\nCatatan: Analisis AI ini merupakan pendukung awal. Diagnosis definitif harus ditetapkan oleh dokter patologi anatomi yang kompeten sesuai standar WHO Classification of Tumors.';
    result.kesanDesk = 'Spesimen memerlukan evaluasi mikroskopis lengkap oleh patologis anatomi. Diagnosis definitif belum dapat ditetapkan berdasarkan gambar saja dan memerlukan pemeriksaan imunohistokimia apabila diperlukan.';
    result.saranDesk = '1. Disarankan pemeriksaan imunohistokimia untuk menegakkan diagnosis\n2. Konsultasi dengan patologis anatomi senior apabila diperlukan\n3. Korelasi dengan data klinis, hasil pemeriksaan penunjang, dan temuan radiologis\n4. Pertimbangkan pemeriksaan molekuler apabila indikasi terpenuhi';
    result.topografiDesk = 'Topografi spesimen perlu dikonfirmasi oleh klinisi. Klasifikasi ICD-O topografi dan morfologi sesuai standar WHO.';
    result.morfologiDesk = 'Morfologi tumor/jaringan perlu ditentukan melalui evaluasi histopatologis lengkap sesuai klasifikasi WHO terbaru.';
  } else if (jp.indexOf('sitologi') >= 0) {
    result.makroskopisDesk = 'Diterima spesimen sitologi berupa smear/ preparat apus yang diwarnai dengan pewarnaan yang sesuai. Evaluasi seluler dilakukan secara menyeluruh.';
    result.mikroskopisDesk = 'Preparat sitologi menunjukkan kecukupan seluler. Evaluasi morfologi sel, nukleus, sitoplasma, dan latar belakang perlu dilakukan lebih detail oleh patologis anatomi.\n\nCatatan: Hasil sitologi merupakan screening awal dan diagnosis definitif memerlukan konfirmasi histopatologi.';
    result.kesanDesk = 'Spesimen sitologi memerlukan evaluasi lengkap oleh patologis anatomi. Hasil bersifat preliminer dan memerlukan korelasi klinis.';
    result.saranDesk = '1. Korelasi dengan data klinis\n2. Pertimbangkan biopsi untuk konfirmasi histopatologi\n3. Follow-up dan evaluasi berkala\n4. Konsultasi patologis anatomi apabila temuan tidak khas';
    result.topografiDesk = 'Topografi sesuai lokasi pengambilan spesimen oleh klinisi.';
    result.morfologiDesk = 'Morfologi sel perlu dievaluasi lebih lanjut sesuai klasifikasi WHO.';
  } else if (jp.indexOf('papsmear') >= 0 || jp.indexOf('pap') >= 0) {
    result.makroskopisDesk = 'Diterima preparat Pap Smear. Evaluasi dilakukan sesuai sistem pelaporan Bethesda 2014.';
```

```
    result.mikroskopisDesk = 'Evaluasi preparat Pap Smear menunjukkan:\n- Kejelasan dan kecukupan seluler perlu dinilai\n- Komponen sel epitel skuamosa dan/atau glandular perlu dievaluasi\n- Latar belakang inflamasi dan flora bakteri perlu dinilai\n\nCatatan: Interpretasi akhir harus dilakukan oleh patologis anatomi sesuai The Bethesda System for Reporting Cervical Cytology (TBS 2014).';
    result.kesanDesk = 'Preparat memerlukan evaluasi lengkap oleh patologis anatomi untuk klasifikasi Bethesda (NILM, ASC-US, ASC-H, LSIL, HSIL, atau karsinoma invasif).';
    result.saranDesk = '1. Pap Smear berulang apabila preparat tidak memadai\n2. Kolposkopi apabila terdapat abnormitas sel epitel\n3. HPV DNA testing apabila indikasi\n4. Follow-up sesuai protokol screening serviks';
    result.topografiDesk = 'Serviks uteri - Transformation zone.';
    result.morfologiDesk = 'Morfologi sel epitel serviks sesuai klasifikasi Bethesda 2014.';
   } else if (jp.indexOf('fnab') >= 0 || jp.indexOf('fnac') >= 0) {
    result.makroskopisDesk = 'Diterima spesimen FNAB/FNAC berupa preparat smear dari aspirasi jarum halus. Jumlah dan kecukupan materi seluler perlu dievaluasi.';
    result.mikroskopisDesk = 'Evaluasi FNAB menunjukkan kecukupan materi seluler. Pola seluler, karakteristik nukleus, dan arsitektur sel perlu dinilai oleh patologis anatomi.\n\nCatatan: Diagnosis FNAB bersifat preliminer. Konfirmasi histopatologi melalui biopsi eksisi/insisional diperlukan untuk diagnosis definitif.';
    result.kesanDesk = 'Spesimen FNAB memerlukan evaluasi lengkap oleh patologis anatomi. Kecukupan materi dan diagnosis banding perlu ditetapkan.';
    result.saranDesk = '1. Korelasi dengan pemeriksaan klinis dan pencitraan\n2. Pertimbangkan biopsi untuk konfirmasi histopatologi\n3. Pemeriksaan imunositokimia pada cell block apabila diperlukan\n4. Konsultasi patologis anatomi untuk kasus yang sulit';
    result.topografiDesk = 'Topografi sesuai lokasi aspirasi oleh klinisi.';
    result.morfologiDesk = 'Morfologi sel aspirasi perlu diklasifikasikan sesuai standar WHO.';
   } else {
    result.makroskopisDesk = 'Diterima spesimen patologi anatomi. Evaluasi makroskopis dilakukan terhadap ukuran, bentuk, warna, dan konsistensi spesimen.';
    result.mikroskopisDesk = 'Evaluasi mikroskopis diperlukan oleh patologis anatomi untuk menentukan diagnosis.';
    result.kesanDesk = 'Spesimen memerlukan evaluasi lengkap oleh patologis anatomi.';
    result.saranDesk = 'Konsultasi dengan patologis anatomi untuk evaluasi dan diagnosis definitif.';
    result.topografiDesk = 'Topografi sesuai data klinis.';
    result.morfologiDesk = 'Morfologi perlu dievaluasi lebih lanjut.';
   }

   if (nama) {
    result.makroskopisDesk = 'Pasien: ' + nama + (jk ? ' (' + jk + ')' : '') + (umur ? ', ' + umur : '') + '. ' + result.makroskopisDesk;
   }
   return { ok: true, data: result };
  } catch (e) { return { ok: false, msg: e.message }; }
}
```